



Project:	ISOLDE: High Performance, Safe, Secure, Open-Source Leveraged RISC-V Domain-Specific Ecosystems
Reference number:	101112274
Project duration:	01.05.2023 - 31.10.2026
Work Package:	WP3: Accelerators and Extensions
Deliverable	**D3.4**
Title	**Final implementation for Safety & Security Modules**
Type of deliverable	Report
Deadline	31.07.2026
Creation Date	09.10.2025
Dissemination Level	ISOLDE - public

Authors	Marcus Borrmann, Erik Kraft, Marcel Medwed - NXP-AT Daniel Gracia Pérez, Sylvain Girbal - TRT Stefan Wallentowitz - HM Holger Blasum, Darshak Sheladiya - SYSGO Jaume Abella, Francisco Fuentes, Sergi Alcaide, Francisco J. Cazorla, Ramon Canal, Feng Chang, Juan Carlos Rodríguez, Juan Antonio Rodríguez, Xavier Carril - BSC Esther Soriano, Daniel Cebrián - FENTISS Wolfgang Ecker, Endri Kaja, Ares Tahiraga, Jad Al Halabi - IFX Mladen Berekovic, Rainer Buchty, Amrit Sharma Poudel, Nguyen-Hoang-Hai Pham, Saleh Mulhem - UzL Angelo Garofalo - UNIBO Andrea Galimberti, D.Zoni, W. Fornaciari - POLIMI Cătălin Ciobanu - IMT Sven Mehlhop - OFFIS
Involved grant recipients:	NXP Semiconductors Austria GmbH & Co KG (NXP-AT) Thales SA as Thales Research & Technology (TRT) Hochschule München University of Applied Sciences (HM) SYSGO GmbH (SYSGO) Barcelona Supercomputing Center (BSC) Fent Innovative Software Solutions S.L. (FENTISS)

Infineon Technologies AG (IFX)
Universität zu Lübeck (UzL)
Universita di Bologna (UNIBO)
Politecnico di Milano (POLIMI)
Institutul National de Cercetare-Dezvoltare Pentru Microtehnologie (IMT)
OFFIS e.V. (OFFIS)

Coordinator: Darshak Sheladiya, SYSGO, darshak.sheladiya@sysgo.com
Reviewers: Erik Kraft, NXP
Sven Mehlhop, OFFIS
Jaume Abella, BSC
Amrit Sharma Poudel, UzL

Table of Contents

1 Executive Summary **2**

2 Introduction **3**
2.1 General Information . . . 3
2.2 Purpose and Scope . . . 3

3 Safety and Security extensions **5**
3.1 Safety and Security Modules . . . 5
3.1.1 Inline Encryption Engine (IEE) – NXP-AT . . . 7
3.1.2 Backward-Edge Control Flow Integrity (BCFI) – NXP-AT . . . 18
3.1.3 Cryptographically Tagged Memory (CTM) – NXP-AT . . . 30
3.1.4 Enclave Memory Isolation (EMI) – NXP-AT . . . 41
3.1.5 Forward-Edge Control Flow Integrity (FCFI) – NXP-AT . . . 53
3.1.6 Context-Aware Performance Monitor Counter (CA-PMC) – TRT . . . 67
3.1.7 Memory Subsystem Support for Bytecode VMs – HM . . . 80
3.1.8 Safety-Related Traffic Injector (SafeTI) – BSC . . . 88
3.1.9 Safety and Security Control Unit (SSCU) – IFX . . . 97
3.1.10 Safety Island - Interface Definition – UzL . . . 111
3.1.11 Root-of-Trust Unit (RoT) – UNIBO . . . 112
3.1.12 High-Performance Cache Analysis (CACHE) – SYSGO . . . 118
3.2 Monitoring Infrastructure . . . 124
3.2.1 Context-Aware PMC Interface (CA-PMC-IF) – TRT . . . 126
3.2.2 Run-Time Power Monitoring Instrumentation (RTPM) – POLIMI . . . 135
3.2.3 Safety-Related Statistics Unit (SafeSU) – BSC . . . 143
3.2.4 Time Contract Monitoring Co-Processor (TCCP) – OFFIS . . . 157

4 Conclusion **164**

1 Executive Summary

This document refines the architecture and describes the prototype implementations of the ISOLDE project hardware modules related to safety and security. These modules have been developed within the WP3 *“Accelerators and Extensions"* work package of the ISOLDE project, more particularly in tasks T3.1 *“Safety & Security Modules”* and T3.3 *“Monitoring infrastructure”*. Earlier versions of the hardware modules had previously been described in deliverables D3.1 *“Initial architecture definition”* and D3.2, *“Safety & Security modules prototype implementation”*.

The main contribution of this deliverable lies in the comprehensive evaluation of the developed IPs and state-of-the-art comparison. While D3.2 reported preliminary implementation and performance results, the current document presents a substantially more mature assessment based on implementations and evaluations. The evaluation methodology is described in detail and provides implementation results tailored to the characteristics of each IP, including metrics such as performance, area, power consumption, and energy efficiency, where applicable. In addition, comparisons with state-of-the-art solutions reported in the literature are included whenever relevant and available, providing further context and positioning the developed IPs with respect to existing approaches.

2 Introduction

2.1 General Information

Work Package 3 (WP3) focuses on developing hardware modules and extensions enhancing RISC-V systems based on the foundational cores provided by WP2 to create and demonstrate high-performance computing systems within WP5. The objective of WP3 is to improve system-level performance and efficiency through specialized hardware extensions that complement general-purpose processing capabilities. The activities carried out within this work package encompass the full development flow, from architectural definition to implementation and validation of synthesizable IP blocks.

Deliverable D3.4, titled "*Final implementation for Safety & Security Modules*", refines the architecture description of each safety/security hardware module presented in Deliverables D3.1 and D3.2, provides updates to implementation details from D3.2, and also results on evaluation for each module.

2.2 Purpose and Scope

The purpose of this deliverable is to present the final implementation results and comprehensive evaluation of the safety and security modules in WP3 of the ISOLDE project. It documents the progression from the prototype implementations reported in D3.2 to fully implemented, synthesizable IP blocks, incorporating design refinements and validation outcomes obtained during the final development phase. The description of each module follows the same structured presentation approach adopted in D3.2, while providing updated and extended information reflecting the developments achieved since that deliverable. Each module is documented through a structured IP Card, which provides a concise and standardized summary of its main features, including interface definition, system integration considerations, and key implementation parameters, ensuring consistency and traceability with respect to the previous deliverable. The main focus of this deliverable, however, is the evaluation and quantitative assessment of the developed modules. The evaluation is conducted using FPGA implementations and ASIC design flows, reporting key metrics such as performance and silicon area, and where applicable additional indicators including power consumption and energy efficiency. In order to ensure a coherent and comparable assessment across heterogeneous application domains, performance improvements are reported with respect to representative Arm- and RISC-V-based baseline implementations, which serve as common reference points across WP3 activities. Where relevant, additional comparisons with state-of-the-art solutions reported in the scientific and technical literature are provided to further contextualize the achieved results. In addition, achieved results are compared to the respective requirements of each module as defined in deliverable D1.4 together with an indication if the designed IP is Compliant (C), Partially Compliant (PC), or Non-Compliant (NC). This deliverable together with D3.5 represents the final reference documents for WP3 outcomes and consolidates the contributions of all participating partners, ensuring a unified presentation of results across the different technical developments.

Section 3 reports work done in Task *"T3.1: Safety & Security Modules"* (Section 3.1), and Task

"*T3.3: Monitoring Infrastructure*" (Section 3.2). Both subsections start with an overview and then contain in-depth technical information about each specific building block, providing:

- An **IP card** [27] of the hardware module summarizing technical information.
- **General information** about the context the hardware module is used in, providing a scope beyond the specific module itself.
- The **purpose and scope** of this specific hardware module, providing a brief high-level description of the module purpose.
- A (refined) **architecture description** with updates since the initial architecture description in deliverables D3.1 and D3.2, and covering its placement in the system with block diagrams.
- A list of both control and data **interfaces** to the other IPs this specific hardware component is connected to, as well as dependencies to existing IPs.
- A description of any **instruction set architecture (ISA) specialization** associated with the hardware module, as well as potential hardware abstraction layers (HAL) or high-level API.
- Report on **evaluation prototype** that reports the implementation and developmental progress related to the safety/security module, as well as strategies for clocking, resetting, power management, verification, and debugging, possibly including a test-bench prototype and early results prior to the integration to WP5 use cases.
- A **start-of-the-art comparison**, e.g., with Arm.
- A **compliance assessment** with requirements previously stated in deliverable D1.4. "*C*" means "*compliant*", "*PC*" means "*partially compliant*", "*NC*" means "*not compliant*".

3 Safety and Security extensions

3.1 Safety and Security Modules

Task 3.1 has focused on the development of technologies supporting safety and security. The developed safety and security modules include components for memory encryption, control flow integrity (CFI), memory isolation, hardware support for bytecode virtual machine interpreters, and hardware root of trust.

IP	Lead Partner	Domain	Dependencies	Licensing
IEE (3.1.1)	NXP-AT	Security	None	Proprietary closed source
BCFI (3.1.2)	NXP-AT	Security	RV32I processor, IEE (3.1.1)	Proprietary closed source
CTM (3.1.3)	NXP-AT	Security	RV32I processor, IEE (3.1.1)	Proprietary closed source
EMI (3.1.4)	NXP-AT	Security	RV32I processor, IEE (3.1.1)	Proprietary closed source
FCFI (3.1.5)	NXP-AT	Security	RV32I processor	Proprietary closed source
CA-PMC (3.1.6)	TRT	Safety	CA-PMC-IF (3.2.1)	Proprietary closed source
Memory support for Bytecode VMs (3.1.7)	HM	Security	None	Permissive open source
SafeTI (3.1.8)	BSC	Safety	None	Permissive open source
Safety & Security Control Unit (3.1.9)	IFX	Safety, Security	None	Permissive open source
Safety Island (3.1.10)	UZL	Safety	N/A (not developed finally)	N/A
RoT (3.1.11)	UNIBO	Security	CVA6, OpenTitan	Permissive open source
High-Perf. Cache Analysis (3.1.12)	SYSGO	Security	CVA6, CEA cache (Tristan)	N/A

Table 3.1: Overview of Task 3.1 contributions

In general, the solutions provided are either (i) the first realization in the context of RISC-V, (ii) provide different subsets of features than existing alternatives, hence fitting different objectives, or (iii) provide similar or more powerful features with comparable or lower cost, while being open

source in some cases.

In more detail, each module comparison against state-of-the-art is as follows:

- The proposed Inline Encryption Engine (IEE) achieves lower latency and area than ciphers operating on the same or smaller block bits.
- In the context of target deeply embedded 32-bit processors without MMUs, the proposed Backward-Edge Control Flow Integrity (BCFI) provides stronger protection against both replay and physical attacks while also requiring a smaller memory footprint than existing comparable solutions.
- The proposed Cryptographically Tagged Memory (CTM) is, to our knowledge, the first one targeting deeply embedded 32-bit processors without MMUs and achieves higher security levels and protection with lower cost than existing solutions.
- The proposed Enclave Memory Isolation (EMI) targets deeply embedded 32-bit processors without MMUs. There is no comparable Arm solution, and RISC-V solutions are not yet fully developed.
- The proposed Forward-Edge Control Flow Integrity (FCFI) offers heterogeneous security guarantees and features compared to state-of-the-art solutions, which, in essence, do not match the requirements of FCFI in its context.
- The proposed Context-Aware Performance Monitor Counter (CA-PMC) provides features also provided by Arm alternatives, but with a much simpler unit, and enables multiple entities to manage monitoring. The CA-PMC and SafeSU for RISC-V also presented in this document provide complementary features, with the CA-PMC focusing on per-context monitoring of any type of event and the SafeSU on the monitoring and quota control of multicore events and combinations of events.
- The Memory Subsystem Support for Bytecode VMs provides acceleration specifically for the operand-stack traffic that is common to all stack-based bytecode formats and leaves decode and dispatch in software, which makes it more widely usable than existing solutions in the field.
- The proposed SafeTI provides more flexibility to inject traffic than comparable AMD/Xilinx alternatives, and on top is the only one open source and allowed to be deployed in ASIC devices.
- The proposed Safety and Security Control Unit provides, in principle, similar features to those of Arm solutions. However, Arm solutions are proprietary, so no direct comparison or adoption for RISC-V is possible for those existing solutions.
- The proposed Root-of-Trust Unit (RoT) is an extension of lowRISC's OpenTitan Earl Grey architecture, which is refactored and extended to turn it into a silicon-secure element easily integrable within larger designs.
- The HPDcache integrated in CVA6 shows to be effective at managing multiple pending misses in parallel, with comparable performance trends to those of Arm Cortex-A cores.

3.1.1 Inline Encryption Engine (IEE) – NXP-AT

3.1.1.1 IP Card

Basic Info		
IP name	Inline Encryption Engine (IEE)	
License	Closed-source proprietary	
Repository	N/A	
Architecture		
Clock	Number of clock domains	1
	Synchronous with system	Y
	Clock generated internally	N
Ctrl Interface	ISA extension?	Y (RV32I)
	Memory mapped?	N
	Protocol	N/A
	Address Map	N/A
Initiator Interface	Protocol	OBI, AHB
	Cached?	depends on integration platform
	IOMMU?	
Interrupts	Interrupts	N
Microarchitecture		
Parametrization	Parametric no. cores?	N
	Parameteric config?	Y
Programmability	Contains programmable cores?	N
	ISA	N/A
Software		
Compiler	Requires specialized compiler?	N
	Compiler repository	N/A
Hardware Abstraction Layer	N	
High-level API	Is there a high-level API/SDK?	N
	SDK repository	N/A
	Is there a domain-specific compiler?	N
Integration		
IP Distribution	Manifest type (if any)	N
	Standalone simulation?	N
	(if standalone sim) SW requirements?	N/A
	Integration documented / examples?	Y, closed-source proprietary
Synthesis	Is the IP synthesizable?	Y
	FPGA synthesis example available?	Y, closed-source proprietary
	ASIC synthesis example available?	Y, closed-source proprietary
Simulation	Closed-source simulation?	Y (Xcelium)
	Open-source simulation?	N
Evaluation	PPA results available?	Y

3.1.1.2 General Information

Traditionally, isolation of different workloads and their associated sensitive assets is achieved using the processor's privilege mode and logical memory isolation (e.g., using memory protection or management units). The logical memory isolation primitive limits access triggered by specific privilege modes and is configured by trusted software running in a higher privilege mode, for example, the firmware running in machine mode or the operating system running in supervisor mode. However, research has shown that this logical isolation is insufficient against a range of physical attack techniques (e.g., probing the memory bus, injecting faults flipping bits in memory [70, 47], or injecting faults affecting the memory protection configuration by the operating system [56]).

Adding a memory encryption engine to the system, which encrypts all data before it reaches the external bus, helps mitigate some of these attacks. The resulting ciphertexts usually depend on the physical address (used as a tweak) in addition to the secret key to mitigate attacks exchanging encrypted data words. If a pure encryption scheme is used as a primitive for memory encryption, then the confidentiality of data on the external bus and in memory is protected, and controlled modifications are harder to achieve. If verification of the data integrity is required, an authenticated encryption scheme must be used, but then also the memory overhead will be higher. Memory encryption engines help mitigate many physical attacks, but they also impose considerable area and latency overhead, while not increasing resilience against logical attacks.

As this deliverable provides updated information about the IP state since deliverable D3.2, please refer to the earlier deliverables for additional information about the Inline Encryption Engine (IEE) module.

3.1.1.3 Purpose and Scope

The IEE module adds a tweakable memory encryption engine based on a low-latency cipher to the base core. The difference to the schemes mentioned in the previous section is that we use a tweak input to make the resulting ciphertext depend on additional metadata. This metadata is provided by other extensions contributed by NXP-AT, which add defense mechanisms against various logical attacks. For more information on these extensions, please refer to the description of the Backward-Edge Control Flow Integrity (BCFI; see Section 3.1.2), Cryptographically Tagged Memory (CTM; see Section 3.1.3) and Enclave Memory Isolation (EMI; see Section 3.1.4) extension provided by NXP-AT. Systems that already include a tweakable memory encryption engine especially benefit from this approach, as adding the mentioned defenses against logical attacks comes at little additional cost.

3.1.1.4 Refined architecture description

The general architecture of the IEE extension remains unchanged compared to the deliverable D3.2 and is illustrated in Figure 3.1. Elements of the CV32E40S core affected by IEE integration are highlighted in orange. The following components have been updated since the deliverable D3.2:

- The access privilege level was removed from the tweak for memory encryption and decryption. The NXP-AT EMI extension should be used to isolate different software contexts. Using

the access privilege level in addition leads to duplication and increased software complexity (e.g., copying to and from user memory).

- The 32-bit scrambling/unscrambling primitive developed in ISOLDE was further refined and its security analyzed in follow-up work together with Graz University of Technology. The outcome of this follow-up work is BEANIE, a 32-bit block cipher designed to mitigate logical attacks in microcontrollers. The BEANIE tweak-key schedule replaces the previously used KeystreamGen module in both the Instruction IEE and Data IEE module, while the BEANIE encryption/decryption datapath replaces the earlier scrambling/unscrambling module. More information about BEANIE, including a comparison with similar designs, is provided in Section 3.1.1.8 and in the paper by Gerhalter et al. [35].

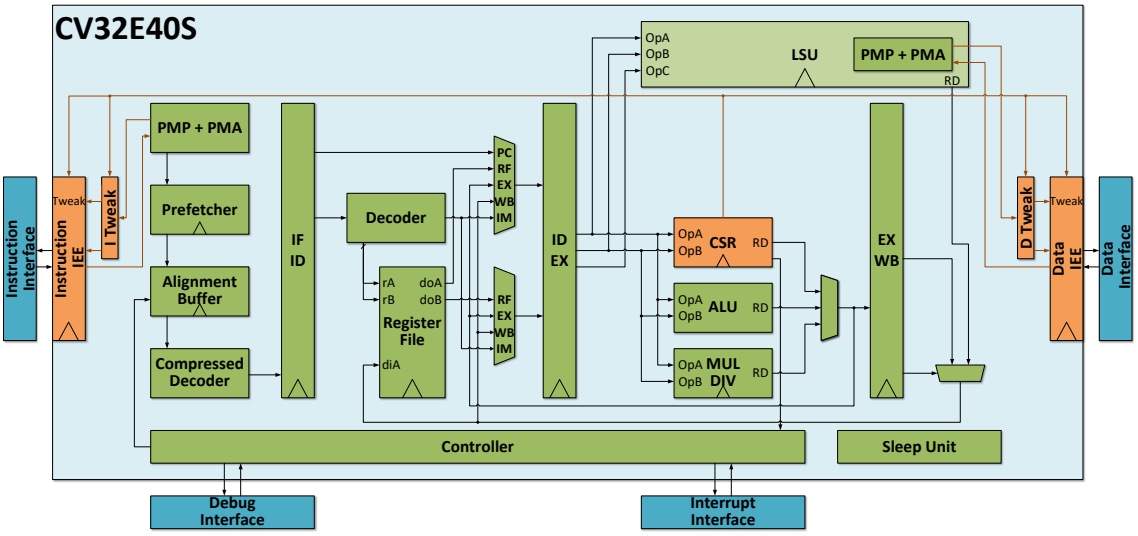


Figure 3.1: Architecture of the IEE extension

As the chosen demonstration platform does not include caches, no further adaptations of the memory subsystem were required to support the Inline Encryption Engine (IEE). If components are present between the initiator and the IEE (e.g., caches), they must store and propagate the metadata required by the IEE to construct the encryption / decryption tweak. For caches, this implies the following modifications (cf. Figure 3.2):

- The metadata required to construct the tweak, provided by the NXP-AT extensions, must be stored alongside each cache line (called `IEE MD` in the figure). Instruction caches only require a subset of the full metadata stored in data or unified caches (i.e., the EMI tweak) as most NXP-AT extensions apply exclusively to data regions, the remaining metadata is not stored and fixed to 0. While the metadata introduced by most NXP-AT extensions is relatively small, the EMI extension contributes 60 bit per memory request, which would result in a substantial storage overhead for typical embedded caches (e.g., + 21.6 % for a 2-way 16 KiB cache with 32 B lines). A design that mitigates this overhead without impacting average performance is illustrated in the figure and discussed later.
- The cache hit and replacement logic must be extended to prevent aliasing and security

issues. A *cache hit* occurs only if both the `Tag` and `IEE MD` fields of the cache line match the incoming memory request. Otherwise, the access is treated as a *cache miss*. If a cache line exists where the `Tag` field matches but the `IEE MD` field differs, the cache logic must evict this line before storing the new data. Failing to do so could allow multiple cache lines to map to the same physical address, leading to aliasing and potential security violations.

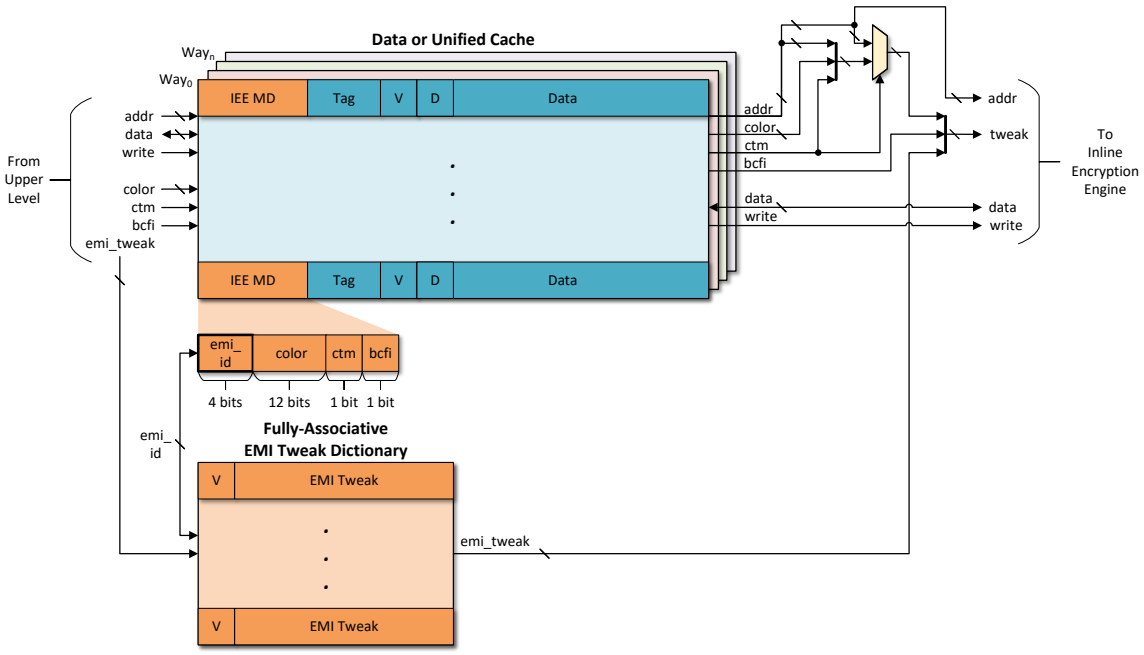


Figure 3.2: Cache design compatible with downstream IEE

On a typical embedded system, only a small number of enclaves will run concurrently (e.g., five enclaves with isolated code, data, and shared memory regions). Consequently, the number of distinct active EMI tweaks (e.g., 15 in this scenario) is small compared to the theoretical space of possible EMI tweaks (e.g., 2^{60}, required to meet the desired security level). To reduce the associated storage overhead, the full EMI tweak is not stored directly alongside each cache line. Instead, a compact identifier (e.g., a 4-bit `emi_id`) is stored, while the full EMI tweak values are maintained in a small, fully-associative dictionary. For a 2-way 16 KiB cache with 32 B lines, this approach reduces the EMI-related storage overhead from 21.6 % to approximately 2.1 %. However, this design introduces additional behavioural complexity and is only beneficial when the number of concurrently active EMI tweaks is significantly lower than the theoretical maximum. The average runtime impact is negligible as long as the EMI tweaks used by the active enclaves fit into the dictionary and the (re)starting of enclaves happens rarely.

With this design, on a cache miss, the dictionary is searched for a EMI tweak that matches the memory request. If a match is found, the corresponding index is stored in the cache line as `emi_id`. Otherwise, the EMI tweak is inserted into a free dictionary entry and the corresponding index is stored in the cache line. If no free entry is available, an existing dictionary entry must be

evicted according to a replacement policy (e.g., least-recently-used EMI tweak). Upon eviction of a dictionary entry, all cache lines referencing the corresponding `emi_id` must also be evicted to maintain correctness.

3.1.1.5 Interfaces

The IEE modules interface with the CV32E40S core via its two internal OBI ports (instruction and data) and expose these to the external memory system through a corresponding external OBI interface, enabling encryption or decryption of data before it is written to memory or passed to the core. Additionally, the ISA is extended with new CSRs that store the encryption key and the boundary of the encrypted memory region. Finally, as mentioned in Section 3.1.1.3, the IEE module can optionally be connected to the other security extensions provided by NXP-AT to increase protection against logical attacks.

3.1.1.6 ISA specialization

The specification of the ISA extension did not change since deliverable D3.1. The IEE extension adds CSRs representing the encryption key (`ieekey0` - `ieekey3`) and the end of the encrypted memory region (`mieeencend` and its read-only alias `uieeencend`). The base address of the encrypted memory region can be configured using the `IEE_ENCRYPTION_BASE` design parameter.

Table 3.2 shows the addresses assigned for the IEE CSRs after integration with the CV32E40S core.

CSR	Index	Privilege	Function
ieekey0	0x7E0	MRW	Least Significant Word (LSW) of the memory encryption key
ieekey1	0x7E1	MRW	32-bits of the memory encryption key
ieekey2	0x7E2	MRW	32-bits of the memory encryption key
ieekey3	0x7E3	MRW	Most Significant Word (MSW) of the memory encryption key
mieeencend	0x7E4	MRW	One byte beyond the last address of the encrypted memory region
uieeencend	0xCC0	UR	User-mode read-only alias of mieeencend

Table 3.2: CSRs introduced by the IEE extension

3.1.1.7 Evaluation prototype

The evaluation prototype consists of a CV32E40S core enhanced with NXP-AT's security extensions, including the here described IEE extension, embedded in one of the following evaluation environments:

- *CORE-V Verification Environment*
  The enhanced core replaces the stock core in the CV32E40S testbench. The resulting testbench can be evaluated using RTL simulation (tested with the Xcelium simulator).
- *CORE-V MCU*
  The enhanced core replaces the CV32E40P core in the CORE-V MCU design. The resulting design can be synthesized to a Nexys A7-100T or Genesys 2 FPGA board and evaluated there (tested with Genesys 2).

A suite of self-checking tests has been written for the IEE extension. These tests can be and were executed in both supported environments.

ASIC

PPA analysis using a 16 nm process with worst timing, targeting a clock frequency of 500 MHz (the highest frequency before the area increases significantly), shows an increase in area of 77.79 %, 37295 Gate Equivalent (GE) in absolute numbers, caused by the IEE extension compared to the CV32E40S baseline core. The IEE extension increased the combinatorial delay of the critical path by 16.6 % compared to the CV32E40S baseline core.

FPGA

The evaluation prototype was tested on the Genesys 2 FPGA board (featuring a XC7K325T-2FFG900C FPGA IC). The design was synthesized using Vivado v2025.2, targeting a frequency of 20 MHz. The IEE extension increased the usage of lookup tables by 80.99 % (absolute: 8933) and registers by 58.88 % (absolute: 2826) compared to the CV32E40S baseline core.

While these increases are significant, memory encryption is often required for security-sensitive products.

3.1.1.8 State-of-the-Art Comparison

The overheads and security properties of a memory encryption engine are mainly-determined by the underlying cryptographic algorithm. The NXP-AT IEE module uses BEANIE [35], due to the target application of logical attack mitigation in microcontrollers. In the following, BEANIE is compared with other small-state ciphers (SCARF [23], BipBip [16], ChiLow [17]) and larger 64-bit state ciphers (PRINCEv2 [20], QARMAv2 [15]). Due to the different construction goals, target applications, and security assumptions, it is difficult to make a truly meaningful comparison.

BEANIE

BEANIE is a tweakable, low-latency 32-bit block cipher designed to aid logical attack mitigation in microcontrollers (e.g., see NXP-AT BCFI, CTM and EMI extensions). Due to this targeted application area, BEANIE supports up to 128-bit tweaks required to accept tweak modifiers from various sources (e.g., the tweak modifiers from BCFI, CTM and EMI described in ISOLDE require 91 bit) and has a low-latency encryption and decryption path.

BEANIE's security claims (2^{80} time and 2^{40} data complexity) hold only within a U-shape attack model relevant in the targeted application area: an attacker may retrieve decryption results under a tweak different from that used during encryption, but cannot directly access the ciphertext (i.e., memory encryption cannot be bypassed). For the classical attack model, where the attacker has direct access to the ciphertext, the security reduces below the practical security claim. However, for the envisioned use with internal memories of microcontrollers, physical attacks that would enable the classical attack model are considered infeasible.

BEANIE's design separates the tweak-key schedule from the data path, allowing both components to be parallelized and executed in different cycles, thereby reducing the overall latency of memory encryption. This separation aligns well with typical bus read transactions, which consist of distinct address and data phases. Consequently, the tweak-key schedule can be computed in parallel

Metric	Value
Min	0 %
Max	+37.53 %
Geometric Mean	+6.84 %
Geometric SD	7.39 %

Table 3.3: Relative Cycle Overhead of NXP-AT IEE Memory Encryption (BEANIE-Based) on Embench IoT Benchmarks (LLVM18, `-Os`)

with the address phase, while the data path operates in parallel with the data phase, without introducing stall cycles.

Stall cycles are required for write transactions (one cycle for word writes and two cycles for sub-word writes), with sub-word writes representing the worst case, since systems without caches must perform a read–decrypt–modify–encrypt-write sequence. In practical applications, however, most memory transactions are reads, followed by word writes, while sub-word writes are comparatively infrequent. Unlike ciphers with 64-bit block size, BEANIE's 32-bit blocks match the native word size of microcontrollers, making the overall performance impact of read–decrypt–modify–encrypt-write operations negligible since they are only required for less frequent sub-word writes.

Table 3.3 presents the relative cycle overhead on Embench IoT Benchmarks when memory encryption is enabled, using the NXP-AT IEE extension configured with BEANIE as the cryptographic primitive.

SCARF, BipBip, ChiLow

SCARF, BipBip, and ChiLow, like BEANIE, are small-state ciphers whose security is valid only within their intended application domains, which is typical for this class of designs. SCARF is a tweakable, low-latency, encryption-only 10-bit block cipher with up to 48-bit tweaks, optimized for cache randomization and offering a security level matching BEANIE. BipBip is a 24-bit block cipher supporting up to 40-bit tweaks, designed for pointer-slice encryption and claiming the same security level as BEANIE while featuring a low-latency decryption path. ChiLow is a tweakable cipher that supports 32- or 40-bit blocks and up to 64-bit tweaks, targeted at authenticated code encryption and claiming 128-bit security (but only up to 2^8 queries per tweak and 2^{40} total encryption and decryption queries), also with a low-latency decryption path.

Although these ciphers are well-matched to their specific use cases, none is suitable for the application scenario addressed by BEANIE. Their tweak sizes, capped at 64 bits or less, are insufficient to combine multiple feature inputs to prevent logical attacks. A 32-bit block size, which is beneficial in limiting memory encryption overheads on 32-bit microcontrollers without caches, is only offered by ChiLow, but both ChiLow and BipBip prioritize low-latency decryption while leaving encryption latency substantially higher. When the block size of the cipher is smaller than the native word size of the processor, each word access requires multiple encryption or decryption operations, increasing either area (through additional cipher instances) or latency (by spreading the operations across multiple cycles). Overall, existing small-state ciphers are not well-suited for the application area that NXP-AT IEE targets.

PRINCE, QARMA

PRINCE and QARMA are general-purpose ciphers, which are also secure in the classical attack setting due to their larger state size. PRINCE is a non-tweakable, low-latency 64-bit block cipher, optimized for usage in constrained, embedded devices (e.g., for memory encryption). QARMA is a tweakable, low-latency cipher that supports 64- or 128-bit blocks with a matching-size tweak, optimized for general-purpose usage and suitable for memory encryption and generating MACs.

Although these ciphers offer a higher security level and are secure in the classical attack setting, their impact on the performance of 32-bit microcontrollers without caches is significant due to the 64-bit block size (requiring read–decrypt–modify–encrypt-write sequences for all stores). Hence, they are not well-suited for the application area that NXP-AT IEE targets.

Hardware Cost

Table 3.4 lists the hardware cost of BEANIE and other similar constructions implemented on NanGate 15 nm technology. For PRINCE and QARMAv2-64-σ_0, hardware costs of round-reduced variants are reported to better align their security level with BEANIE. For example, a 5+5-round BEANIE roughly corresponds to an 8- or 10-round PRINCE [35].

Although a strictly meaningful comparison is difficult given the differing construction goals, target applications, and underlying security assumptions, some general observations can still be drawn from Table 3.4:

- O1: BEANIE's datapath offers lower latency than the other evaluated ciphers, except for the single-circuit configuration, which trades performance for reduced area. Although a BEANIE-based memory encryption engine requires both the tweak-key schedule (BEANIE TKS) and the datapath, the TKS can be computed in parallel with the address phase of read transactions. Since both latencies are similar, this scheduling approximately halves the impact of BEANIE on the read path latency while not introducing any stall cycle. For write transactions, one or two stall cycles are required, but the overall impact is small, as read transactions are typically more frequent than writes (especially if instruction memory is encrypted).
- O2: The latency and area required by the tweak-key schedule are lower than for comparable constructions.
- O3: The combined area of the BEANIE TKS and its decryption datapath (latency optimized: 3400 μ^2, area optimized: 2335 μ^2) is comparable to ChiLow-32 (when both are optimized for latency, and even smaller when optimized for area), whose state size matches BEANIE but whose encryption latency is substantially worse. Compared to QARMAv2-64-σ_0, a low-latency block cipher with a 64-bit block size and a tweak size equal to BEANIE's, QARMA achieves a smaller footprint requiring approximately 40 % less area. However, QARMA's 64-bit block size also introduces a significantly higher runtime overhead for memory encryption, as each store requires a read–decrypt–modify–encrypt–write sequence on a 32-bit microcontroller.

3.1.1.9 Requirement Compliance

Table 3.5 provides a comparison between the obtained results and the corresponding IP requirements from WP1. The table summarizes the verification outcomes for each requirement and

Cipher	Block bits	Latency optimized			Area optimized		
		Time ps	Area μ^2	Area GE	Time ps	Area μ^2	Area GE
SCARF (Enc) [17]	10	217	1096	5577	298	969	4928
BipBip (11R Enc) [15]	24	683	10334	52561	1520	7031	35760
BipBip (11R Dec) [15]	24	327	6554	33843	710	5344	27180
ChiLow-32 (Dec) [17]	32	283	3582	18218	302	3418	17384
ChiLow-40 (Dec) [17]	40	278	3205	16303	299	3043	15480
BEANIE (5R Enc)	32	192	677	3443	289	477	2424
BEANIE (5R Dec)	32	203	769	3910	293	473	2407
BEANIE (split circuit)	32	203	1446	7352	293	950	4831
BEANIE (single circuit)	32	232	1446	7353	467	810	4118
PRINCE (8 R)	64	251	1566	7964	362	1145	5823
PRINCE (10 R)	64	312	2083	10592	446	1438	7316
PRINCE (12 R)	64	376	2464	12531	537	1723	8762
QARMAv2-64-σ_0 $(r = 4)$ [15]	64	305	2381	12110	447	1583	8053
BEANIE TKS (5R)	128	212	2631	13383	341	1862	9470
QARMAv2-128-128 $(r = 9)$	128	620	9052	46043	925	6272	31901
Orthros	128	356	7482	38055	516	5811	29555

Table 3.4: Hardware cost of BEANIE and other ciphers on NanGate 15 nm technology (adapted from [35], licensed under CC BY 4.0)

indicates the achieved level of compliance.

REQ-DOC	REQ-ID	REQ-TYPE	REQ-Description	Verification Method	Activities / Justification	Compliance (C / PC / NC)
D1.4	IEE-01	F-HW_ARCH-FEA	IEE shall support a 128-bit tweak.	Testbench (RTL simulation, FPGA)	Covered by reviews and passing unit tests	C
D1.4	IEE-02	F-HW_ARCH-S	IEE shall support a configurable security level.	Reviews (specification, code)	Covered by reviews (number of cipher rounds is configurable)	C
D1.4	IEE-03	F-HW_ARCH-S	The memory range where IEE is active shall be configurable by M-mode software.	Testbench (RTL simulation, FPGA)	Covered by passing unit tests	C

Continued on next page

D1.4	IEE-04	NF-HW_PH-PR	IEE shall not considerably impact the maximum frequency.	RTL Synthesis Reports	ASIC synthesis results indicate that frequencies up to 500 MHz can be achieved without excessive area growth. As the target demonstrator requires a substantially lower operating frequency of 189.84 MHz, the module provides sufficient timing margin for the intended use case.	C
D1.4	Target Performance		The average runtime overhead shall be up to 5 %.	Benchmarks (Embench IoT)	The measured average runtime overhead is 6.84 %, exceeding the target by 1.84 percentage points. While the requirement is not satisfied, the measured overhead remains close to the target value. The evaluation was performed on a cacheless platform, representing a conservative configuration that is expected to overestimate the overhead compared to cache-equipped systems.	PC

Continued on next page

D1.4	Target Performance	The achievable clock frequency on the FPGA shall be greater than or equal to 50 MHz.	Synthesis Results	Even the unmodified baseline platform on the Genesys 2 FPGA only marginally achieved this frequency. The intended objective was to ensure that integration of the NXP-AT extensions would not reduce the operating frequency of the CORE-V-MCU baseline platform on the Genesys 2 FPGA (20 MHz). This objective was achieved.	NC

Table 3.5: Requirement Compliance Matrix IEE

3.1.2 Backward-Edge Control Flow Integrity (BCFI) – NXP-AT

3.1.2.1 IP Card

Basic Info		
IP name	Backward-Edge Control Flow Integrity (BCFI)	
License	Closed-source proprietary	
Repository	N/A	
Architecture		
Clock	Number of clock domains	1
	Synchronous with system	Y
	Clock generated internally	N
Ctrl Interface	ISA extension?	Y (RV32I; partially complies with Zicfiss [63])
	Memory mapped?	N
	Protocol	N/A
	Address Map	N/A
Initiator Interface	Protocol	
	Cached?	N/A
	IOMMU?	
Interrupts	Interrupts	N
Microarchitecture		
Parametrization	Parametric no. cores?	N
	Parameteric config?	Y
Programmability	Contains programmable cores?	N
	ISA	N/A
Software		
Compiler	Requires specialized compiler?	Y
	Compiler repository	N/A (closed-source proprietary)
Hardware Abstraction Layer	N	
High-level API	Is there a high-level API/SDK?	N
	SDK repository	N/A
	Is there a domain-specific compiler?	N
Integration		
IP Distribution	Manifest type (if any)	N
	Standalone simulation?	N
	(if standalone sim) SW requirements?	N/A
	Integration documented / examples?	Y, closed-source proprietary
Synthesis	Is the IP synthesizable?	Y
	FPGA synthesis example available?	Y, closed-source proprietary
	ASIC synthesis example available?	Y, closed-source proprietary
Simulation	Closed-source simulation?	Y (Xcelium)
	Open-source simulation?	N
Evaluation	PPA results available?	Y

3.1.2.2 General Information

According to Google Project Zero, memory corruption vulnerabilities are the most used path to gain unintended remote control over digital devices [39]. In 2023, 75 % zero-day exploits in the wild were based on memory corruption vulnerabilities. Programming languages like C and C++

that offer neither memory nor type safety are especially affected. While memory-safe programming languages (like Rust) gain momentum, C is still one of the most popular programming languages [24], especially for embedded system development. Making matters worse, constrained embedded environments include only subsets of the defense mechanisms employed in larger systems (e.g., no address space layout randomization or only with low entropy), leading to easier exploitation. Hence, during the transition period to memory-safe programming languages or for legacy code, additional security layers are needed to mitigate these attack paths, especially for constrained embedded devices.

The exploitation of memory safety vulnerabilities may enable an attacker to modify the program behavior and take control. For example, the attacker could replace function return addresses spilled on the stack with addresses containing attack gadgets. Such attacks, which aim to modify the backward-edge control flow of programs, are among the most common and known ones. A typical memory safety issue is a buffer overflow vulnerability, which occurs when user input is not correctly sanitized. For example, missing bound checks that otherwise ensure the user input has a valid length could allow an attacker to craft an input that overwrites return addresses on the stack when (part of) the input is copied to internal data structures.

Modern systems feature a variety of countermeasures, like stack canaries and Address Space Layout Randomization (ASLR), to mitigate these attacks. However, these countermeasures have weaknesses and can be bypassed. For example, both are vulnerable to information disclosure attacks. A more recent countermeasure, not suffering from these weaknesses, are hardware-supported shadow stacks. In a system with this countermeasure, return addresses are stored on an isolated stack, called a shadow stack, in addition to the regular stack. The isolation guarantees that only special instructions can access the shadow stack while regular memory operations cannot access it. Therefore, an attacker with arbitrary write access to the regular stack can overwrite return addresses stored on the regular stack but cannot modify the corresponding copies stored on the shadow stack. Before executing the return in the function epilogue, the return address from the regular stack is compared with the return address on the shadow stack to detect attacks. RISC-V International ratified a shadow stack design for RISC-V, the *Zicfiss* extension [69].

As this deliverable provides updated information about the IP state since deliverable D3.2, please refer to the earlier deliverables for additional information about the Backward-edge Control Flow Integrity (BCFI) module.

3.1.2.3 Purpose and Scope

While the Zicfiss extension requires an MMU to isolate the shadow stack, our BCFI extension isolates the shadow stack cryptographically using a tweakable memory encryption engine like the NXP-AT IEE extension (see Section 3.1.1). Cryptographical isolation is achieved by using distinct encryption tweaks for memory accesses resulting from shadow stack operations, differentiating them from regular memory accesses. Further, a hash value uniquely identifying the current return address chain is stored in an isolated CSR. The previous (intermediate) hash values are spilled onto the shadow stack instead of the actual return addresses, preventing the replay of encrypted return addresses (the topmost hash value depends on all previous ones). When returning from a function, the current and previous hash values are passed to the inverse hash function to reconstruct the return address. For more information about the concept of our BCFI extension, which is based on the PACStack paper [49], see the detailed description in Section 3.1.2.2 of deliverable

D3.1.

The BCFI extension can be added to systems already including a tweakable memory encryption engine (i.e., needed to fulfill the security requirements) and does not rely on the presence of an MMU. Further, it offers better protection against physical attacks.

3.1.2.4 Refined architecture description

The architecture of the BCFI extension did not change since deliverable D3.2.

3.1.2.5 Interfaces

The BCFI extension is tightly integrated in the CV32E40S processor and requires NXP-AT's IEE extension.

3.1.2.6 ISA specialization

The specification of the ISA extension did not change since deliverable D3.1. The BCFI extension adds instructions to manipulate data on the shadow stack, allowing to push (`sspush`, `c.sspush`), pop (`sspopchk`, `c.sspopchk`), load (`sslw`), and atomically swap (`ssamoswap`) words. Further, it adds instructions for receiving (`ssrdp`) and increasing the shadow stack pointer value (`ssincp`, `c.ssincp`). To enable the configuration of the extension, the `mseccfg` and `menvcfg` CSRs are modified. The universal hash function is parametrized with the `ssuhfx` and `ssuhfxinv` CSRs. The shadow stack pointer is stored in the `ssp` CSR, and the topmost hash value is stored in the `sstca` CSR.

Table 3.6 shows the addresses assigned for the BCFI CSRs after integration with the CV32E40S core.

CSR	Index	Privilege	Function
ssp	0x011	URW	Shadow Stack Pointer
sstca	0x830	URW	Topmost Chained Address
ssuhfx	0x7D0	MRW	Parameter X of UHF
ssuhfxinv	0x7D1	MRW	Parameter X^{-1} of IUHF

Table 3.6: CSRs introduced by the BCFI extension

3.1.2.7 Evaluation prototype

The evaluation prototype and available evaluation environments are the same as described in Section 3.1.1.7 for NXP-AT's IEE module.

A suite of self-checking tests has been written for the BCFI extension. These tests can be and were executed in both supported environments.

ASIC

PPA analysis using a 16 nm process with worst timing, targeting a processor clock frequency of 500 MHz (the highest frequency before the area increases significantly), shows a 9.13 % (ab-

solute: 4377 GE) increase in area caused by the BCFI extension compared to the CV32E40S baseline core.

FPGA

The evaluation prototype was tested on the Genesys 2 FPGA board (featuring an XC7K325T-2FFG900C FPGA IC). The design was synthesized using Vivado v2025.2, targeting a frequency of 20 MHz. The BCFI extension increased the usage of lookup tables by 11.15 % (absolute: 1230) and registers by 5.40 % (absolute: 259) compared to the CV32E40S baseline core.

3.1.2.8 State-of-the-Art Comparison

In this section, NXP-AT BCFI is compared with the Armv8.1M Pointer Authentication and Branch Target Identification extension (PACBTI) [13, Ch. B6][7], the RISC-V Shadow Stack Extension (Zicfiss) [65, Sec. 33.3][66, Sec. 22.2] and PACStack [49]. BCFI and PACBTI target deeply embedded 32-bit processors without MMUs, while the other features require 64-bit processors with an MMU. In contrast to PACBTI, BCFI provides stronger protection against both replay and physical attacks while also requiring a smaller memory footprint.

Table 3.7 summarizes the comparison of BCFI, PACBTI, Zicfiss, and PACStack. Because some of the mechanisms target different processor classes and offer varying security guarantees and features, a purely quantitative comparison of hardware or software overhead is often not meaningful. Already, the microarchitecture of the baseline processor may differ considerably, e.g., in-order versus out-of-order or single-issue versus dual-issue, making direct numerical comparisons problematic. Therefore, Table 3.7 focuses primarily on qualitative aspects, while numerical values are included only where available in the literature to provide a rough, high-level orientation.

Additional background information on the introduced features can be found in the following paragraphs.

Arm

Arm introduced PACBTI as an optional extension beginning with Armv8.1-M. Pointer Authentication Codes (PACs) can provide backward-edge control-flow integrity, and therefore we focus on the Pointer Authentication (PA) feature. PACs are cryptographic signatures computed from a value to be protected (held in an input register), a secret key implicitly selected by the privilege level and security state, and a modifier providing context (held in a register, e.g., the stack pointer).

PACBTI introduces instructions to generate and authenticate PACs: the computed PAC is written to an output register, and a failed authentication raises a synchronous INVSTATE usage fault. These instructions are part of the NOP-compatible space, so protected binaries can still be executed on older ISA versions. The key required for the PAC instructions is implicitly selected from four 128-bit keys stored in system registers (one for each privilege level and security state combination). If the software stack consists of multiple contexts (e.g., OS and multiple tasks), then each context should use a distinct PA key. The PA feature can be enabled per privilege level and security state.

To mitigate attacks of tampering with return addresses (e.g., return-oriented programming), the return address stored in the link register is signed in the function prologue using the current stack pointer as a modifier. Then, both the return address and the PAC are pushed on the stack.

Although an attacker may overwrite these values, generating a valid PAC remains infeasible as long as the key is unknown and the PAC width is sufficiently large. During the function epilogue, both the return address and its associated PAC are popped from the stack, and the return address is authenticated using its PAC and the stack pointer as a modifier. If either the return address or its PAC is corrupted, or if the modifier differs from the one used when generating the PAC, authentication fails.

This scheme does not mitigate all replay attacks: (return address, PAC) pairs with matching modifiers (e.g., generated by functions whose stack frame starts at the same address) can be swapped without detection. The set of possible replay candidates can be reduced by adding additional information to the modifier. For example, Armv9.4-A pointer authentication supports adding the PC of the signing instruction to the modifier [9, Sec. C6.2.307, Sec. C6.2.27]. However, the Armv8-M architecture provides no hardware support for this improvement, nor does the Arm toolchain emulate it for the M profile.

RISC-V

The RISC-V Shadow Stack extension (Zicfiss) introduces a hardware-protected shadow stack for RISC-V, similar to Intel CET for x64 [44, Ch. 18]. A shadow stack is a separate, isolated stack used solely for backward-edge control flow transitions. It can only be manipulated by dedicated instructions but not using regular load and stores (which an attacker would exploit to overwrite memory contents). During the function prologue, the return address is pushed both on the regular and on the shadow stack. During the function epilogue, the presumed return address is popped from both stacks and compared. If the values differ, a synchronous exception is raised.

The Zicfiss extension isolates the shadow stack using a new page type and hence depends on paging. This implies that the shadow stack cannot be used in M mode, S mode must be implemented, and an MMU must be present (`satp.MODE != Bare`). If these dependencies are met, then pages can be marked as Shadow Stack (SS) pages using page descriptor bits, and the shadow stack can be enabled for modes below M. SS pages can only be manipulated using a dedicated shadow stack pointer (`ssp` CSR) and a set of special instructions introduced by Zicfiss. If verification of a return address fails, then a software-check exception is raised. All introduced instructions are implemented as a subset of the May-Be-Operation instructions defined by the Zimop and Zcmop extensions, which means protected binaries can be executed by processors that do not support Zicfiss.

PACStack

The PACStack paper [49] was the main inspiration for the BCFI mechanism. However, its design relies on the A-profile pointer authentication and therefore applies only for 64-bit architectures with an MMU. Porting the approach to 32-bit systems would introduce additional memory overhead, as the PAC cannot be embedded in pointers there. BCFI avoids this overhead by chaining return addresses using an invertible universal hash function and encrypting each chain element with a unique tweak whenever it is spilled into memory, as described in Section 3.1.2.3.

The Armv8.3-A Pointer Authentication (PA) feature is a more advanced version of Armv8.1-M PACBTI described in the Arm section. The main differences are:

- Armv8.3-A PA embeds the PAC in the upper bits of a pointer that are not used during address translation, avoiding memory overhead but limiting the PAC width and hence the security

level.
- Armv8.3-A PA does not raise a fault after a failed authentication. Instead, it flips a high-order bit in the pointer so that a memory translation fault occurs later when the pointer is used to access memory.
- Armv8.3-A PA uses the same set of key registers for each privilege level, and each key is associated with a dedicated set of signature generation and verification instructions.

A pointer with an embedded PAC is called a signed or authenticated pointer. The maximum width of a PAC depends on the virtual address space size configured by software. For example, modern Arm processors support virtual address spaces of 39, 48 or 52 bits, which result in PAC widths of 16, 7 or 3 bits (`55 - VA_SIZE`).

PACStack uses the PA mechanism to cryptographically bind the PAC of a return address to all previous signed return addresses by including the preceding signed return address in the modifier (forming a chain). As a result, the topmost PAC becomes a statistically unique representation of the entire sequence of signed return addresses. By ensuring that the topmost signed return address cannot be modified by an adversary (e.g., by storing it in a dedicated register not used for other purposes), any tampering with the chain can be detected probabilistically. Hence, the intermediate signed return addresses can be spilled onto the stack while still ensuring the detection of any modification, including the replay attacks discussed in the Arm section. However, since the PAC width is limited, the attacker may be able to find PAC collisions. For example, considering the best case of 16 bit PACs, an attacker only needs to observe approximately 321 PACs on average until a collision (birthday paradoxon). Once a collision has been found, the attacker can exchange the chains that resulted in a matching PAC. PACStack masks signed return addresses before pushing them onto the stack to prevent an attacker from recognizing PAC collisions, which mitigates this attack vector.

Criterion	Mechanism	Description
Processor Class	BCFI	Embedded 32-bit RISC-V without MMU
	PACBTI	Embedded 32-bit Arm M Profile without MMU
	Zicfiss	32-bit or 64-bit RISC-V with S mode and MMU
	PACStack	High-performance 64-bit Arm A Profile with MMU
Dependencies	BCFI	Zicsr extension, memory encryption engine (NXP-AT IEE, see Section 3.1.1)
	PACBTI	Armv8.1-M with PACBTI and encryption unit
	Zicfiss	Zimop, Zcmop, Zicsr extensions
	PACStack	Armv8.3-A with FEAT_PAuth and encryption unit

Continued on next page

Security Properties	BCFI	BCFI provides asynchronous, indirect detection of backward-edge control-flow violations. Its overall security is determined by the hash and encryption block size, the underlying cryptographic algorithm (see Section 3.1.1), and the executed use case. BCFI uses a block size of 32 bit to avoid memory overhead, which leads to potentially exploitable hash collisions after about 82137 values on average. However, because the memory encryption engine includes the destination address in the tweak (see Section 3.1.1), observable hash collisions are restricted to identical memory locations, significantly reducing their occurrence, especially in embedded use cases. Beyond collisions, successfully forging a specific return address outside the call graph has a probability of 2^{-32}, matching PACBTI and PACStack. Unlike these schemes, BCFI only detects violations if the use case execution faults. A lower bound for the detection rate can be derived by assuming an exception is raised when control flow jumps into non-executable or misaligned memory. The executable memory region of typical microcontrollers can be approximated by their flash sizes. For example, the maximum probability of an undetected attack attempt is 2^{-14} for a 512 KiB or 2^{-12} for a 2 MiB flash ($\frac{S_F}{2^{33}}$). Real-world detection rates will likely be higher, since after an attack attempt execution continues at a pseudorandom address inconsistent with the expected processor and memory state (e.g., leading to invalid memory accesses). Combining BCFI with a forward-edge CFI scheme (e.g., NXP-AT FCFI) further increases detection reliability and is recommended to mitigate BCFI bypasses. The replay attacks affecting PACBTI, which is the only other mechanism suitable for microcontrollers, do not apply to BCFI due to the cryptographic chaining and encryption of return addresses. While BCFI may exhibit a lower detection probability than other schemes, it inherits the security properties of memory encryption and therefore provides stronger protection against physical attacks.

Continued on next page

	PACBTI	PACBTI enables synchronous, direct detection of backward-edge control flow violations. The return address signing for Armv8.1-M implemented in the Arm toolchain only uses the stack pointer value as context modifier for PAC creation. Hence, (return address, PAC) pairs generated by function invocations with stack frames starting at the same address can be exchanged without detection. If an attacker can leak stack contents, this weakness becomes significant. Apart from this weakness, PACBTI uses 32-bit signatures, yielding a success probability of 2^{-32} for attacks trying to divert execution outside the call graph, matching both BCFI and PACStack.
	Zicfiss	Zicfiss offers the highest assurance of backward-edge control flow integrity of all compared mechanisms, assuming purely logical attacks. Shadow stacks can detect any tampering deterministically and synchronously assuming the attacker cannot execute a gadget bypassing the shadow stack isolation.
	PACStack	PACStack enables synchronous, direct detection of backward-edge control flow violations. The security level depends on the PAC width ($\|pac\|$) and the underlying cryptographic algorithm. Since PACStack prevents an attacker from observing collisions, the attacker can only guess which of the observed signed return addresses collide, leading to a success probability of $2^{-\|pac\|}$. The success probability of an attack trying to divert execution outside the call graph is $2^{-2\|pac\|}$, matching BCFI and PACBTI assuming the best-case scenario of 16-bit PACs. The replay attacks affecting PACBTI do not work against PACStack.
Scalability	BCFI	BCFI can be enabled for software contexts in any privilege level while incurring only minimal data overhead (BCFI state saved during context switches). Different software contexts can be cryptographically isolated from one another, preventing a successful attack in one context from propagating to others. The data overhead is independent of the protected task and its call stack depth. As the return address is replaced by an encrypted version, protected programs break ABI compatibility, but this is acceptable for embedded use cases often anyway requiring dedicated toolchains. As described, the attack detection probability depends on the size of the executable memory region, which inherently limits BCFI's effectiveness to embedded systems.

Continued on next page

	PACBTI	PACBTI can be enabled for software contexts in any privilege level and security state. Different software contexts can be cryptographically isolated from one another, preventing a successful attack in one context from propagating to others. Unlike BCFI, where the data overhead is independent from the protected task and its call stack depth, a 32-bit PAC is generated and stored together with every return address.
	Zicfiss	Zicfiss can be enabled for software contexts in privilege levels below M, but it cannot be used in M mode as it requires paging. For every protected software context, memory for a shadow stack storing duplicates of return addresses must be allocated. The shadow stacks associated with different software contexts are isolated by using unique shadow stack regions, preventing a successful attack in one context from propagating to others. In contrast to BCFI, the memory overhead (required shadow stack size) depends on the protected task and its call stack depth.
	PACStack	PACStack builds on the Armv8.3-A PA feature, which can be enabled for software contexts at any privilege level, although the paper focuses on user-space applications. As in BCFI, different software contexts can be cryptographically isolated from one another. Like BCFI, its theoretical memory overhead is minimal and independent from the protected task and its call stack depth because PACs are embedded directly in pointers. However, PACStack stores both the original and the signed return address on the stack to remain compatible with the AArch64 ABI. The security level depends on the effective PAC width, which decreases as the virtual address space grows. With a 39-bit virtual address space (e.g., used in mobile systems), the PAC is 16 bits wide, offering security comparable to BCFI and PACBTI. However, high-end server systems may use a 52-bit virtual address space, limiting the PAC to 3 bits, which is insufficient for security purposes.
Hardware Overhead	BCFI	Relative to CV32E40S (16nm 500 MHz): +9.13 % (4377 GE)
	PACBTI	Not public
	Zicfiss [50] (+Zicfilp)[1]	Relative to CVA6 (GF22FDX 800 MHz): +1 % (976 μm^2)
	PACStack	Not public (relies on Arm Pointer Authentication)
Software Overhead	BCFI	Baseline: CV32E40S with NXP-AT IEE running Embench IOT Compiler: LLVM17, `-Os`, C-library protected Runtime : +0 %-4.14 % (geometric mean: +0.65 %) Memory: none, return addresses moved to shadow stack within the regular stack region Code Size: +0.1 %-1.98 % (geometric mean: +0.79 %)

Continued on next page

	PACBTI [28] (PAC + BTI)[2]	Baseline: Renesas EKRA8M1 Compiler: LLVM 17.0.1 Embedded Toolchain for Arm with `-O0` Runtime: • BEEPS: +0.1-28.7 % (geometric mean: +4.7 %) • CoreMark PRO: +0.1-2.6 % (geometric mean: +1.1 %) Memory: not reported Code Size: • BEEPS: +3.3-9.4 % (geometric mean: +6.8 %) • CoreMark PRO: +1.1-7 % (geometric mean: +4.8 %)
	Zicfiss [50] (+Zicfilp)[1]	Baseline: CVA6 running MiBench Compiler: SiFive GNU toolchain for RISC-V 13.3 Runtime: +0-15.6 % (geometric mean: +4.47 %) Memory: not reported Code Size: +7.6-9.2 % (geometric mean: +8 %)
	PACStack	Baseline: Amazon EC2 `a1.metal` with Armv8.2-A cores (PA impact emulated) Compiler: LLVM 9.0.1 cross compiler for Aarch64 with `-O2` Runtime: • SPEC CPU 2017 SPECrate: +0-17 % (geometric mean: +2.75 %) • SPEC CPU 2017 SPECspeed: +0-18 % (geometric mean: +3.28 %) Memory and Code Size: not reported
Software Support	BCFI	Specialized compiler (modified function prologue and epilogue), adapted irregular return and unwinding procedures (`setjmp`,`longjmp`, `libunwind`, `libgcc_s`) and OS (context switch)
	PACBTI	
	Zicfiss	
	PACStack	
Debug Support	BCFI	Adaption of frame unwinding, support to access shadow stack memory, disassembling of custom instructions
	PACBTI	Adaption of frame unwinding, disassembling of custom instructions
	Zicfiss	Adaption of frame unwinding, support to access shadow stack memory, disassembling of custom instructions
	PACStack	Adaption of frame unwinding, disassembling of custom instructions

Table 3.7: Qualitative comparison of NXP-AT BCFI, Arm PACBTI, RISC-V Zicfiss and PACStack

[1] The paper by Manoni et al. evaluates Zicfilp (landing pads for forward-edge control flow integrity) and Zicfiss together. The authors note that the runtime overhead is primarily determined by the shadow stack operations (Zicfiss), but a direct comparison of other metrics is not meaningful, and the included numbers should be treated as a rough upper bound.

[2] Like the paper by Manoni et al., the paper by Cirne et al. evaluates BTI (landing pads for forward-edge control flow integrity) and PAC together. The authors note that the code size (landing pad is included in PAC instructions) and runtime overhead are primarily determined by the signing and authentication of return addresses (PAC), but a direct comparison of other metrics is not meaningful, and the included numbers should be treated as a rough upper bound.

3.1.2.9 Requirement Compliance

Table 3.8 provides a comparison between the obtained results and the corresponding IP requirements from WP1. The table summarizes the verification outcomes for each requirement and indicates the achieved level of compliance.

REQ-DOC	REQ-ID	REQ-TYPE	REQ-Description	Verification Method	Activities / Justification	Compliance (C / PC / NC)
D1.4	BCFI-01	F-HW_ARCH-FEA	BCFI can be enabled and disabled dynamically per privilege level.	Testbench (RTL simulation, FPGA)	Covered by passing unit tests	C
D1.4	BCFI-02	F-HW_ARCH-S	BCFI shall support a mode where the return address is stored exclusively on the shadow stack.	Testbench (RTL simulation, FPGA)	Covered by passing unit tests	C
D1.4	BCFI-03	F-HW_ARCH-S	BCFI shall allow the change of the growth direction of the shadow stack.	Testbench (RTL simulation, FPGA)	Covered by passing unit tests	C
D1.4	BCFI-04	NF-HW_ARCH-STD	BCFI should be optionally compliant to the Zicfiss RISC-V ISA extension.	Review of specification, tests generated using a compiler supporting the Zicfiss extension	Covered by reviews and passing unit tests (CSRs, instructions, and exceptions compatible with ratified Zicfiss version)	C
D1.4	BCFI-05	NF-HW_ARCH-PR	BCFI shall have less data memory overhead than a regular shadow stack.	Benchmarks (Embench IoT)	Benchmark results show that BCFI incurs no data memory overhead when return addresses are stored exclusively on the shadow stack.	C

Continued on next page

D1.4	BCFI-06	NF-HW_ARCH-PR	BCFI shall have a performance and code size overhead comparable to a regular shadow stack.	Benchmarks (Embench IoT)	Runtime overhead is negligible (<1 %) and lower than reported Zicfiss implementations on more advanced microarchitectures (Table 3.7). Code size overhead matches Zicfiss due to ISA compatibility.	C
D1.4	Target Performance		The average runtime overhead when return addresses are exclusively stored on the shadow stack shall be up to 1 %.	Benchmarks (Embench IoT)	Achieved based on benchmarking results (0.65 %)	C
D1.4	Target Performance		The achieveable clock frequency on the FPGA shall be greater than or equal to 50 MHz.	Synthesis Results	Even the unmodified baseline platform on the Genesys 2 FPGA only marginally achieved this frequency. The intended objective was to ensure that integration of the NXP-AT extensions would not reduce the operating frequency of the CORE-V-MCU baseline platform on the Genesys 2 FPGA (20 MHz). This objective was achieved.	NC

Table 3.8: Requirement Compliance Matrix BCFI

3.1.3 Cryptographically Tagged Memory (CTM) – NXP-AT

3.1.3.1 IP Card

Basic Info		
IP name	Cryptographically Tagged Memory (CTM)	
License	Closed-source proprietary	
Repository	N/A	
Architecture		
Clock	Number of clock domains	1
	Synchronous with system	Y
	Clock generated internally	N
Ctrl Interface	ISA extension?	Y (RV32I)
	Memory mapped?	N
	Protocol	N/A
	Address Map	N/A
Initiator Interface	Protocol	
	Cached?	N/A
	IOMMU?	
Interrupts	Interrupts	N
Microarchitecture		
Parametrization	Parametric no. cores?	N
	Parameteric config?	Y
Programmability	Contains programmable cores?	N
	ISA	N/A
Software		
Compiler	Requires specialized compiler?	Y
	Compiler repository	N/A (closed-source proprietary)
Hardware Abstraction Layer	N	
High-level API	Is there a high-level API/SDK?	N
	SDK repository	N/A
	Is there a domain-specific compiler?	N
Integration		
IP Distribution	Manifest type (if any)	N
	Standalone simulation?	N
	(if standalone sim) SW requirements?	N/A
	Integration documented / examples?	Y, closed-source proprietary
Synthesis	Is the IP synthesizable?	Y
	FPGA synthesis example available?	Y, closed-source proprietary
	ASIC synthesis example available?	Y, closed-source proprietary
Simulation	Closed-source simulation?	Y (Xcelium)
	Open-source simulation?	N
Evaluation	PPA results available?	Y

3.1.3.2 General Information

As this deliverable provides updated information about the IP state since deliverable D3.2, please refer to the earlier deliverables for additional information about the Cryptographically Tagged Memory (CTM) module.

3.1.3.3 Purpose and Scope

As explained in Section 3.1.2, memory corruption vulnerabilities are the most used path to gain remote control over computing devices. To mitigate attacks exploiting memory safety issues to modify function return addresses spilled on the stack, we introduced the BCFI extension in Section 3.1.2. Further, a device may isolate different tasks logically (i.e., using memory protection or management units) or cryptographically (using memory encryption engines, see Section 3.1.1 for more information). However, there exist other fine-grained assets within a task's stack, heap, or global objects that are of interest to attackers, such as

- Function pointers
- Values indirectly affecting the control flow (e.g., evaluated in conditional branches)
- Sensitive data like cryptographic keys

These sensitive assets cannot be sufficiently protected with the mentioned technologies, and hence an additional mechanism is required.

Memory tagging can be used to mitigate this security gap. It assigns additional metadata, the color or tag, with memory blocks of a defined size. Every genuine pointer and its associated memory blocks are assigned the same statistically unique color at memory allocation. Therefore, only the designated pointer received from the allocator can be used to access the allocated data. This lock-and-key mechanism prevents out-of-bound accesses (spatial bugs) using another pointer or accesses using a dangling pointer (temporal bugs).

Note that the colors must be stored in addition to the rest of the data, increasing the overall memory usage. Therefore, existing memory tagging implementations, like Armv8.5-A MTE [8], allocate only a few bits for the colors, making them unsuitable for security purposes. Cryptographically tagged memory [55] avoids this additional memory overhead by implicitly linking memory blocks with the genuine color. This implicit association is achieved by including the color in the tweak for memory encryption and decryption. Hence, assuming a memory encryption engine is available, this technique allows us to use more bits for the color without increasing the memory overhead. The major behavioral difference to the existing schemes is that an attacker can still misuse pointers to access sensitive memory blocks. However, it is much harder to perform a successful attack, as access triggered from a misused pointer will not be useful:

- Suppose misusing a pointer triggers an out-of-bounds read of a memory location associated with a different color. Then, the decryption results are wrong, as the color does not match, ensuring the confidentiality of the corresponding data.
- Suppose misusing a pointer triggers an out-of-bounds write to a memory location associated with a different color. Then, the color of the genuine pointer will not match the one of the misused pointer. Hence, the attacker cannot modify the data in a controlled way.

Until now, cryptographically tagged memory has only been implemented on 64-bit processors, where unused bits in the virtual address space are utilized to store the tags. The CTM extension

adds cryptographically tagged memory to 32-bit processors without requiring a memory management unit. The smaller address space poses additional challenges, as usually no unused bits in pointers are available, and therefore, a different strategy to embed the colors in tagged pointers is needed.

3.1.3.4 Refined architecture description

The overall architecture of the CTM extension remains unchanged compared to deliverable D3.2 and is illustrated in Figure 3.3. Elements of the CV32E40S core affected by the CTM integration are highlighted in orange. The following components have been updated since deliverable D3.2:

- The color extraction and removal logic was moved from the D Tweak module into the execute stage before the LSU unit to ensure tagged pointers are compatible with PMP access checks. The transformed and original address are passed to the LSU unit, which passes them on to the D Tweak unit as part of the internal data request signals. The D Tweak module uses this information to assemble the final encryption tweak.

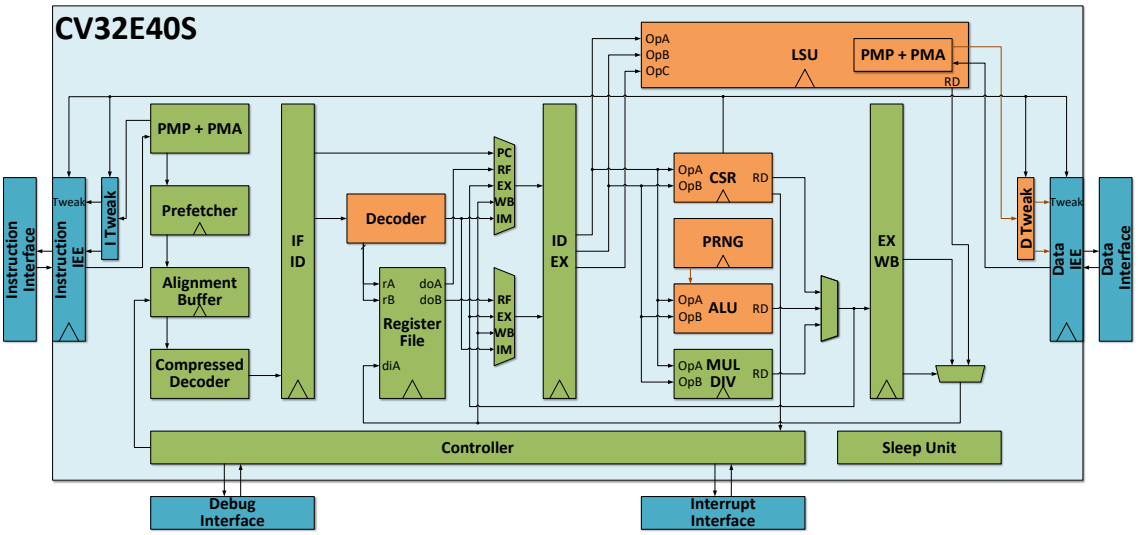


Figure 3.3: Architecture of the CTM extension

3.1.3.5 Interfaces

The CTM extension is tightly integrated in the CV32E40S processor and requires NXP-AT's IEE extension.

3.1.3.6 ISA specialization

The specification of the ISA extension did not change since deliverable D3.1. The CTM extension adds two instructions for tagging (`ctmtag`) and untagging (`ctmuntag`) pointers. Further, it adds the `CTME` field to the `mseccfg` CSR, allowing software to disable CTM.

3.1.3.7 Evaluation prototype

The evaluation prototype and available evaluation environments are the same as described in Section 3.1.1.3 for NXP-AT's IEE module.

A suite of self-checking tests has been written for the CTM extension. These tests can be and were executed in both of the supported environments.

ASIC

PPA analysis using a 16 nm process with worst timing, targeting a processor clock frequency of 500 MHz (the highest frequency before area increases significantly), shows a 0.03 % (absolute: 14 GE) area increase caused by the CTM extension compared to the base CV32E40S core.

FPGA

The evaluation prototype was tested on the Genesys 2 FPGA board (featuring an XC7K325T-2FFG900C FPGA IC). The design was synthesized using Vivado v2025.2, targeting a frequency of 20 MHz. The CTM extension increased the usage of lookup tables by 0.87 % (absolute: 96) and registers by 0.90 % (absolute: 43) compared to the base CV32E40S core.

3.1.3.8 State-of-the-Art Comparison

In this section, NXP-AT CTM is compared with the Arm Memory Tagging Extension (AMTE) [9, Ch. D10][8], the RISC-V Memory Tagging Extension (RMTE) [64] and CrypTag [55]. The NXP-AT CTM extension targets deeply embedded 32-bit processors without MMUs, while the other features require 64-bit processors with an MMU. Compared to AMTE and RMTE, CTM supports higher security levels without increasing memory usage, avoids additional bandwidth for tag loading, enables more fine-grained protection granules, and provides stronger protection against physical attacks.

Table 3.9 summarizes the comparison of NXP-AT CTM, AMTE, RMTE, and CrypTag. Because some of these mechanisms target different processor classes and offer varying security guarantees and features, a purely quantitative comparison of hardware or software overhead is often not meaningful. Already, the microarchitecture of the baseline processor may differ considerably, e.g., in-order versus out-of-order or single-issue versus dual-issue, making direct numerical comparisons problematic. Therefore, Table 3.9 focuses primarily on qualitative aspects, while numerical values are included only where available in literature to provide a rough, high-level orientation.

Additional background information on the introduced features and a quick comparison with CHERI (Capability Hardware Enhanced RISC Instructions) can be found in the following paragraphs.

Arm

Arm introduced AMTE with Armv8.4-A. When AMTE is enabled, the physical memory is divided into 16 B granules, called data granules below. Every allocated data granule is associated with a 4 bit data tag stored in a memory region isolated from normal data. This 4 bit tag value is added to genuine pointers at allocation, stored in upper bits that are otherwise unused according to the ISA (i.e., the Arm A profile currently supports a max virtual address space size of 56 bits, so the most-significant byte can always be reused). AMTE relies on the Top Byte Ignore (TBI) feature to ensure that the most-significant byte is ignored during address translation, allowing the tag to be

stored in `VA[59:56]`.

During a regular load or store access to a tagged memory region, the pointer tag (key) is compared with the implicitly fetched data tag (lock) of the data granule associated with the pointer address. If the values do not match, then either a synchronous Tag Check Fault is raised or the check results are asynchronously accumulated in a system register so that software running at a higher privilege level can later take appropriate action (e.g., when the affected program performs a system call). As highlighted in Section 3.1.3.3, this lock-and-key mechanism mitigates the exploitation of spatial and temporal memory safety issues. If the allocator ensures that sequential objects are always associated with different tags, then sequential memory safety violations can be detected deterministically. Memory safety violations that give the attacker full control of the pointer (i.e., arbitrary read or write gadgets) can only be mitigated probabilistically. Due to the small 4-bit tags, the 93.75 % detection probability of these violations is insufficient for security-critical systems.

The enablement of AMTE and designation of a memory region as tagged involves fields in system registers and stage 1 page descriptor bits. The isolation of the tag storage is implementation-specific. For example, in a Cortex-A710 and CI-700 system, a dedicated Memory Tag Unit (MTU) in front of the DDR controller divides the DRAM into a data and tag storage [3]. The system software must program the MTU with the base address of the tag region at boot; then the MTU can calculate the tag physical address from the associated data physical address. Afterward, the MTU can generate data requests (triggered by regular load or store instructions) and tag requests (implicitly triggered by load or store to a tagged memory region or explicitly triggered by tag manipulation instructions). If the processor supports memory encryption contexts, then the tag storage may be additionally encrypted. To ensure compatibility of AMTE checks with caches, the data tags must be added to the cache line tags.

AMTE adds various instructions for tag manipulation (adding pseudorandom tag to pointers and getting or setting data tags), pointer arithmetic (modifying tag and pointers and subtraction ignoring tag), system support (bulk data tag manipulation), and cache maintenance to the ISA.

RISC-V

RMTE, currently under development, follows the same principles as AMTE: the memory is divided into 16 B data granules, the data granules are associated with tags stored in an isolated memory location, and tag values are added to the unused upper bits of genuine pointers at allocation (relies on the RISC-V Pointer Masking Extensions [66, Ch. 24]). During regular load and store operations to tagged memory regions, the tags are checked. RMTE introduces instructions similar to AMTE for tag manipulation (adding a pseudorandom tag to a pointer, adding an immediate to a pointer tag, and getting or setting data tags) and system support (bulk setting of data tags and bulk checking of tags). Like with AMTE, RMTE can be enabled for selected memory regions using a combination of CSRs and stage 1 page descriptor bits.

In contrast to AMTE, RMTE supports multiple tag sizes (4 bit or 7 bit) enabling increased detection rates, allows MTE-enabled binaries to be executed on systems without MTE support, and uses a virtually indexed tag storage. The base address of the virtually-indexed tag storage is defined using the `xVITT_BASE` CSR per execution mode, and it extends to the end of the corresponding virtual address space. Regular loads and stores with addresses within the tag store raise access faults.

CrypTag

The CrypTag paper [55] was the main inspiration for the CTM module, but it targets 64-bit RISC-V systems with an MMU (i.e., CVA6) and is not suitable for 32-bit embedded RISC-V systems. CrypTag combines logical memory safety using memory tagging with memory encryption (mitigating selected physical attacks) by including the pointer tag in the tweak of the memory encryption engine. Especially for systems that anyway require memory encryption due to their threat model (e.g., the attacker has physical access to the device) and sensitive assets, realizing memory tagging using tweakable memory encryption has various advantages: memory overhead is independent of tag width and hence the security level (assuming the tag fits into the pointer), no additional memory bandwidth is consumed by memory tagging, and it inherits the security properties of memory encryption (e.g., mitigation of cold-boot attacks or Rowhammer), and the overhead on top of memory encryption is small. Therefore, CryptTag supports larger tag sizes up to 25 bit.

As other memory tagging solutions, CrypTag divides the memory space into data granules of a fixed size and associates the data granules and their referencing pointers with pseudorandom tag values. However, instead of storing the data tags in an isolated physical location, CrypTag implicitly binds the granule data to the tag by including it in the tweak of the memory encryption engine. Additionally, CrypTag introduces two main modifications to the cache subsystem: it embeds data tag values directly into cache lines, and it requires that a cache hit occur only when the pointer tag matches the data granule tag stored in the corresponding line. The security properties provided by CrypTag depend on the encryption mode used:

1. Authenticated Encryption: Synchronous detection of memory safety violations independent of use case
2. Encryption only (like CTM): Asynchronous, indirect detection of memory safety violations due to the use of pseudorandom values resulting from decryption with a mismatching tag (e.g., function pointer overwritten with wrong tag $\rightarrow$ fault due to non-executable address)

CrypTag adds one new instruction for adding a random tag to a pointer to the ISA, but the authors note that additional instructions are required to support certain pointer operations efficiently (subtraction, adding, shifting, and multiplication).

CHERI

A different option to achieve memory safety is to use capabilities that restrict the memory accessible through pointers. CHERI extends existing ISAs (e.g., RISC-V [81]) to include hardware support for capabilities, turning pointers into “fat pointers” enriched with metadata. CHERI capabilities enforce bound checks and access permissions, protect pointer integrity through unforgeable tags, and ensure monotonicity (access permissions can only be restricted, never broadened). This enables deterministic, synchronous, and fine-grained spatial memory safety and enables software to efficiently enforce temporal memory safety.

Although CHERI’s protection model offers many advantages over the probabilistic memory tagging schemes discussed above, achieving these benefits requires substantial changes to both the hardware (e.g., protected capability storage in registers and memory, capability-aware load and store pipelines, and a program-counter capability) and software architecture (e.g., compiler, operating system, memory allocators, and new ABIs). In contrast, the introduced memory tagging schemes can be adopted with only minor ISA and software modifications. Consequently, CHERI represents a paradigm shift rather than an incremental security measure, offering stronger guar-

antees but at significantly higher adoption complexity. For this reason, it was not included in Table 3.9.

Criterion	Mechanism	Description
Processor Class	CTM	Embedded 32-bit RISC-V without MMU
	AMTE	High-performance 64-bit Arm A Profile (includes MMU)
	RMTE	64-bit RISC-V with MMU
	CrypTag	
Dependencies	CTM	Zicsr extension, memory encryption engine (NXP-AT IEE, see Section 3.1.1)
	AMTE	Armv8.4-A with FEAT_MTE (mandatory in Armv9-A)
	RMTE	RISC-V Zpm, Svukte, Zimop, Zcmop extensions
	CrypTag	Memory encryption engine
Security Properties	CTM	Assuming a typical 32-bit embedded system with 512 KiB SRAM (e.g., i.MX RT1050), where tagged objects would reside, CTM allows the usage of up to 12-bit tags. This tag size is larger than for AMTE and RMTE, but lower than CrypTag, providing a good compromise balancing memory overhead (tags still embedded in pointers) and security guarantees for 32-bit embedded systems. For CTM, the asynchronous, indirect detection of memory safety violations depends both on the tag size (99.998 % chance that tag does not match), use case and encryption algorithm (see Section 3.1.1). If the tag does not match at a load or store, then decryption returns pseudorandom content ensuring confidentiality. The detection relies on a subsequently failing use case execution (e.g., attacker overwrites function pointer → jumping to non-executable address). Therefore, providing concrete bounds for detection probability and the time until detection requires knowledge of the use case. As CTM includes the tag into the tweak of the memory encryption, it also inherits the security properties of memory encryption.
	AMTE	The 4-bit tags allow synchronous, deterministic detection of sequential memory safety violations if the allocator ensures sequential objects are uniquely tagged. However, wild violations (attacker controls full pointer) can only be detected probabilistically and there 4 bit are insufficient for security-critical systems (93.75 % detection probability).
	RMTE	Similar to AMTE, but supporting 7 bit tags in addition. Hence, it supports a higher detection probability than AMTE (99.22 %), but it is still insufficient for security-critical systems.

Continued on next page

	CrypTag	CrypTag uses asynchronous, indirect detection when the underlying memory-encryption engine operates in encryption-only mode (like CTM), and synchronous, probabilisitic detection when the engine provides authenticated encryption (like AMTE and RMTE). Tag widths up to 25 bit are supported, resulting in the highest detection probability among the compared features.
Scalability	CTM	CTM is supported in a memory region fixed at design (e.g., SRAM where data objects reside). While this does not allow to select protected memory regions at runtime as possible with the other schemes, this is acceptable for embedded systems without an MMU. The tagged and untagged memory region are distinguished using the MSB of a pointer. In contrast to AMTE and RMTE, the tag width can be chosen independently from the memory and bandwidth overhead as long as the tags can still be embedded into pointers, which is important for constrained devices.
	AMTE	AMTE can be enabled per privilege level and for specific memory pages using stage 1 page descriptors, it can also be globally disabled. The memory overhead depends on the tag width and the data granule size as the tags associated with data granules are stored in an isolated memory region. For the supported 4-bit tags and 16 B granules, the overhead amounts to 3.125 % of protected memory (neglecting additional overhead from the stricter alignment requirements). However, adopting 12-bit tags as used in CTM would increase the memory overhead by a factor of three, which is particularly critical in resource-constrained embedded systems where even modest memory growth can lead to disproportionately higher device costs. Furthermore, the increased tag width would also lead to higher memory bandwidth requirements as the tags need to be loaded alongside the data.
	RMTE	Like AMTE, RMTE can be enabled per privilege level and for specific memory pages using stage 1 page descriptors. The memory overhead as a function of tag width and data granule size follows the same relationship as for AMTE. Note that when selecting 7-bit pointer tags, the implementation still allocates 8 bits per tag in memory.
	CrypTag	CrypTag is enabled per pointer, when the tag bits are set to all-zero or all-one (i.e., matching canoncial virtual addresses), then the location is treated as untagged. Like CTM, which was derived from CrypTag, the memory and bandwidth overhead is independent from the tag width.
Hardware Overhead	CTM	Baseline CV32E40S: +0.87 % (96) LUTs, +0.90 % (43) FFs (see Section 3.1.3.7)
	AMTE	Not public, Google expects low-single digit percentage [76]

Continued on next page

	RMTE	No implementation available (likely similar to AMTE)
	CrypTag	Baseline CVA6 + QUARMA (encryption only): +0.67 % (373) LUTs, +0.18 % (57) FFs Baseline CVA6 + ASCON (authenticated encryption): +0.53 % (304) LUTs, +0.14 % (47) FFs
Software Overhead	CTM	Baseline: CV32E40S with NXP-AT IEE running Embench IOT Compiler: LLVM 17, `-Os`, C-library protected Runtime: +0 %-28.46 % (geometric mean: +3.24 %) Memory: • all: +2.32 %-2050 %[1] (geometric mean: +77.38 %) • excluding benchmarks with memory consumption below 512 B: +2.32 %-61.19 % (geometric mean: +15.65 %) Code Size: • full protection: +1.15 %-67.39 %[2] (geometric mean: +17.88 %) • only local objects: +0.1 %-3.8 % (geometric mean: +1.57 %)
	AMTE [57]	Compiler: LLVM 18 Runtime (geometric mean): • Pixel 8 A510 (SPEC CPU2006): +8 % for async and +14 % for sync mode • Pixel 8 A710 (SPEC CPU2006): +18 % for async and +23 % for sync mode • Pixel 8 X3 (SPEC CPU2006): +8 % for async and +56 % for sync mode • Apple M5 (SPEC CPU2017): +2 % for sync mode • Ampere (SPEC CPU2017): +3 % for sync mode Memory: only tag-storage overhead reported (3.125 % of protected physical memory), not actual impact on benchmarks. Code Size: not reported
	RMTE	No implementation available (likely similar to AMTE)
	CrypTag	Compiler: LLVM Runtime (average; on-top of memory encryption): • CVA6 (SPECspeed 2017 Integer, SPECrate 2017 Floating Point): +5.2 % for async and +6.1 % for sync mode • CVA6 (SciMark2): +3.9 % for async and +4.79 % for sync mode • CVA6 (MiBench): +1.5 % for async and +4.9 % for sync mode Memory: not reported Code Size (SPECspeed 2017 Integer): +1.02 %
Software Support	CTM AMTE RMTE CrypTag	Specialized compiler (tagging global and stack objects), adapted heap allocator (tagging and aligning heap objects), and OS (context switch, copy from and to userspace)

Continued on next page

External Debug Support	CTM	Tagged data can only be read or written using the correct tag.
	AMTE	Tag checks can be disabled during debugging, so tagged data can be read and written directly.
	RMTE	
	CrypTag	Tagged data can only be read or written using the correct tag.

Table 3.9: Qualitative comparison of NXP-AT CTM, Arm MTE (AMTE), RISC-V MTE (RMTE), and CrypTag

3.1.3.9 Requirement Compliance

Table 3.10 provides a comparison between the obtained results and the corresponding IP requirements from WP1. The table summarizes the verification outcomes for each requirement and indicates the achieved level of compliance.

REQ-DOC	REQ-ID	REQ-TYPE	REQ-Description	Verification Method	Activities / Justification	Compliance (C / PC / NC)
D1.4	CTM-01	F-HW_ARCH-FEA	CTM shall support sufficiently large tag sizes.	Reviews (specification, code)	Covered by reviews: The tag size depends on the size of the protected memory region and is, for typical embedded systems, significantly larger than the tag sizes provided by the Arm and RISC-V Memory Tagging Extensions (see also Table 3.9).	C
D1.4	CTM-02	NF-HW_ARCH-FEA	The size of tagged pointers shall match the size of regular pointers.	Reviews (specification, code)	Covered by reviews: CTM operations interact with at most one input and one output GPR, demonstrating that pointer values retain their original architectural width.	C

Continued on next page

[1] To protect global objects, CTM introduces redirection and patching tables. Consequently, benchmarks with small global objects and overall small data sections exhibit disproportionately large relative overheads. This effect is reflected in the overhead results after excluding benchmarks whose memory consumption is below 512 bytes.

[2] Global object protection requires the inclusion of fixed-size support routines (290 B) for tagging and patching global objects, indirect access through tagged pointers, and a specialized memmove implementation. Without global object protection, the average code-size overhead would be reduced to 1.57 %.

D1.4	CTM-03	NF-HW_ARCH-PR	CTM shall have negligible impact on code size.	Benchmarks (Embench IoT)	Not met, as the average code-size overhead was 17.88 %, primarily due to the additional complexity of protecting global objects. Global object protection requires the inclusion of fixed-size support routines (290 B) for tagging and patching global objects, indirect access through tagged pointers, and a specialized memmove implementation. Without global object protection, the average code-size overhead would be reduced to 1.57 %.	NC
D1.4	Target Performance		The average runtime overhead shall be up to 6 %.	Benchmarks (Embench IoT)	Achieved based on benchmarking results (3.24 %)	C
D1.4	Target Performance		The achievable clock frequency on the FPGA shall be greater than or equal to 50 MHz.	Synthesis Results	Even the unmodified baseline platform on the Genesys 2 FPGA only marginally achieved this frequency. The intended objective was to ensure that integration of the NXP-AT extensions would not reduce the operating frequency of the CORE-V-MCU baseline platform on the Genesys 2 FPGA (20 MHz). This objective was achieved.	NC

Table 3.10: Requirement Compliance Matrix CTM

3.1.4 Enclave Memory Isolation (EMI) – NXP-AT

3.1.4.1 IP Card

Basic Info		
IP name	Enclave Memory Isolation (EMI)	
License	Closed-source proprietary	
Repository	N/A	
Architecture		
Clock	Number of clock domains	1
	Synchronous with system	Y
	Clock generated internally	N
Ctrl Interface	ISA extension?	Y (RV32I)
	Memory mapped?	N
	Protocol	N/A
	Address Map	N/A
Initiator Interface	Protocol	
	Cached?	N/A
	IOMMU?	
Interrupts	Interrupts	N
Microarchitecture		
Parametrization	Parametric no. cores?	N
	Parameteric config?	Y
Programmability	Contains programmable cores?	N
	ISA	N/A
Software		
Compiler	Requires specialized compiler?	N
	Compiler repository	N/A
Hardware Abstraction Layer	N	
High-level API	Is there a high-level API/SDK?	N
	SDK repository	N/A
	Is there a domain-specific compiler?	N
Integration		
IP Distribution	Manifest type (if any)	N
	Standalone simulation?	N
	(if standalone sim) SW requirements?	N/A
	Integration documented / examples?	Y, closed-source proprietary
Synthesis	Is the IP synthesizable?	Y
	FPGA synthesis example available?	Y, closed-source proprietary
	ASIC synthesis example available?	Y, closed-source proprietary
Simulation	Closed-source simulation?	Y (Xcelium)
	Open-source simulation?	N
Evaluation	PPA results available?	Y

3.1.4.2 General Information

Modern systems often run a range of workloads on one physical general-purpose processor as it is cost-efficient to reuse existing infrastructure. These workloads may include sensitive assets and their vendors may not trust each other. Hence, to guarantee the confidentiality and integrity

of the assets, it is necessary to isolate the workloads from each other. Traditionally, this isolation is realized by dedicated hardware components (memory protection and management units) that enforce that tasks in lower privilege modes can only access allowed regions. The operating system, which runs in a higher privilege mode, sets up the memory regions and configures suitable access permissions for each task.

However, research has shown that this logical isolation is insufficient against a range of physical attacks techniques (e.g., probing the memory bus, injecting faults flipping bits in memory [70, 47], or injecting faults affecting the memory protection configuration by the operating system [56]). Adding a memory encryption engine that encrypts all data before it is stored in memory mitigates the first attack example, raises the bar for the second as injecting controlled modifications is harder, but does not help against the third example without additional countermeasures.

As this deliverable provides updated information about the IP state since deliverable D3.2, please refer to the earlier deliverables for additional information about the Enclave Memory Isolation (EMI) module.

3.1.4.3 Purpose and Scope

As described in the previous section, logical isolation is insufficient to protect sensitive information in a malicious environment. Hence, the purpose of the EMI module is to enable workload-specific memory encryption for RISC-V cores together with NXP-AT's Inline Encryption Engine (IEE) module (see Section 3.1.1). Further, the design of the EMI module mitigates the impact of selected fault attacks including skipping instructions during the context switch. Combining the classical logical isolation and the cryptographic isolation provided by the EMI module results in stronger security guarantees (i.e., mitigation of the previously explained physical attacks). These stronger guarantees benefit use cases requiring strong isolation between workloads like trusted execution environments. Specifically, EMI enables a security monitor executing in machine mode to cryptographically isolate confidential workloads running at lower privilege levels by assigning each workload a unique encryption context.

3.1.4.4 Refined architecture description

The overall architecture of the EMI extension remains unchanged compared to deliverable D3.2 and is illustrated in Figure 3.4. Elements of the CV32E40S core affected by the EMI integration are highlighted in orange. The following components have been updated since deliverable D3.2:

- The refined EMI extension introduces an additional CSR, `memictl`. This CSR allows load or store instructions executed in M-mode to be associated with the memory-access modifiers of the privilege level encoded in `mstatus.MPP`, while leaving PMP access checks unaffected. This mechanism enables M-mode to copy data between memory-encryption contexts without repeatedly switching `mstatus.MPRV`.

3.1.4.5 Interfaces

The EMI extension is tightly integrated in the CV32E40S processor and internally connected to NXP-AT's IEE modules.

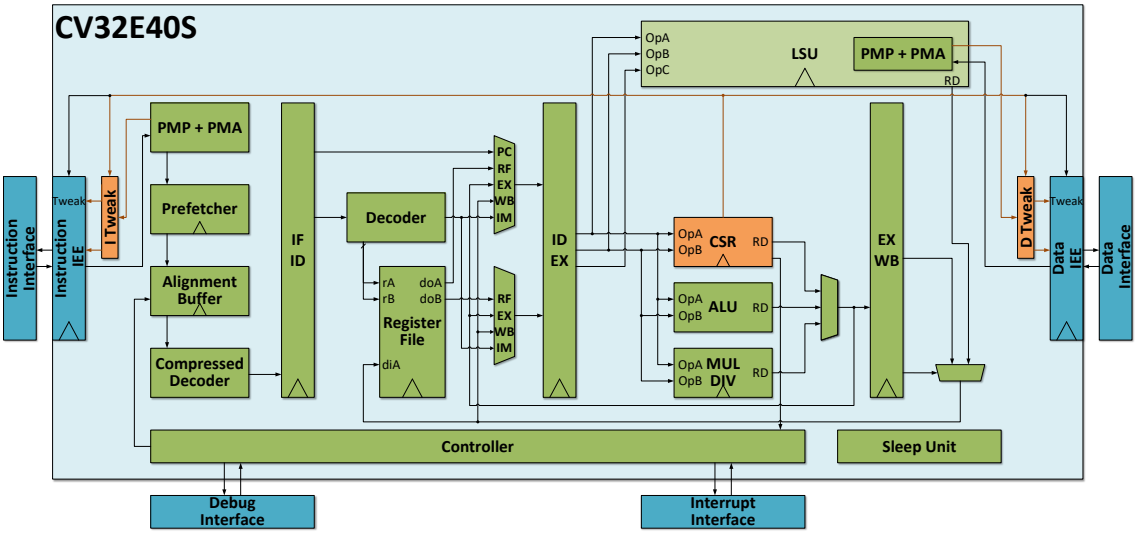


Figure 3.4: Architecture of the EMI extension

3.1.4.6 ISA specialization

The EMI extension adds multiple CSRs (`mfetchmod{0,1}`, `ufetchmod{0,1}`, `mloadmod{0,1}`, `uloadmod{0,1}`, `mstoremod{0,1}`, `ustoremod{0,1}`) representing modifiers that are combined to an enclave tweak depending on the current privilege level and access type (load, store, fetch), as shown in Figure 3.5. Further, the CSR `memictl` is added to switch the memory-access modifiers used by M-mode load and store instructions to those of the privilege level encoded in `mstatus.MPP`, while leaving PMP access checks unaffected.

Table 3.11 shows the addresses assigned for the EMI CSRs after integration with the CV32E40S core.

3.1.4.7 Evaluation prototype

The evaluation prototype and available evaluation environments are the same as described in Section 3.1.1.3 for NXP-AT's IEE module.

A suite of self-checking tests has been written for the EMI extension. These tests can be executed in both of the supported environments.

ASIC

PPA analysis using a 16 nm process with worst timing, targeting a processor clock frequency of 500 MHz (the highest frequency before area increases significantly), shows a 7.91 % (absolute: 3764 GE) area increase caused by the EMI extension compared to the base CV32E40S core.

CSR	Index	Privilege	Field	Bit(s)	Function
mfetchmod0	0x7C0	MRW	-	-	LSW of fetch modifier for M-mode
mfetchmod1	0x7C1	MRW	-	-	MSW of fetch modifier for M-mode
mloadmod0	0x7C2	MRW	-	-	LSW of load modifier for M-mode
mloadmod1	0x7C3	MRW	-	-	MSW of load modifier for M-mode
mstoremod0	0x7C4	MRW	-	-	LSW of store modifier for M-mode
mstoremod1	0x7C5	MRW	-	-	MSW of store modifier for M-mode
ufetchmod0	0x820	MRW	-	-	LSW of fetch modifier for U-mode
ufetchmod1	0x821	MRW	-	-	MSW of fetch modifier for U-mode
uloadmod0	0x822	UW	-	-	LSW of load modifier for U-mode
uloadmod1	0x823	UW	-	-	MSW of load modifier for U-mode
ustoremod0	0x824	UW	-	-	LSW of store modifier for U-mode
ustoremod1	0x825	UW	-	-	MSW of store modifier for U-mode
memictl	0x7C6	MRW	LPRV	0	Associate M-mode loads with the modifiers of the privilege level encoded in `mstatus.MPP`
			SPRV	1	Associate M-mode stores with the modifiers of the privilege level encoded in `mstatus.MPP`

Table 3.11: CSRs introduced by the EMI extension

FPGA

The evaluation prototype was tested on the Genesys 2 FPGA board (featuring a XC7K325T-2FFG900C FPGA IC). The design was synthesized using Vivado v2025.2, targeting a frequency of 20 MHz. The EMI extension increased the usage of lookup tables by 13.51 % (absolute: 1490) and registers by 13.38 % (absolute: 642) compared to the base CV32E40S core.

3.1.4.8 State-of-the-Art Comparison

In this section, the NXP-AT EMI extension is compared with Arm Memory Encryption Contexts (MEC) [9, Ch. A2.3.4, Ch. D7.4, Ch. D8.12] and the RISC-V Confidential VM Extension (CoVE) for Confidential Computing [61].

The NXP-AT EMI extension targets deeply embedded 32-bit processors without MMUs and hardware virtualization, whereas the MEC extension is part of the 64-bit Arm A-profile architecture and integrated with both of these features. Furthermore, EMI provides architectural protection against certain fault injection attacks and enables more efficient access to multiple shared memory regions. The Arm M-profile is closer to the processor class targeted by the EMI extension, but lacks workload-specific memory encryption.

RISC-V CoVE mentions workload-specific memory encryption, but leaves the design and realization of such mechanisms as implementation-specific. Currently, a RISC-V International task group is assembled that will develop specifications for the control and signaling of memory encryption contexts, but no more design details are available yet. Hence, CoVE is not included in Table 3.12.

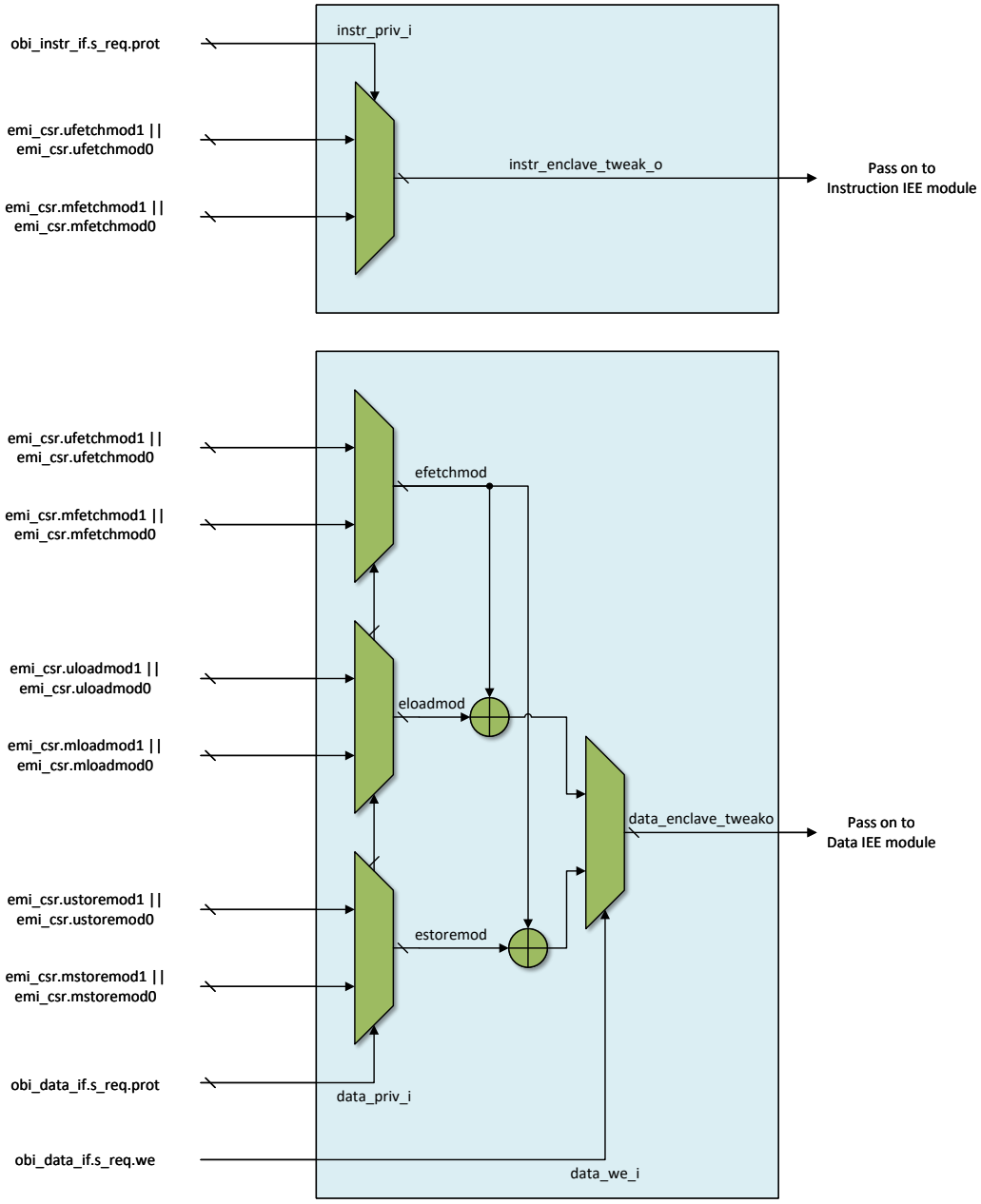


Figure 3.5: EMI enclave tweak formation logic

Table 3.12 summarizes the comparison of NXP-AT EMI and Arm MEC. Because they target different processor classes and offer varying security guarantees and features, a purely quantitative comparison of hardware or software overhead is often not meaningful. Already, the microarchi-

tecture of the baseline processor may differ considerably, e.g., in-order versus out-of-order or single-issue versus dual-issue, making direct numerical comparisons problematic. Therefore, Table 3.12 focuses primarily on qualitative aspects, while numerical values are included only where available in the literature to provide a rough, high-level orientation.

Additional background information on the introduced features can be found in the following paragraphs.

Arm

The Arm MEC extension provides optional support for workload-specific memory encryption for the 64-bit Arm A-profile architecture starting with Armv9.2. It builds on the optional Realm Management Extension (RME) introduced with Armv9.1 [9, 12, Ch. A2.3.3]. Like EMI, MEC requires the presence of a memory encryption engine in the system, called Memory Protection Engine (MPE) in Arm's terminology. RME forms a core component of the Arm Confidential Compute Architecture (CCA) [11], which protects sensitive code and data within Realm VMs from observation or tampering.

Figure 3.6 shows the software components involved in RME and indicates the hardware security boundaries that prevent untrusted software components from accessing Realm VMs executing at EL1 and EL0. RME isolates Realm VMs from untrusted software components by introducing additional security states (*Realm* and *Root*) alongside the existing ones, and by enforcing access control through a dedicated Granule Protection Checker (GPC) that enables dynamic assignment of physical memory granules to different security states. The isolation between Realm VMs executing at EL1 and EL0 in the Realm State, necessary as their owners may not trust each other, is achieved using hardware virtualization configured by the trusted Realm Management Monitor (RMM) executing at EL2 in the Realm State. Additionally, the memory regions in the Realm physical address space associated with different Realms can be encrypted using unique memory encryption contexts to mitigate certain physical attacks. The active memory encryption context is selected using a Memory Encryption Context Identifier (MECID) programmable using system registers and page descriptor bits.

The RMM owns the configuration of all security primitives required to ensure the isolation between Realm VMs (e.g., stage 2 translation and MECIDs). In addition, the lightweight and trusted RMM performs all actions that must access sensitive assets of confidential workloads, such as context switching or attestation. The RMM exposes this functionality as a service to a larger, untrusted hypervisor executing in the Non-secure State, which controls certain aspects of Realms (e.g., creation, scheduling, resource allocation) but cannot access sensitive data. This architecture makes it possible to reuse established hypervisors while reducing the attack surface to the smaller RMM. Note that the untrusted hypervisor may manage multiple untrusted VMs like the RMM may manage multipe Realms. As in previous Arm architectures, the Secure State can be used to host trusted services and is isolated from the Realm State and Non-secure State. The context switches between the different states are handled by the trusted Monitor executing in the Root State.

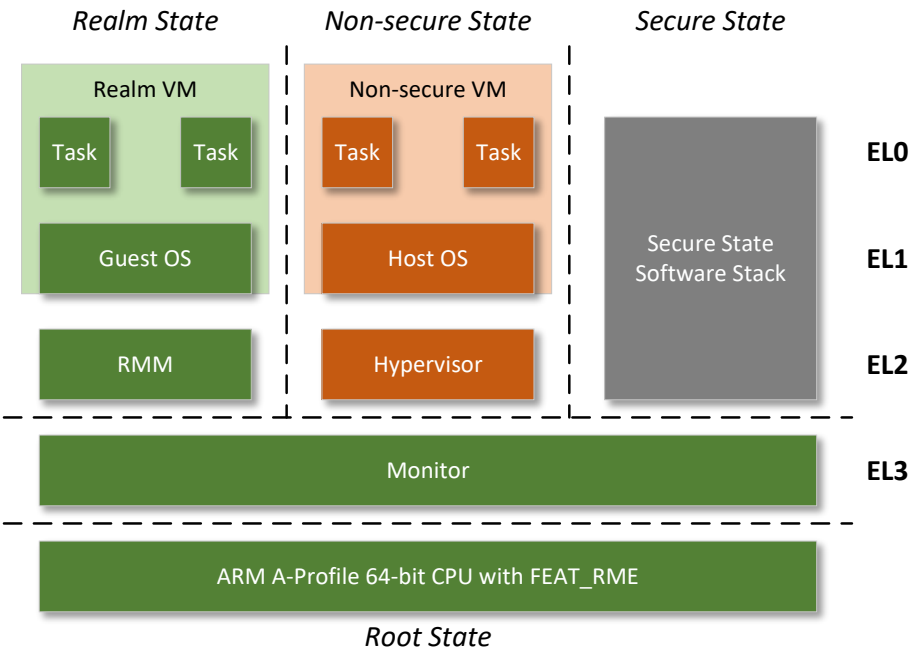


Figure 3.6: Arm RME Overview

RISC-V

The Confidential VM Extension (CoVE) for Confidential Computing, currently under development, addresses the same problem as Arm CCA and uses a similar architecture (see Figure 3.7). The confidential workloads are deployed as virtual machines, called TEE Virtual Machines (TVMs), which are isolated using hardware virtualization configured by a trusted HS-mode component called TEE Security Manager (TSM). Similar to Arm CCA, the untrusted host OS or VMM makes decisions about the TVM's lifecylce state or resource allocation which are enforced by the trusted TSM limiting the attack surface. The untrusted domain (host OS and non-confidential VMs) and confidential domain (TSM and TVMs) execute at the same privilege levels (HS, VS or VU) and must be isolated by an appropriate security primitive owned by the TSM-driver executing at machine mode (e.g., MTT checker described by Smmtt extension [68] or PMP). Additionally, the memory regions associated with TVMs may be encrypted with unique memory encryption keys.

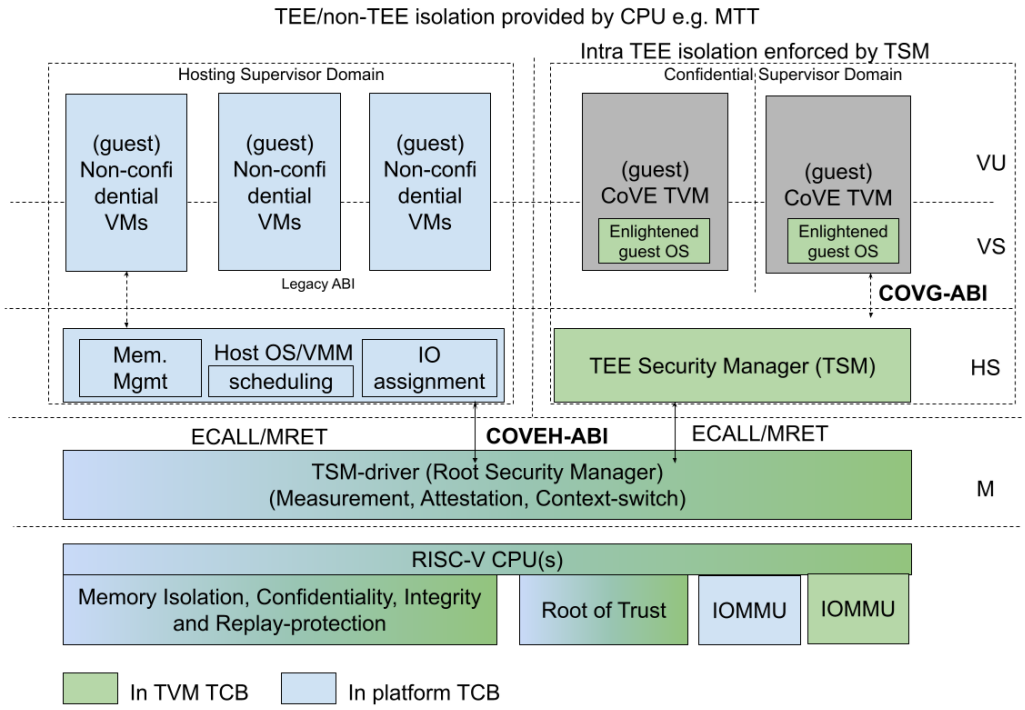


Figure 3.7: RISC-V CoVE Overview by RISC-V International, licensed under CC BY 4.0

The CoVE specification notes that workload-specific memory encryption may be required for certain threat models and outlines a set of corresponding requirements, while explicitly leaving the design and realization of such mechanisms as implementation-specific. At the time of writing, a dedicated task group, the Memory Encryption Contexts Task Group [67], is forming under the RISC-V International umbrella and is expected to develop specifications for the control and signaling of memory encryption contexts. Consequently, this deliverable cannot provide further details in this area.

Criterion	Mechanism	Description
Processor Class	EMI	Embedded 32-bit RISC-V without MMU and virtualization
	MEC	High-performance 64-bit Arm A Profile (supports MMU and virtualization)
Dependencies	EMI	Zicsr extension, memory encryption engine (NXP-AT IEE, see Section 3.1.1)
	MEC	Armv9.2-A with FEAT_RME, memory encryption engine

Continued on the next page

Security Properties	EMI	The security level depends on the width of the tweak modifiers (`{m,u}{fetch,load,store}X`; 60 bit in ISOLDE), the number of active encryption contexts and the encryption algorithm. Because EMI allows a protected workload to exchange its own load and store modifiers, the effective security decreases as the number of active encryption contexts grows. With more contexts, it becomes increasingly feasible for an attacker to guess a valid modifier associated with a confidential workload. The design of EMI reduces the feasibility of certain physical attacks targeting its configuration (i.e., exploitable fault attacks must change both the fetch and load or store modifier in a controlled manner; one is not enough) and selected logical attacks (i.e., code injected using a buffer overflow cannot be executed as code and data regions use different encryption contexts).
	MEC	The security level depends on the size of the key or tweak representing an encryption context and the encryption algorithm but is independent from the number of active encryption contexts, as the protected workload cannot modify its own MECIDs according to the threat model. Unlike EMI, the design of MEC offers no additional protection against physical or logical attacks. However, other security features present in processors implementing MEC may reduce the feasibility of such attacks.
Scalability	EMI	In general, the number of active encryption contexts is limited by the width of the tweak modifiers. However, as described above, the chosen width must take into account that the security level decreases as the number of active encryption contexts increases. For the expected EMI application domain, which primarily targets embedded use cases, this impact is limited in practice because only a small number of encryption contexts are typically active concurrently. EMI allows a protected workload to exchange its own load and store modifiers, allowing efficient access (i.e., without a context switch to the security monitor) to data associated with different encryption contexts (e.g., useful for multiple shared memory regions).

Continued on the next page

	MEC	The number of active encryption contexts is limited by the MECID width (up to 16 bit) and the number of encryption contexts supported by the MPE. MEC allows the configuration of a primary and alternative MECID; the active one can be selected using a field in the page descriptors set up by the RMM. While this scheme enables efficient access to data associated with two encryption contexts, it is less flexible than EMI. However, if two encryption contexts suffice, it incurs a lower average runtime overhead because no system register needs to be reprogrammed.
Hardware Overhead	EMI	See Section 3.1.4.7
	MEC	Not public
Software Overhead[1]	EMI	Baseline: CV32E40S with NXP-AT IEE running OpenMZ Compiler: LLVM 18, `-Os` Runtime: • Context Switch: – ISA (theoretical minimum): 10 instructions – Cycles: +51 (+17.2 %) on top of OpenMZ context switch • Shared Memory: – Cycles: constant +12 cycles regardless of copy size (the relative overhead is negligible for larger copies) Memory: 16 B per encryption context
	MEC	Runtime: • Context Switch: – ISA (theoretical minimum): 2 instructions (no SHM), 4 instructions (one SHM region) • Shared Memory (one region): No overhead Memory: 2 B per encryption context
Software Support	EMI	Requires a security monitor extension for initialization and context switching
	MEC	Similar to EMI: Requires RMM (trusted part of hypervisor) extension for initialization and context switching
External Debug Support	EMI	Requires a debug authentication extension to grant debug access to selected encryption contexts and either a debug module extension to access data for those contexts via the interconnect or a debugger plugin to access data from inactive encryption contexts via the processor.
	MEC	Like EMI [10]

Table 3.12: Qualitative comparison of NXP-AT EMI and Arm MEC

[1] Only the overheads for context switching and accessing shared memory are shown, as these operations occur periodically, while the initial setup is expected to happen sporadically. The runtime overhead resulting from memory encryption is evaluated in Section 3.1.1.

3.1.4.9 Requirement Compliance

Table 3.13 provides a comparison between the obtained results and the corresponding IP requirements from WP1. The table summarizes the verification outcomes for each requirement and indicates the achieved level of compliance.

REQ-DOC	REQ-ID	REQ-TYPE	REQ-Description	Verification Method	Activities / Justification	Compliance (C / PC / NC)
D1.4	EMI-01	F-HW_ARCH-FEA	EMI supports cores without MMU.	Reviews (specification, code)	Covered by review confirming target core includes no MMU	C
D1.4	EMI-02	F-HW_ARCH-FEA	EMI provides cryptographic isolation.	Testbench (RTL simulation, FPGA)	Covered by passing unit tests	C
D1.4	EMI-03	NF-HW_ARCH-FEA	Employing EMI shall need no compiler adaptions.	Testbench (RTL simulation, FPGA)	Covered by passing unit tests	C
D1.4	EMI-04	F-HW_ARCH-S	Enclaves shall be able to gain access to granted shared memory regions without continuous involvement of the security monitor.	Testbench (RTL simulation, FPGA)	Covered by passing unit tests	C
D1.4	EMI-05	NF-HW_ARCH-FEA	EMI shall offer architectural resilience against physical fault attacks causing instruction skips.	Testbench (RTL simulation, FPGA)	Covered by passing unit tests	C

Continued on the next page

D1.4	Target Performance	The average runtime overhead for context switches shall be up to 5 %.	Benchmarks (OpenMz)	The measured average context-switch runtime overhead of 17.2 % exceeds the target significantly. However, this figure is strongly influenced by OpenMZ's low baseline latency (296 cycles). Initial explorations considered Keystone as security monitor, whose substantially larger baseline context-switch cost (1.8 K to 2.6 K cycles) would have resulted in considerably lower relative overheads. Moreover, context switching usually accounts for only a small fraction of execution time, limiting the impact on overall performance.	NC
D1.4	Target Performance	The achievable clock frequency on the FPGA shall be greater than or equal to 50 MHz.	Synthesis Results	Even the unmodified baseline platform on the Genesys 2 FPGA only marginally achieved this frequency. The intended objective was to ensure that integration of the NXP-AT extensions would not reduce the operating frequency of the CORE-V-MCU baseline platform on the Genesys 2 FPGA (20 MHz). This objective was achieved.	NC

Table 3.13: Requirement Compliance Matrix EMI

3.1.5 Forward-Edge Control Flow Integrity (FCFI) – NXP-AT

3.1.5.1 IP Card

Basic Info		
IP name	Forward-Edge Control Flow Integrity (FCFI)	
License	Closed-source proprietary	
Repository	N/A	
Architecture		
Clock	Number of clock domains	1
	Synchronous with system	Y
	Clock generated internally	N
Ctrl Interface	ISA extension?	Y (RV32I)
	Memory mapped?	N
	Protocol	N/A
	Address Map	N/A
Initiator Interface	Protocol	
	Cached?	N/A
	IOMMU?	
Interrupts	Interrupts	N
Microarchitecture		
Parametrization	Parametric no. cores?	N
	Parameteric config?	Y
Programmability	Contains programmable cores?	N
	ISA	N/A
Software		
Compiler	Requires specialized compiler?	Y
	Compiler repository	N/A (closed-source proprietary)
Hardware Abstraction Layer	N	
High-level API	Is there a high-level API/SDK?	N
	SDK repository	N/A
	Is there a domain-specific compiler?	N
Integration		
IP Distribution	Manifest type (if any)	N
	Standalone simulation?	N
	(if standalone sim) SW requirements?	N/A
	Integration documented / examples?	Y, closed-source proprietary
Synthesis	Is the IP synthesizable?	Y
	FPGA synthesis example available?	Y, closed-source proprietary
	ASIC synthesis example available?	Y, closed-source proprietary
Simulation	Closed-source simulation?	Y (Xcelium)
	Open-source simulation?	N
Evaluation	PPA results available?	Y

3.1.5.2 General Information

Systems operating in potentially malicious environments are subject to logical and physical attacks. Fault injection attacks based on optical, electromagnetic, clock, or voltage glitches are a form of active physical attacks. The injected disturbances can cause different effects in a pro-

cessor depending on the affected logic and form of the fault signal. For example, the processor may skip instructions or execute altered instructions. Attackers exploit these effects to bypass security measures, e.g., by skipping their configuration or enablement. Research demonstrated various successful attacks exploiting instruction skips introduced by fault attacks. Among those are bypassing signature verification to load malicious firmware [22] or skipping the reconfiguration of memory protection units to gain access to protected data [56].

3.1.5.3 Purpose and Scope

The FCFI extension ensures the integrity of the instruction stream by calculating a running checksum over the executed instructions and regularly comparing the current checksum value with pre-computed reference values. If a mismatch is detected, then a software check exception is raised. Hence, the FCFI extension enables the detection of fault injection attacks that aim to modify the instruction stream.

Even with sophisticated fault injection attacks, such as single-spot laser fault injection, it is challenging to arbitrarily alter multiple instructions. The potential outcomes are probabilistic and their probabilities depend on the configuration of the fault injection setup and the technology node used to manufacture the IC. Further, considering a single-spot laser fault injection setup, an attacker would need to realign the laser to prepare the next fault injection, which is not feasible before the next checksum check. Consequently, even for advanced attackers it is hard to hide introduced errors by trying to modify subsequent instructions before the next checksum check occurs.

Further, the FCFI extension also includes instructions that allow implementing landing pads. Hence, it can also detect modifications of forward-edge control flow transfers (indirect calls) by logical attacks. Note that also the *Zicfilp* RISC-V extension offers similar landing pads, but it cannot be efficiently combined with FCFI's instruction stream integrity feature.

However, the FCFI extension cannot ensure the integrity of the data memory. Hence, it also cannot mitigate attacks aiming to modify backward-edge control flow transfers (i.e., altering function return addresses on the stack). To reduce the attack surface further, it can be combined with NXP-AT's BCFI module described in Section 3.1.2.

The FCFI extension is optimized for low hardware and software overhead while mitigating fault-injection attacks up to and including single-spot laser injection.

3.1.5.4 Refined architecture description

The overall architecture of the FCFI extension remains unchanged compared to deliverable D3.2.

To support nested traps, the `MPEXCP` field has been added to the `ucsstate` CSR. Upon entering a trap to M-mode, `MPEXCP` is set to the current value of `EXCP`, after which `EXCP` is set to suspend FCFI operation. When executing `mret`, the value stored in `MPEXCP` is used to restore the `EXCP` state that was active prior to the trap.

Further, the `CSB` field has been added to the `ucsstate` CSR, allowing FCFI branch instructions to be enabled only when necessary to reduce code size overhead. When disabled, the freed instruction encoding space can be repurposed for instructions to be delegated to a coprocessor via the XIF interface.

3.1.5.5 Interfaces

The FCFI extension is tightly integrated in the CV32E40S processor.

3.1.5.6 ISA specialization

The FCFI extension modifies the ISA of the CV32E40S core. Since deliverable D3.2, the `MPEXCP` and `CSB` fields have been added to the `ucsstate` CSR, as described in the previous section.

Table 3.14 shows the addresses assigned for the FCFI CSRs after integration with the CV32E40S core (registers accessible by all privilege levels and zero initialized).

CSR	Index	Field	Bits	Function
`ucsstate`	0x818	`MPEXCP`	28	Saves the current value of `EXCP` upon entering a trap to M mode
		`EXCP`	25	Set upon entering a trap (suspends FCFI) and restored using `MPEXCP` upon exiting
		`CSB`	19	Enable FCFI branch instructions
		`LPE`	18	Landing pads are enabled
		`LPX`	17	Landing pad is expected
		`CSE`	16	Instruction stream integrity is enabled
		`CURCRC`	15...0	Current CRC value
`ucsrefcrc`	0x819	`VALID4`	18	Bits 3...0 of `ucsrefcrc.VALUE` are valid
		`VALID5`	17	Bits 4...0 of `ucsrefcrc.VALUE` are valid
		`VALID11`	16	Bits 15...5 of `ucsrefcrc.VALUE` are valid
		`VALUE`	15...0	Collected reference CRC value
`ucslabel`	0x81A	`VALID5`	17	Bits 4...0 of `ucslabel.VALUE` are valid
		`VALID11`	16	Bits 15..5 of `ucslabel.VALUE` are valid
		`VALUE`	15...0	Label value
`ucswdog`	0x81C	`WDE`	16	Watchdog for ensuring regular CRC checks is enabled
		`INIT`	15...8	Initial value for watchdog
		`CUR`	7...0	Current value of watchdog
`ucsinitmask`	0x81D	`MASK`	15..0	XOR-mask for initializing the current CRC
`ucscontrol`	0x81B	`NEEDLPLABEL`	5	Enforce usage of labels for landing pads
		`NEED11`	1	Need at least 11 reference bits
		`NEED5`	0	Need at least 5 reference bits

Table 3.14: CSRs introduced by the FCFI extension

3.1.5.7 Evaluation prototype

The evaluation prototype and available evaluation environments are the same as described in Section 3.1.1.3 for NXP-AT's IEE module.

A suite of self-checking tests has been written for the FCFI extension. These tests can be and were executed in both of the supported environments.

ASIC

PPA analysis using a 16 nm process with worst timing, targeting a processor clock frequency of 500 MHz, shows a 10.64 % (absolute: 5102 GE) area increase caused by the FCFI extension compared to the base CV32E40S core.

FPGA

The evaluation prototype was tested on the Genesys 2 FPGA board (featuring a XC7K325T-2FFG900C FPGA IC). The design was synthesized using Vivado v2025.2, targeting a frequency of 20 MHz. The FCFI extension increased the usage of lookup tables by 8.83 % (absolute: 974) and registers by 9.25 % (absolute: 444) compared to the base CV32E40S core.

3.1.5.8 State-of-the-Art Comparison

In this section, NXP-AT FCFI is compared with other signature-based and scrambling-based CFI schemes, as well as RISC-V and Armv8.1M landing pads.

Table 3.15 summarizes the comparison of signature-based and scrambling-based CFI schemes, and RISC-V and Armv8.1M landing pads. Because the mechanisms offer varying security guarantees and features, a purely quantitative comparison of hardware or software overhead is often not meaningful. Therefore, Table 3.15 focuses primarily on qualitative aspects, while numerical values are included only where available in the literature to provide a rough, high-level orientation.

Additional background information on the introduced features can be found in the following paragraphs.

Signature-Based Schemes

Existing work describes a range of schemes that ensure execution integrity by computing and verifying signatures. Examples include Werner et al. [83], SCI-FI [25], MAFIA [26] and NXP-AT FCFI developed within ISOLDE. These schemes maintain a running signature that is updated by each executed instruction and periodically compare the current signature against the reference values precomputed at compilation. Because the inserted checks directly detect tampering attempts, and their execution frequency can be monitored by a watchdog, the detection latency can be bounded. Existing proposals mainly differ in coverage and granularity (e.g., protection of microarchitectural control signals, constraints on indirect control-flow transfers) and the hardware/software overheads they incur. NXP-AT FCFI focuses on minimizing hardware and software overhead by introducing custom control-flow instructions that also load reference bits, and it offers a scalable security level that allows selectively trading security for reduced overhead.

MAFIA [26] is a recent signature-based control flow integrity scheme and builds on the concepts of previous work [83] [25]. It provides instruction integrity (protecting instruction words in memory), control-signal integrity (covering pipeline control signals influenced by instruction words), sequential execution integrity (ensuring correct order and completeness of instruction execution within basic blocks), forward-edge and coarse-grained backward-edge CFI. MAFIA uses suitable control signals from the decode stage for signature computation which has the advantage that also fault injection attacks on the decode stage can be detected. Further, MAFIA uses typical hardware countermeasures like redundancy schemes to protect control signals that could not have been integrated in the signature computation (e.g., control signals depending on data like

branch decisions which cannot easily be modeled during compilation) or are associated with later stages. The advantage of integrating control signals in the signature computation over traditional countermeasures is the reduced hardware overhead.

At the ISA level, MAFIA uses the signature of the previous basic block as the initial signature value for the current basic block. Each instruction within a basic block updates the running signature, and the signature is compared against reference values using dedicated control-flow transfer instructions. If a basic block has multiple predecessors, the final signature value of all but one predecessor must be patched using a dedicated instruction so that all paths converge to a common value. The reference values are stored directly after the instructions that perform a signature check, while patch values are stored in a dedicated memory section. To achieve full control-flow protection, MAFIA also protects indirect calls and returns (other indirect branches are eliminated using compiler options) by deriving unique initial signature values for valid target basic blocks. Indirect call sites are replaced by a dispatcher gadget that uses direct calls to change control flow to the correct target and applies target-specific patches to establish the expected signature state. The set of valid targets is determined by the function prototype. Because basic blocks following call sites with identical function prototypes share the same signature, MAFIA enforces only coarse-grained backward-edge control-flow integrity. MAFIA supports traps by associating each trap with a unique initial signature value stored in a table similar to an interrupt vector table. Upon trap entry, the current signature is saved and the trap-specific signature is loaded from the table. Similarly, upon trap return, the previously saved signature is restored.

The security level of MAFIA depends on the signature function: the authors analyze implementations using a CBC-MAC based on PRINCE (keyed, hence offering code authenticity) and CRC32 (less overhead, but invertible). While CRC32 detects up to 8-bit flips in the input sequence, it is not a cryptographic primitive and an advanced adversary can potentially craft collisions, so it provides weaker guarantees than the keyed MAC construction.

Scrambling-Based Schemes

In contrast to signature-based schemes, scrambling-based schemes transform instruction words dynamically based on execution history. The scrambling may depend on previous instructions, data-independent pipeline control signals, a secret key, or a combination of those. The execution history is tracked implicitly in the internal state of the scrambling and unscrambling primitives. As with signature-based schemes, patching the unscrambler state is required when a basic block has multiple predecessors. If an instruction word or a control signal is tampered with, the unscrambler enters an unexpected state, producing pseudorandom values when unscrambling subsequent instructions. For sparse ISAs such as RISC-V RV32IM, only a small fraction of all bit patterns correspond to valid instructions, so pseudorandom code typically triggers an exception after a small number of instructions. Unlike signature-based schemes, however, this indirect detection makes it difficult to derive and enforce a strict bound on detection latency. Depending on the scrambling primitive, different security properties can be achieved: constructions using cryptographically secure primitives can additionally provide instruction confidentiality and authenticity, but at higher cost, while lightweight constructions typically offer only instruction and control-flow integrity. Existing proposals mainly differ in coverage, granularity, additional security properties (e.g., protection of microarchitectural control signals, constraints on indirect control-flow transfers, or provision of instruction authenticity), and the hardware/software overhead they incur. If instruction authenticity is cryptographically guaranteed, direct detection of fault injection attacks

becomes possible, similar to signature-based schemes.

Gousselot et al. (Code Encryption for Confidentiality and Execution Integrity down to Control Signals [40]) extend the SCFP approach [82] by providing control signal integrity in addition to instruction confidentiality and integrity, sequential execution integrity, fine-grained forward-edge and coarse-grained backward-edge CFI. It uses a cryptographically secure, sponge-based construction for on-the-fly instruction encryption and decryption based on ASCON or PRINCE, thereby ensuring instruction confidentiality in addition to control-flow integrity. Encryption and decryption depend on previous instructions and data-independent pipeline control signals. Unlike other schemes, patches are accessed in parallel with instructions by indexing a lookup table in a protected memory region using the instruction PC for direct control-flow changes and the target PC for indirect control-flow changes. Hence, no instruction insertions are required, resulting in zero code-size and runtime overhead. However, the number of supported targets without code modifications is limited, and the memory overhead is higher. Further, instead of deriving over-approximated sets of valid indirect control-flow targets using static analysis, dynamic analysis is employed, resulting in tighter but potentially under-approximated sets.

Confidaent [72] provides instruction confidentiality, integrity, and authenticity, as well as sequential execution integrity, coarse-grained forward-edge and fine-grained backward-edge CFI. However, the backward-edge protection is not resilient to replay attacks. It is intended to mitigate fault injection attacks and reuses an existing memory encryption cipher, assumed to be based on ASCON, for authenticated encryption of instructions. The authenticated encryption operates on 128-bit blocks, providing confidentiality, integrity, and authenticity for groups of four instructions. Execution integrity is implemented on top of memory encryption by XORing a 32-bit mask with each instruction. The initial mask of a basic block is randomly generated, and masks for subsequent instructions are derived using an unkeyed permutation. Due to the diffusion property of the permutation, a single bit flip in the mask propagates to affect the entire mask after only a small number of instructions. The masks for the successor basic blocks are set using special instructions executed by predecessor blocks. A separate mask value can be configured for exception handlers to support traps. Masks for indirect jump or call targets are stored at the target location itself, resulting in coarse-grained forward-edge CFI, as any indirect jump or call site may target any indirect jump or call target in the binary. Fault injection attacks are detected via authentication tags generated by ASCON when instruction words in memory are tampered with, or via exceptions triggered by executing subsequent pseudorandom instructions when masks are corrupted or mismatching. Confidaent does not detect tampering with pipeline control signals.

SecDec [48] provides instruction integrity, control signal integrity, sequential execution integrity, and coarse-grained forward-edge and backward-edge CFI, targeting mitigation of fault injection attacks. Instructions are masked using values derived from the previously unmasked instruction (pre-decoder variant) or signals from the decode stage related to the previous instruction (post-decoder variant). To protect the decode stage also when using the pre-decoder variant, the mask is additionally regenerated from data-independent decoder signals associated with the previous instruction and compared against the mask derived from the previously unmasked instruction. Any mismatch triggers an error, enabling direct detection of single fault injections in the decoding logic. Mask generation relies on an unkeyed permutation with strong diffusion properties, ensuring that a single flipped bit in a masked instruction rapidly propagates to all bits of subsequent masks. A fault injection targeting an instruction word results in pseudorandom values for subsequent unmasked instructions and typically leads to an exception after a few instructions, as the RV32

ISA is sparse. To handle basic blocks with multiple predecessors, jumps and taken branches use a static mask. Consequently, any jump or branch target is considered a valid successor for any jump or branch site, resulting in coarser forward-edge CFI enforcement than in other approaches. If a trap occurs, the masking state is automatically saved and restored via a CSR. Detection of fault injection attacks affecting a single instruction word is guaranteed, while carefully coordinated attacks controlling bits in multiple instructions could bypass the scheme, although this is unlikely due to rapid diffusion. Because the permutation is unkeyed, no additional properties such as instruction confidentiality are provided.

RISC-V, Arm

The RISC-V and Armv8.1-M ISAs support landing pads by the Zicfilp extension [65, Sec. 33.1] [66, Sec. 22.1] and the BTI feature [13, Ch. B6][7], respectively. Landing pad enforcement can be enabled per privilege level. When enabled, any instruction that performs an indirect control-flow transfer sets a landing-pad-expected bit in the processor state. The instruction subsequently executed at the target address must be a landing pad instruction, which clears this bit. Otherwise, an exception is raised. If an interrupt or exception occurs between an indirect control-flow transfer and the landing pad instruction, the landing-pad-expected bit is automatically saved upon entry and restored upon return.

The main difference between Zicfilp and BTI is that Zicfilp supports further restricting the set of valid targets using labels. Each valid target may be associated with a 20-bit label, for example, derived from the function prototype or the control-flow graph, and encoded in the landing pad instruction. The expected label must be placed in the general-purpose register `x7` prior to executing an indirect control-flow transfer instruction. The landing pad instruction then checks both the landing-pad-expected state and the labels. This mechanism enables more fine-grained enforcement of forward-edge control-flow integrity than BTI.

Landing pads provide forward-edge control-flow integrity protecting against software attackers attempting to manipulate indirect branch targets stored in memory. However, they do not protect against physical attackers who may skip or modify instructions using fault-injection attacks.

Criterion	Mechanism	Description
Processor Class	Signature-Based	Embedded 32-bit RISC-V
	Scrambling-Based	
	Landing Pads	Embedded 32-bit RISC-V (Zicfilp) or Armv8.1-M (BTI)
Dependencies	Signature-Based	Zicsr
	Scrambling-Based	Zicsr Confidaent: Memory Encryption Engine supporting authenticated encryption
	Landing Pads	BTI: Armv8.1-M with feature PACBTI Zicfilp: Zicsr

Continued on next page

Security Properties	Signature-Based	Threat Model: Physical attacker capable of injecting faults in instruction memory (FCFI: up to single-spot laser fault injection attacks) or pipeline control signals (only MAFIA) Protection Scope: instruction integrity, control signal integrity (only MAFIA), execution integrity within basic blocks, forward-edge CFI covering direct (fine-grained) and indirect control-flow transfers (indirect transfers restricted to functions within same equivalence class, i.e., functions with matching prototype), course-grained backward-edge CFI (only MAFIA), watchdog to enforce checks (only FCFI) Underlying Primitive: FCFI uses a 16-bit CRC that guarantees detection of up to five bit flips within a single instruction word (fault-free instruction words cannot mask error), MAFIA employs a 32-bit CRC that guarantees detection of up to eight bit flips per basic block or a 32-bit non-invertible CBC-MAC additionally providing instruction authenticity Tampering Detection: direct, during signature verification
	Scrambling-Based	Threat Model: Physical attacker capable of injecting faults in instruction memory or pipeline control signals (Gousselot et al., SecDec) Protection Scope: instruction integrity, confidentiality and authenticity (depending on underlying primitive; Confidaent provides all three), execution integrity within basic blocks, control signal integrity (Gousselot et al., SecDec), forward-edge CFI covering direct (Gousselot et al., Confidaent: fine-grained and SecDec: course-grained) and indirect transfers (Gousselot et al.: fine-grained, Confidaent and SecDec: course-grained), backward-edge CFI (Confidaent: fine-grained, Gousselot et al. and SecDec: course-grained) Underlying Primitive: Cryptographically secure, non-invertible constructions (Gousselot et al., Confidaent) offering the highest security level and additional properties, such as instruction authenticity (Confideaent) or lightweight invertible 32-bit permutation with good error diffusion (SecDec) Tampering Detection: direct with authentication tags (Confidaent), else indirect relying on faults generated by corrupted instructions

Continued on next page

	Landing Pads	Threat Model: Software attacker capable of tampering with data memory (e.g., function pointers), physical attacks are out of scope Protection Scope: CFI for forward-edge indirect transfers (restricting control flow to functions within same equivalence class) Underlying Primitive: 1-bit (BTI) or 20-bit (Zicfilp) comparator Tampering Detection: direct during landing pad checks
Scalability	Signature-Based	Security Level: FCFI supports switching between 5-bit, 11-bit and 16-bit security during runtime to reduce ISA overhead, while MAFIA requires decision between 32-bit CRC or CBC-MAC during IP integration Execution Environments: support operation across privilege levels and tasks. FCFI disables protection on traps to allow software-controlled saving and restoration of the FCFI state (resulting in a temporary protection interruption requiring software hardening), MAFIA instead saves the current signature on trap entry and loads a trap-specific signature value (no interruption). FCFI supports efficient selective enablement per function to reduce ISA overhead.
	Scrambling-Based	Security Level: Gousselot et al. requires decision at IP integration, Confidaent supports 20-bit or 32-bit, SecDec fixed Execution Environments: Confidaent and SecDec support operation across privilege levels and tasks by saving their state on trap entry and loading a trap-specific state (no protection interruption), Gousselot et al. refer to future work
	Landing pads	Security Level: 1-bit or 20-bit (only Zicfilp) Execution Environments: support operation across privilege levels and tasks by automatically saving and restoring landing-pad-expected state
Hardware Overhead	Signature-Based	FCFI with baseline CV32E40S (XC7K325T-2FFG900C FPGA 20 MHz): +8.83 % (974) LUTs, +9.25 % (444) FFs FCFI with baseline CV32E40S (16nm 500 MHz): +10.64 % (5102 GE) MAFIA with baseline CV32E40P (GF-22FDX 400 MHz): +6.5 % using CRC32 (3360 GE), +23.8 % using CBC-MAC (12300 GE)

Continued on next page

	Scrambling-Based	Gousselot et al. with baseline CV32E40P (Artix-7 200T FPGA 37 MHz): +72.1 % (7395) LUTs, +66.4 % (3435) FFs Confidaent: not reported (likely similar to Gousselot et al. as both use the ASCON cipher) SecDec with baseline CV32E40P (GF-22FDX): +3.86 % (568 GE) for post-decoder variant, +20.38 % (3002 GE) for pre-decoder variant
	Landing Pads	Not reported for BTI/Zicfilp in isolation (only in combination with shadow stack), due to minimal logic (one instruction, state bit, comparator), standalone overhead is expected to be lower than for the other schemes.
Software Overhead	Signature-Based (Embench IoT)	Compiler: • FCFI: LLVM18, `-Os`, C-library not protected • MAFIA: not reported, `-Os`, C-library not protected, no compressed instructions Runtime: • FCFI CRC11: +1.85 %-15.68 % (geometric mean: +7.49 %) • FCFI CRC16: +0.03 %-15.96 % (geometric mean: +6.47 %) • MAFIA CRC32: +3 %-44 % (geometric mean: +18 %) Code Size: • FCFI CRC11: +1.16 %-22.47 % (geometric mean: +11.62 %) • FCFI CRC16: +2.08 %-37.08 % (geometric mean: +18.49 %) • MAFIA CRC32: +8 %-52 % (geometric mean: +29 %) Memory: No impact

Continued on next page

	Scrambling-Based (Embench IoT)	Compiler: • Gousselot et al.: Not required, uses post-processing tool • Confidaent: LLVM, `-O2` • SecDec: LLVM, GCC, `-O2` Runtime: • Gousselot et al.: 0 % • Confidaent (without mem. enc.): +5.5 %-60.2 % (geometric mean: +25.8 %) for 32-bit masks, +0.1 %-36.2 % (geometric mean: +14.3 %) for 20-bit masks • SecDec: +0.6 %-1.6 % (geometric mean: +1.1 %) Code Size: • Gousselot et al.: 0 % • Confidaent (without mem. enc.): +11.3 %-59 % (geometric mean: +33.5 %) for 32-bit masks, +7.9 %-36.3 % (geometric mean: +22.8 %) for 20-bit masks • SecDec: +1.3 %-2 % (geometric mean: 1.6 %) Memory: • Gousselot et al.: +1047 % using ASCON, +247 % using PRINCE • Confidaent (without mem. enc.): Not reported • SecDec: Not reported
	Landing Pads	Not reported for BTI/Zicfilp in isolation
Software Support	Signature-Based	Compiler: Extension to emit special-purpose instructions OS: Extension for context switching of state
	Scrambling-Based	Compiler: Confidaent and SecDec require compiler extensions for emitting special-purpose instructions and binary scrambling, while Gousselot et al. avoid compiler changes and rely on post-processing. OS: Extension for context switching of state
	Landing Pads	Compiler: Extension to emit special-purpose instructions OS: Extension for context switching of state
External Debug Support	Signature-Based	FCFI suspends operation while in debug mode and requires a debugger extension for stack unwinding (previous enablement state is stored in unused return address bits), disassembling custom instructions, code and control flow changes. MAFIA does not discuss external debug support, but the situation is similar, except for not requiring adaptions of the stack unwinder.
	Scrambling-Based	Not mentioned, but similar to signature-based schemes, additionally the debugger extension must support unscrambling and scrambling to read or modify code (e.g., software breakpoint insertion).

Continued on next page

	Landing Pads	Zicfilp saves and clears the landing-pad-expected state upon debug entry, requires debugger extension to disassemble custom instructions

Table 3.15: Qualitative comparison of signature-based (NXP-AT FCFI, MAFIA) and scrambling-based (Gousselot et al., Confidaent, SecDec) CFI schemes, and Armv8.1-M and RISC-V landing pad mechanisms (BTI, Zicfilp)

3.1.5.9 Requirement Compliance

Table 3.16 provides a comparison between the obtained results and the corresponding IP requirements from WP1. The table summarizes the verification outcomes for each requirement and indicates the achieved level of compliance.

REQ-DOC	REQ-ID	REQ-TYPE	REQ-Description	Verification Method	Activities / Justification	Compliance (C / PC / NC)
D1.4	FCFI-01	F-HW_ARCH-S	FCFI shall offer different security levels.	Reviews, instruction perturbation testing	Covered by reviews and perturbation tests conducted on final design	C
D1.4	FCFI-02	F-HW_ARCH-S	FCFI can be enabled and disabled dynamically to ensure compatibility with unprotected code parts.	Testbench (RTL simulation, FPGA)	Covered by passing unit tests	C
D1.4	FCFI-03	F-HW_ARCH-FEA	FCFI shall raise an exception in case of a checksum mismatch.	Testbench (RTL simulation, FPGA)	Covered by passing unit tests	C

Continued on next page

D1.4	FCFI-04	F-HW_ARCH-FEA	The state of FCFI shall be saved and FCFI shall be disabled before jumping to a trap handler. Before resuming execution of the interrupted code, the state of FCFI shall be restored.	Testbench (RTL simulation, FPGA)	Covered by passing unit tests	C
D1.4	FCFI-05	NF-HW_ARCH-FEA	FCFI shall use a small number of custom instruction encodings.	Reviews (specification, code)	Not met: Achieving the target code size overhead required additional custom instructions. An optional configuration disables custom branches (freeing custom-1, OP-32, and OP-IMM-32 major opcodes), leaving custom-0, custom-2, and repurposed compressed encodings (c.flw, c.fsw, and c.fsd), but increases code size.	NC
D1.4	FCFI-06	NF-HW_ARCH-PR	The code size overhead shall be below 15%.	Benchmarks (Embench IoT)	The 15 % target was achieved with CRC5 (9.14 %) and CRC11 (11.62 %), but not with the highest-protection CRC16 configuration (18.49 %).	PC
D1.4	Target Performance		The average runtime overhead shall be up to 15 %.	Benchmarks (Embench IoT)	Achieved for all security configurations (max 7.49 %)	C

Continued on next page

D1.4	Target Performance	The achieveable clock frequency on the FPGA shall be greater than or equal to 50 MHz.	Synthesis Results	Even the unmodified baseline platform on the Genesys 2 FPGA only marginally achieved this frequency. The intended objective was to ensure that integration of the NXP-AT extensions would not reduce the operating frequency of the CORE-V-MCU baseline platform on the Genesys 2 FPGA (20 MHz). This objective was achieved.	NC

Table 3.16: Requirement Compliance Matrix FCFI

3.1.6 Context-Aware Performance Monitor Counter (CA-PMC) – TRT

3.1.6.1 IP Card

Basic Info		
IP name	Context-Aware Performance Monitor Counter (CA-PMC)	
License	Open-source (SolderPad Hardware License v0.51)	
Repository	https://github.com/ThalesGroup/cva6-context-aware-monitoring.git	
Architecture		
Clock	Number of clock domains	1
	Synchronous with system	Y
	Clock generated internally	N
Ctrl Interface	ISA extension?	N
	Memory mapped?	Y (see also CA-PMC-IF in Section 3.2.1.5)
	Protocol	AXI or AXI Lite
	Address Map	Y (see Section 3.2.1.5)
Initiator Interface	Protocol	N/A
	Cached?	N
	IOMMU?	N
Interrupts	Interrupts	Y (generates 1 interrupt)
Microarchitecture		
Parametrization	Parametric no. cores?	N
	Parameteric config?	Y
Programmability	Contains programmable cores?	N/A
	ISA	N/A
Software		
Compiler	Requires specialized compiler?	N
	Compiler repository	-
Hardware Abstraction Layer	N/A	
High-level API	Is there a high-level API/SDK?	N
	SDK repository	N/A
	Is there a domain-specific compiler?	N
Integration		
IP Distribution	Manifest type (if any)	N
	Standalone simulation?	N
	(if standalone sim) SW requirements?	-
	Integration documented / examples?	Y
Synthesis	Is the IP synthesizable?	Y
	FPGA synthesis example available?	Y (as component in CVA6 based design)
	ASIC synthesis example available?	N
Simulation	Closed-source simulation?	Y (QuestaSim)
	Open-source simulation?	Y (Verilator)
Evaluation	PPA results available?	N

3.1.6.2 General Information

As already presented in Section 3.3.1 of Deliverable D3.1, the Context Aware Monitoring framework is a set of hardware IPs to enrich monitoring with software context information such as the currently running virtual machine in the hypervisor, the currently running process in the operating

system, or the currently running thread/function at the application level.

The Context-Aware Monitoring framework is composed of 4 different hardware IPs defined in both WP2 and WP3:

1. The **CA-CORE extension** extends the CVA6 [84] core design with an extra CSR register storing the software context information and providing both a software and a hardware interface to set up and access the context information.
2. The **CA-BUS extension** decorates memory requests with information related to the software context, both at core-level up to the L1 cache and at the NoC level for DDR accesses on the AXI bus.
3. The **CA-PMC extension** implements context-based performance monitor collection, filtering collected hardware events based on the software context, counting events only for a specific process of the operating system, or a specific critical or cryptographic function in the application.
4. The **CA-PMC-IF extension** (Section 3.2.1) provides a memory-mapped interface to the software context and the monitored core performance monitor counters (PMCs), enabling remote out-of-core monitoring.

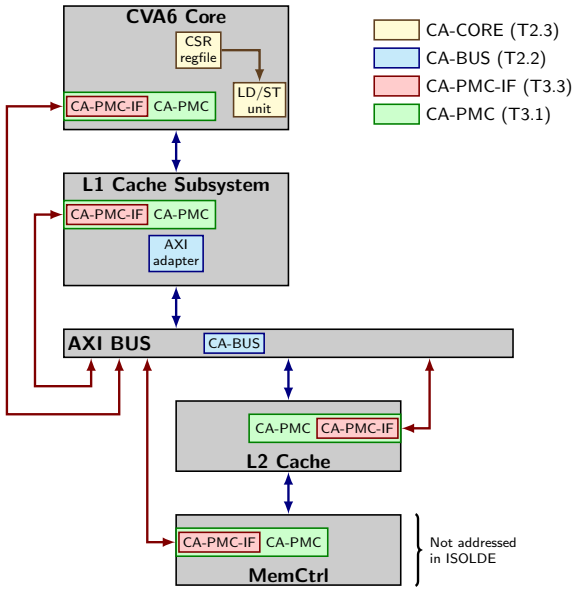


Figure 3.8: Context-Aware monitoring architecture concept

Figure 3.8 shows an example of an architecture using the 4 IPs provided by the Context-Aware Monitoring framework.

Prior to ISOLDE, we developed METrICS [37], a software-level Measurement Environment for multi-core Time Critical Systems dedicated at non-intrusively collecting performance monitoring information. Such information is currently used to characterize the behavior of the software on the hardware in terms of hardware resource usage, and anticipate the timing interference issue inherent to multi-core systems.

We then developed the Cyber BlackBox [38] as a way to couple this monitoring information with machine learning to learn the expected behavior of critical software and detect deviations from this expected behavior as anomalies, an indicator of either safety-related failures or security-related cyberattacks.

A drawback of a monitoring framework at the software level is that it provides the user with a new attack surface (the monitoring framework itself), introducing new potential vulnerabilities and the ability to spy on the critical application.

The ISOLDE project and the open hardware ecosystem are opportunities to implement such a monitoring infrastructure at the hardware level, with dedicated memories not accessible from the software layer. The safety/security extensions presented above allow us to collect context-aware monitoring information at the hardware level from a core dedicated to monitoring the applicative cores.

3.1.6.3 Purpose and Scope

The purpose of the CA-PMC IP is to implement the counters associated with the monitored IP, providing a memory-mapped bus interface to control and read the performance monitor counters, enabling the counters' access to a monitoring core.

3.1.6.4 Refined architecture description

The architecture of the CA-PMC IP depends on the IP it is monitoring, e.g. in Figure 3.8 the core has a different CA-PMC module, aka the CA-PMC Core module, than the one in the memory controller, aka the CA-PMC Memory Controller module. To simplify the design of the different CA-PMC modules the CA-PMC-IF module has been introduced. The CA-PMC-IF module provides an uniform interface to all the CA-PMC modules. The CA-PMC-IF provides storage for the counters and their management, and a memory-mapped bus interface, so all the CA-PMC modules can provide the same interface to the monitoring core(s).

With the CA-PMC-IF module instantiated the only task the CA-PMC module needs to do is to convert the monitoring element signals into events that the CA-PMC-IF can use to update the counters it manages. For example, a CA-PMC Core module might generate a number of instructions committed each cycle from the signals on the Core Scoreboard. The CA-PMC input signals and the signal to event translation logic are highly dependent on the module the CA-PMC is associated with and their description is out of the scope for this document. Figure 3.9 presents a high-level view of the CA-PMC architecture.

The CA-PMC is to be included/instantiated in the module it is providing counters for, like depicted in Figure 3.9, or as an external module connected to the module it is providing counters for, like depicted in Figure 3.10. The second is the approach followed in the ISOLDE project, but both are possible. To simplify our diagrams, when the CA-PMC module is associated with a single module (e.g. a CVA6 core module), we represent the CA-PMC module as a submodule of the module it is providing counters for, as show in Figure 3.8. When the CA-PMC module is providing counters for multiple modules we represent it as an external module connected to the modules it is providing counters for, see for example Figure 3.14 where a single CA-PMC module is used to provide the counters of two CVA6 core modules.

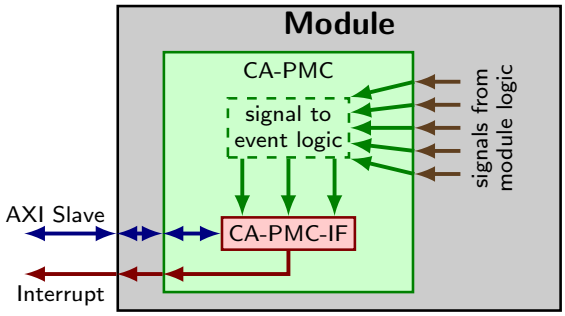


Figure 3.9: CA-PMC module architecture and example of instantiation in module

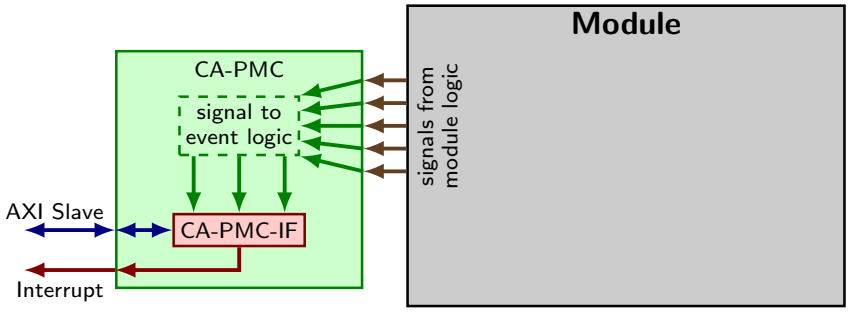


Figure 3.10: Alternative CA-PMC module instantiation

Finally, as shown in Figures 3.9 and 3.10 the CA-PMC module routes the interrupt signal and the AXI Slave provided by the CA-PMC-IF instance, that respectively are forwarded typically to an interrupt controller and to an AXI bus.

3.1.6.5 Interfaces

As described in Section 3.1.6.4, the CA-PMC module provides three interfaces:

1. Signals from the monitored module (input): Signals from the monitored module allowing the generation of events that can be counted by the CA-PMC module (through the support provided by the CA-PMC-IF module). The signals also contain the context information generated by the CA-CORE module and forwarded by intermediate modules (e.g. bus, caches, etc.).
2. AXI Subbordinate interface (input/output): An addressable slave interface, that only responds to requests, to control the performance monitoring counters operation (e.g. set the event the counter is tracking, enable a counter, enable a counter to generate overflow, etc.) and read/write the counters. The actual implementation of the AXI Slave interface and its memory mapping is done by the CA-PMC-IF module instantiated by the CA-PMC and the latter simply forwards the interface. For a complete description of the interface and its memory map, refer to the CA-PMC-IF description in Section 3.2.1.
3. Interrupt (output): An interrupt signaling that one of the counters has overflowed. This

signal is actually generated by the CA-PMC-IF module instantiated by the CA-PMC and simply forwarded by the latter. For a more complete description of the conditions on which an interrupt can be generated, refer to the CA-PMC-IF description in Section 3.2.1.

3.1.6.6 ISA specialization

The CA-PMC module has no impact on the ISA. However, to exploit the context monitoring capability, IPs generating events should be capable to pass the context information (through the context signal presented in Section 3.2.1.6). Generators of this context information might need ISA modifications to generate it, as proposed by the CA-CORE extension developed in Task T2.3.

3.1.6.7 Evaluation prototype

FPGA

A CA-PMC prototype of the CVA6 Core has been implemented. This prototype uses the CA-PMC-IF version 2 IP to provide the CA-PMC memory-mapped interface. CA-PMC-IF version 2 allows enhanced management to the CA-PMC compared to CA-PMC-IF version 1 evaluated in Deliverable D3.2 [18]; for a detailed description of the CA-PMC-IF IP module version 2 and the differences with version 1 please refer to Section 3.2.1. The prototype design provides a parameterizable number of counters (from 1 to 256) and all the events available in the CVA6 version 5.01 (24 in the current implementation) can be mapped to any of the counters. The CVA6 CA-PMC module follows the design of the CVA6 core that integrates the first-level cache into the core. Compared with the CVA6 CA-PMC module presented in Deliverable D3.2 [18], the CVA6 CA-PMC module has been updated to integrate events from multiple cores, allowing more compact designs as presented in the third prototype design hereafter.

The CVA6 CA-PMC module has been instantiated in three different designs. The first design instantiates the CVA6 CA-PMC alongside a single-core design of the CV32A6, as depicted in Figure 3.11. This design allows to validate the functionalities of the counters provided by the CVA6 CA-PMC module. This design is particularly used to:

- initialize platform to debug the design of the CVA6 CA-PMC and its integration in a SoC;
- validate the address mapping and access to the CVA6 CA-PMC module from the CVA6 core through the system bus;
- validate the counters counting operation by comparing them to regular performance monitoring counters provided by the regular CVA6 core;
- validate the CA-PMC-IF module control capabilities, including context filtering and the different fast access masks;
- and validate the counter overflow interrupts.

This single-core design has been mostly validated in simulations under Verilator. It has also been succesfully synthesized for the Genesys 2 board (Xilinx Kintex-7 FPGA) and validated with baremetal tests.

The second design is a multi-core design with two CVA32A6 cores (but more could be instantiated) each with its own level 1 cache, connected to the system bus. Each core has its own CA-PMC Core module, i.e. the design contains as many CA-PMC Core modules as cores, and the CA-PMC core modules are connected to the system bus allowing one core to access the performance counters of the other core through memory requests (i.e. load and store instructions).

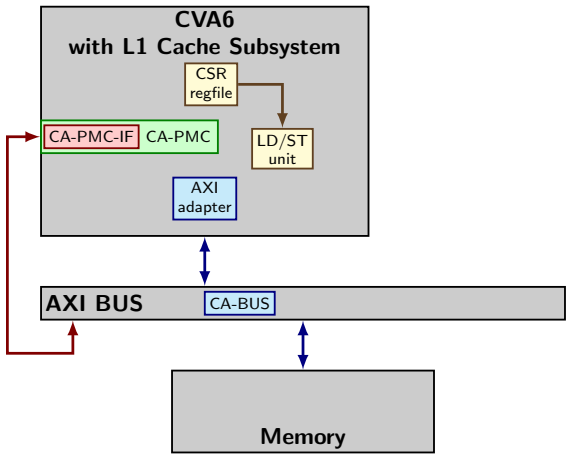


Figure 3.11: Single-core prototype of the Context Aware Monitoring framework

This design as the first one does not have a level 2 cache. Figure 3.12 shows an instantiation of this second design. This design allows to validate the access of the counters from anywhere in the system (as long as the core and the accessed CA-PMC share the same address space). Various tests have succesfully been performed, but as in the first design the context awareness has not been tested and in this case neither the counter overflow interrupts. This design has been validated using simulation with baremetal tests. This design has also been synthesized for the Genesys 2 FPGA board. On the FPGA board the design has been validated with baremetal tests and with the Zephyr hypervisor [2]. For the latter, two instances of Zephyr have been deployed (AMP), one on each core and a driver for the CA-PMC CVA6 module created. Figure 3.13 shows the output of one of tests on the FPGA board with Zephyr. In the test the Zephyr instance running in core 1 is continuously switching between two executions (thread A and thread B) of MNIST, setting the context CAMCSR to 1 before launching thread A and to 2 before launching thread B, and setting the context CAMCSR to 0 when finishing any of the thread executions. The Zephyr instance running in core 0 sets six counters in the CA-PMC module connected to core 1 (i.e. monitoring the events in core 1) to count the number of cycles executed, the number of cycles executed while the context was set to 1 (i.e. thread A), the number of cycles executed while the context was set to 2 (thread B), the number of instructions executed, the number of instructions executed by thread A, and the number of instructions executed by thread B. Once the counter registers are set and enabled, core 0 enters an endless loop in which it regularly reads the six counter values to monitor core 1 execution without interfering its execution.

The third design is a dual-core design with CVA32A6 cores, like the second design, but with a shared CA-PMC Core module. The CA-PMC Core module receives monitoring signals from the two cores and provides a single memory mapped interface for all of them. This design is depicted in Figure 3.14. This design has the advantage of requiring less FPGA resources as a single bus interface is required, however unlike the previous design on which each CA-PMC module can be mapped to a core, the single CA-PMC should be mapped to a single core for control purposes to

[2] https://www.zephyrproject.org/

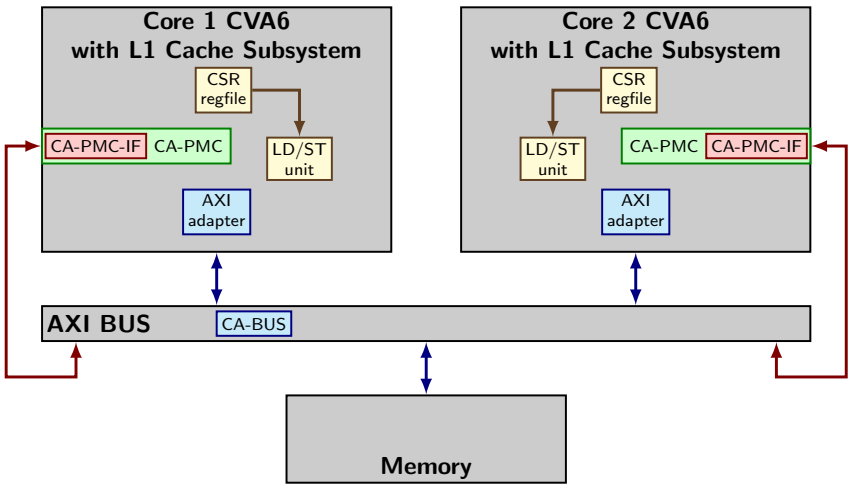


Figure 3.12: Dual-core prototype of the Context Aware Monitoring framework

simplify its management. As for the previous design, it has been validated using simulation and on FPGA with two Zephyr instances.

Listing 3.1 provides the output of one of the bare metal tests on the second design shown in Figure 3.12, dual-core with a CA-PMC Core module for each core. For this test, each CA-PMC Core module has 8 counters available. In the test, the two cores synchronize (not shown in the listing) before starting the CA-PMC modules test. After synchronization the two cores start to execute the same program simultaneously:

- First, each core checks the capabilities (see Section 3.2.1) of the CA-PMC Core module attached to it. We can observe that each core detects that their CA-PMC Core module implements version 2.0 (line 5 and 7 in Listing 3.1 for core 0, and line 6 and 8 for core 1) and has:
  - 8 counters available (line 9 and 10 in Listing 3.1),
  - 24 selectionable events (line 11 and 12),
  - no custom capability (line 13 and 14), and
  - a 8 bits width context length filter support.
- Then, each core checks the availability of the context CSR (CAMCSR support and its width (line 17, 19, and 21 for core 0, and line 18, 20, and 22 for core 1).
- Then, lines 23 and 24 in the listing for core 0 and 1 respectively, each core initializes each counter to predetermined events: counters 0 and 2 are set to event 0 that corresponds to the cycle event, counter 1 and 3 to event 1 that corresponds to the instruction executed event, and the other counters to other events. For all the counters, but counter 2 and 3, the context filtering is disabled (or simply not enabled). For counter 2 and 3 the context filtering is enabled and the filtered context is set to the integer 1. During the execution of the test each core sets the context register (CAMCSR) to 1 when performing computations and to 0 when executing I/O operations (printing in the UART console), thus counter 0 will count all the core cycles and counter 2 will count only cycles during which the core is performing

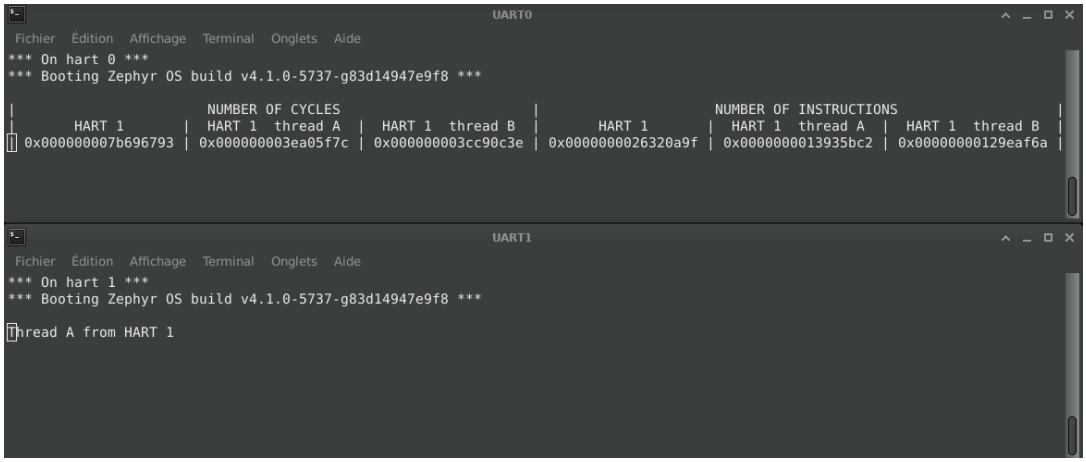


Figure 3.13: Output of a test of the CA-PMC modules in a dual-core design configured in AMP mode with Zephyr

computations other than I/O, and similarly counter 1 will count all the executed instructions and counter 3 will count only the instructions executing during the computation phases.

- Then, each core enables all the counters in the CA-PMC Core modules, lines 25 and 26 in the listing for core 0 and 1 respectively. At this moment, the counters will start to count the events each one is associated with.
- Then, each core starts to perform some matrix computations. The purpose of these computations is to generate events and make the counters count. During these computations, the cores set their context register (CAMCSR) to 1 to stress the context filtering on the CA-PMC, otherwise the context register is set to 0.
- After finishing the matrix computations, each core disables the counters of their associated CA-PMC Core modules, lines 31 and 43 in the listing for core 0 and 1 respectively. At this point the CA-PMC Core counters will stop counting when their associated events occur, i.e. they retain the value they had before being disabled.
- Then, each core dumps the values and associated events of each counter in their associated CA-PMC Core module, lines 32, 33, and 35-40 for core 0 in the listing and lines 44-51 for core 1. After dumping the counters, each core displays the `Goodbye` message, lines 42 and 52 in the listing for core 0 and core 1 respectively.
- After displaying the `Goodbye` message, each core checks if the other one has finished, i.e. already displayed the `Goodbye` message. If the other core has finished, the core simply enters an infinite loop without doing anything. In Listing 3.1 we can observe that it is core 0 that finishes first as there is no more output from that core after displaying the `Goodbye` message. The core that finishes the latest, core 1 in the listing, then dumps all counters values and associated events for each of the CA-PMC Core modules in the design, lines 53 to 68 in the listing. We can observe that the information displayed matches the information previously displayed by each of the cores.

The same bare metal test and others were successfully run on simulations of the two, four and eight cores instantiations of the multi-core designs (i.e. second and third design).

Listing 3.1: Output of baremetal test in dual-core design with a CA-PMC module for each core

```
[C0] Hello
[C1] Hello
[C0] Checking mapped PMC support
[C1] Checking mapped PMC support
[C0] capabilities.version = 2
[C1] capabilities.version = 2
[C0] capabilities.revision = 0
[C1] capabilities.revision = 0
[C0] capabilities.ncounters = 8
[C1] capabilities.ncounters = 8
[C0] capabilities.nevents = 24
[C1] capabilities.nevents = 24
[C0] capabilities.custom = 0
[C1] capabilities.custom = 0
[C0] capabilities.ctxlen = 8
[C1] capabilities.ctxlen = 8
[C0] Checking CAMCSR support
[C1] Checking CAMCSR support
[C0] Max CAMCSR = 0x000000ff
[C1] Max CAMCSR = 0x000000ff
[C0] Checking CAMCSR support OK
[C1] Checking CAMCSR support OK
[C0] Setting counters
[C1] Setting counters
[C0] Enabling counters
[C1] Enabling counters
[C0] Filling matrix
[C1] Filling matrix
[C0] Reducing matrix
[C0] res = 1
[C0] Disabling counters
[C0] Counter 0 event 0 = 0x0000000000000d874
[C0] Counter 1 event 1 = 0x0000000000009010
[C1] Reducing matrix
[C0] Counter 2 event 0 = 0x0000000000000c5a3
[C0] Counter 3 event 1 = 0x0000000000008658
[C0] Counter 4 event 6 = 0x0000000000001001
[C0] Counter 5 event 7 = 0x0000000000001001
[C0] Counter 6 event 3 = 0x0000000000000402
[C0] Counter 7 event 10 = 0x00000000000022c0
[C1] res = 8192
[C0] Goodbye
[C1] Disabling counters
[C1] Counter 0 event 0 = 0x00000000000141a0
[C1] Counter 1 event 1 = 0x0000000000009191
[C1] Counter 2 event 0 = 0x00000000000129f5
[C1] Counter 3 event 1 = 0x0000000000008618
[C1] Counter 4 event 6 = 0x0000000000001001
[C1] Counter 5 event 7 = 0x0000000000001001
[C1] Counter 6 event 3 = 0x0000000000000402
[C1] Counter 7 event 10 = 0x0000000000002280
[C1] Goodbye
[C1] Core 0 counter 0 event 0 = 0x0000000000000d874
[C1] Core 0 counter 1 event 1 = 0x0000000000009010
[C1] Core 0 counter 2 event 0 = 0x0000000000000c5a3
[C1] Core 0 counter 3 event 1 = 0x0000000000008658
[C1] Core 0 counter 4 event 6 = 0x0000000000001001
[C1] Core 0 counter 5 event 7 = 0x0000000000001001
[C1] Core 0 counter 6 event 3 = 0x0000000000000402
[C1] Core 0 counter 7 event 10 = 0x00000000000022c0
[C1] Core 1 counter 0 event 0 = 0x00000000000141a0
[C1] Core 1 counter 1 event 1 = 0x0000000000009191
[C1] Core 1 counter 2 event 0 = 0x00000000000129f5
[C1] Core 1 counter 3 event 1 = 0x0000000000008618
[C1] Core 1 counter 4 event 6 = 0x0000000000001001
[C1] Core 1 counter 5 event 7 = 0x0000000000001001
[C1] Core 1 counter 6 event 3 = 0x0000000000000402
[C1] Core 1 counter 7 event 10 = 0x0000000000002280
```

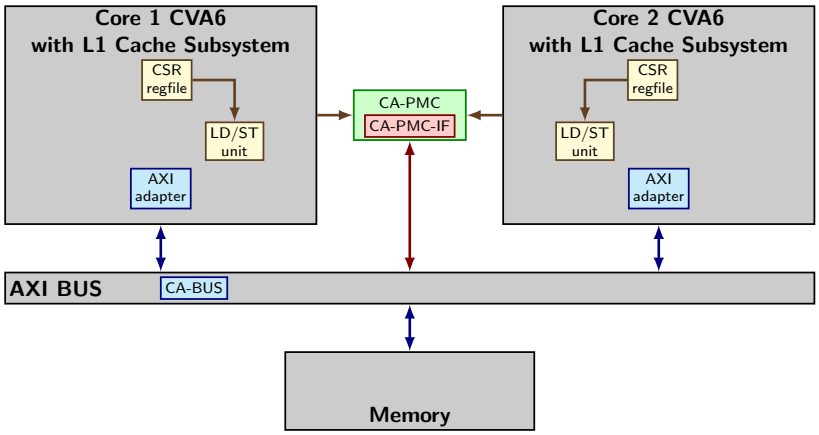


Figure 3.14: Dual-core prototype of the Context Aware Monitoring framework with a CA-PMC module shared between the two cores

Table 3.17 presents resource measurements of a dual-core design with and without the CA-PMC core module, i.e. with and without the cores' performance monitoring counters memory-mapped. Four different designs are considered:

- (a) A dual-core design without CA-PMC Core module.
- (b) A dual-core design with a CA-PMC Core module for each of the cores as presented in Figure 3.12. Each CA-PMC Core module has 8 counters available and the context maximum width is 8 bits.
- (c) A dual-core design with a CA-PMC Core module shared between the two cores as presented in Figure 3.14. As for the previous case, the CA-PMC Core has 8 counters and a maximum context width of 8 bits, which means that in total it provides half the counters of design (b).
- (d) A dual-core design with a CA-PMC Core module shared between the two cores as presented in Figure 3.14. Unlike the previous designs, the CA-PMC Core has 16 counters available and a maximum context width of 8 bits. This design provides the same number of counters than design (b), but provides more flexibility on how which events are monitored. For example, this design allows monitoring 16 events from a core at a given time, while design (b) allows at most 8 events from each core.

All the designs were synthetized for the Genesys 2 FPGA board. For each design, the resource usage of four different components (columns in Table 3.17) is provided:

- *Design*: the resource usage of the complete design.
- *Bus*: the resource usage of the instantiated bus. Effectively, when adding CA-PMC modules to the design, new slaves are added to the bus, which increase the resource usage.
- *CA-PMC (AXI)*: the resource usage of the CA-PMC modules including the logic of the AXI slave interfaces. This component is absent from design (a) as this design does not instantiate any CA-PMC module.
- *CA-PMC*: the resource usage of the CA-PMC modules without the AXI slave interface logic. This component is absent from design (a) as this design does not instantiate any CA-PMC

module.

For each component, different primitive element usages are provided (number of latches, LUTs, etc.) along the total number of primitive elements. The last row shows how many more resources the (b), (c), and (d) complete designs and bus require compared to the (a) complete design and bus.

	(a) Dual-core		(b) Dual-core with 2 CA-PMC (8 counters per CA-PMC)				(c) Dual-core with 1 CA-PMC (8 counters)				(d) Dual-core with 1 CA-PMC (16 counters)			
	Design	Bus	Design	Bus	CA-PMC (AXI)	CA-PMC	Design	Bus	CA-PMC (AXI)	CA-PMC	Design	Bus	CA-PMC (AXI)	CA-PMC
LATCH	49822	9975	54004	11913	2204	1792	51918	10944	1102	896	52821	10950	1999	1792
LUT	79173	9684	86963	12012	4206	644	83032	10959	2159	410	84293	11017	3058	1126
MUXFX	2747	111	3027	122	224	224	2903	54	112	112	3049	54	276	276
CARRY	3474	1152	4244	1352	570	272	3859	1252	285	144	4123	1252	549	348
BMEM	96		96				96				96			
MULT	8		8				8				8			
IO	346		346				346				346			
CLK	17		17				17				17			
DMEM	2601		2601				2601				2601			
OTHERS	15		15				15				15			
Total	138299	20922	151321	25399	7204	2932	144795	23209	3658	1562	147369	23273	5882	3542
Diff			+9.42%	+21.40%			+4.70%	+10.93%			+6.56%	+11.24%		

Table 3.17: Resource usage comparision between a (a) dual-core design without the Context-Aware Monitoring infrastructure and multi-cores designs with (b) one 8-counter CA-PMC per core, (c) one 8-counter CA-PMC shared between the cores, and (d) one 16-counter CA-PMC shared between the cores.

As can be observed in Table 3.17, the designs including the CA-PMC Core module(s) increase the resource usage between 4.70% (design column, configuration c) and 9.42% when compared to design (design column configuration b) that does not have any CA-PMC module. An important part of this increase is due to the AXI bus used to address the CA-PMC modules, as can be observed by the increase of resource usages in the Bus component and the resources used by the AXI slave interface in the CA-PMC modules: more than 50% when the CA-PMC module provides 8 counters, and almost 40% when providing 16 counters. While the impact of the CA-PMC modules on these designs might seem important, it must be considered that the designs used are pretty small: 32-bit cores, only 2 cores, only 1 small level of cache for each core, no large L2 cache, etc. The impact of the CA-PMC modules on larger and more complex designs will be much smaller. Nevertheless, we have different ideas to improve the Context-Aware Monitoring components (CA-PMC, CA-PMC-IF, CA-Core, and CA-Bus) resource usage, like using a lightweight bus solution for these modules and enhanced CA-PMC-IF designs.

3.1.6.8 State-of-the-Art Comparison

Arm

Arm provides a very large set of monitoring facilities. The *Arm Architecture Reference Manual* [9] describes the memory interface for memory-mapped performance monitor registers in Section I3. This interface is more complete than the Context-Aware Monitoring (CAM) solution developed here, notably with a larger number of the selectable events, like cache L2 events. While it does not provide a straight replacement for the context capability, it has other filtering approaches which could reach similar results, like mode filtering. Architecturally, the Arm approach is to provide a

single system-level interface for all the performance monitoring counters. CAM follows a more flexible approach with the CA-PMC modules, and supporting CA-PMC-IF modules, where the different performance monitoring counters can be memory allocated to different entities (cores or applications) in the system, unlike the Arm approach where the memory-mapped performance counters interface must be controlled by a single entity.

Arm also provides performance monitor counters for other resources than the cores with solutions as *Arm's Memory System Resource Partitioning and Monitoring* (MPAM) [5]. In the future, we expect to develop equivalent RISC-V solutions using the CAM infrastructure.

RISC-V

The SafeSU module developed in ISOLDE and presented in this document in Section 3.2.3 provides memory-mapped performance counters like the Context-Aware Monitoring infrastructure. More specifically, it can be compared to the CA-PMC and CA-PMC-IF modules. The SafeSU module is specialized on interconnect/bus monitoring, and provides advanced mechanisms to monitor different aspects of these resources. The CA-PMC-IF targets all kind of monitoring requirements, and as such a first implementation of the CA-PMC module focuses on providing memory-mapped monitoring support for cores (CA-PMC Core module). The CA-PMC/CA-PMC-IF module also provides a capability to filter events depending on context, which SafeSU does not provide for instance. In future work, we expect to extend the CA-PMC-IF module to provide advanced capabilities as those implemented in SafeSU, and develop CA-PMC modules addressing interconnect/bus resources.

Cosimi [29] develops a solution similar to the CA-PMC Core module, with a memory-addressed performance monitoring unit of all the cores in a multi-core system. It provides overflow capabilities like the CA-PMC Core module, and while in [29] the presented solution monitors all the cores in a single IP, the solution should be easy to instantiate multiple times to develop designs as the one presented in Figure 3.12. Unlike the CA-PMC solution, the work of Cosimi does not seem to address the modularity of the solution, i.e. a CA-PMC-IF like module is not developed, and does not provide the context filtering capability.

3.1.6.9 Requirement Compliance

Table 3.18 provides a comparison between the obtained results and the corresponding IP requirements from WP1. The table summarizes the verification outcomes for each requirement and indicates the achieved level of compliance.

Table 3.18: Requirement Compliance Matrix for CA-PMC and CA-PMC-IF hardware modules

REQ-DOC	REQ-ID	REQ-TYPE	REQ-Description	Verification Method	Activities / Justification	Compliance (C / PC / NC)
D1.4	CA-PMC-01	F-HW_UARCH-FEA	Implementation for cache PMCs	Unit tests in SoC FPGA design	1st design prototype of 3.1.6.7	C
D1.4	CA-PMC-02	F-HW_UARCH-FEA	Implementation for bus interfaces PMCs	Unit tests in SoC FPGA design	1st design prototype of 3.1.6.7	C
D1.4	CA-PMC-03	F/NF-HW_UARCH-PR	Bounded event to counter processing time (<10 clock cycles)	Simulation measurement	0 cycle for remote out-of-core monitoring	C
D1.4	CA-PMC-IF-01	F-HW_ARCH-FEA	Interface provides capabilities to configure the context filtering capabilities of the CA-PMCs	Unit tests in SoC FPGA design	1st and 2nd design prototype of 3.1.6.7	C
D1.4	CA-PMC-IF-02	F-HW_ARCH-FEA	Interface provides means to retrieve and reset CA-PMCs counters values	Unit tests in SoC FPGA design	1st and 2nd design prototype of 3.1.6.7	C
D1.4	CA-PMC-IF-03	F-HW_ARCH-FEA	Interface provides means to set up thresholds on CA-PMCs	Unit tests in SoC FPGA design	1st and 2nd design prototype of 3.1.6.7	C
D1.4	CA-PMC-IF-04	F-HW_ARCH-S	Interface provides the means to access multiple CA-PMCs	Unit tests in SoC FPGA design	2nd and 3rd design prototype of 3.1.6.7	C
D1.4	CA-PMC-IF-05	F/NF-HW_ARCH-PR	Remote monitoring should not impact the real-time behaviour of the monitored application	Simulation measurements	0 cycle for remote out-of-core monitoring	C

3.1.7 Memory Subsystem Support for Bytecode VMs – HM

3.1.7.1 IP Card

Basic Info		
IP name	Bytecode VM Memory Subsystem	
License	Open-source (Apache2 License)	
Repository	https://github.com/hm-aemy/wasm-hwstack (pending publication)	
Architecture		
Clock	Number of clock domains	1
	Synchronous with system	Y
	Clock generated internally	N
Ctrl Interface	ISA extension?	Y (CSR)
	Memory mapped?	N
	Protocol	N/A
	Address Map	N/A
Initiator Interface	Protocol	Simplified bus, AXI Wrapper
	Cached?	N
	IOMMU?	N
Interrupts	Interrupts	N
Microarchitecture		
Parametrization	Parametric no. units?	N
	Parameteric config?	Y
Programmability	Contains programmable cores?	N
	ISA	-
Software		
Compiler	Requires specialized compiler?	N
	Compiler repository	-
Hardware Abstraction Layer	N/A	
High-level API	Is there a high-level API/SDK?	N
	SDK repository	N/A
	Is there a domain-specific compiler?	N
Integration		
IP Distribution	Manifest type (if any)	N
	Standalone simulation?	Y (cocotb verification)
	(if standalone sim) SW requirements?	N
	Integration documented / examples?	Y
Synthesis	Is the IP synthesizable?	Y
	FPGA synthesis scripts/example available?	N
	ASIC synthesis scripts/example available?	N
Simulation	Closed-source simulation?	N
	Open-source simulation?	Y (Verilator, Icarus)
Evaluation	PPA results available?	Y

3.1.7.2 General Information

The memory subsystem support for bytecode VMs is a standalone HW module that serves as a hardware-assisted stack. Software is adopted with CSR accesses to substitute memory ac-

cesses, essentially offloading operand-stack handling into hardware, with the goal of single-cycle accesses to the stack.

3.1.7.3 Purpose and Scope

Interpreters for bytecode virtual machines (JVM-class runtimes, WebAssembly runtimes, and comparable stack-based execution engines) spend a large and largely unavoidable fraction of their execution time not on the semantics of the bytecode itself, but on the bookkeeping around the operand stack. Every single bytecode that consumes or produces operands forces the interpreter to materialize the stack pointer, compute an effective address, issue a load or store into the data cache, and write the updated pointer back. On a register-machine host such as RISC-V this bookkeeping is pure overhead: it occupies load/store slots, pollutes the data cache with a working set that is by construction hot but shallow, and serializes the interpreter dispatch loop behind memory latency.

This module removes that overhead by making the operand stack a first-class architectural resource of the core. The stack itself continues to live in main memory, in a region that software describes, owns, and can inspect; what the module virtualizes is the *access* to it. Software no longer computes addresses and issues memory operations, but performs operand-stack operations through dedicated CSRs. Internally, the module keeps the top region of the stack in a small local cache and moves data between that cache and main memory asynchronously and in bursts, so that common operand-stack accesses can be served locally without memory traffic being visible to software, with single-cycle access as the fast-path design target.

The design goal is therefore twofold:

- **Latency:** target single-cycle CSR accesses for dominant operand-stack operations, removing locally served accesses from the core's load/store path.
- **Instruction count:** eliminate the address arithmetic and stack-pointer maintenance from the interpreter's inner loop, which shortens the critical dispatch sequence of every stack-consuming bytecode.

The scope of the module is deliberately narrow. It accelerates the operand stack of a stack machine; it does not decode, dispatch, or execute bytecode, and it does not manage the call stack, the heap, or the object model of the guest runtime. Those remain entirely in software, which keeps the module small, runtime-agnostic, and reusable across different bytecode formats. The stack region remains addressable from software, and a consistent in-memory representation can be obtained by flushing the local cache first, so debugging, garbage collection, exception unwinding, and stack inspection can continue to use the existing mechanisms of the guest runtime (see the software interface below).

The module is designed for integration into RISC-V cores such as CVA6 and related cores that can support the required custom CSR interface; the current evaluation prototype is based on CV32E40P.

3.1.7.4 Place in the System

The module is tightly integrated into the core via the CSR interface, extending the CSR map with custom extension CSRs. As a second interface, the module has a simple bursting memory interface that can trivially be mapped to AXI4-Lite. A wrapper is available.

Because the fast path is a CSR access, the module sits architecturally inside the core rather than on the system bus, and it is visible to software only through the CSR map — no MMIO region, no separate driver, and no MMIO address decoding on the critical path. The second interface faces the memory system and carries only the asynchronous spill and fill bursts between the local stack cache and the backing memory region; its bandwidth requirement is proportional to how far the guest program's stack depth oscillates across the cache boundary, not to the bytecode execution rate.

The one non-trivial integration requirement is that the host core must tolerate multi-cycle CSR accesses. The fast path targets single-cycle completion for stack operations served from the local cache, while accesses that cannot be served locally may assert wait states until the required spill or refill operation has completed. Cores that assume fixed-latency CSR reads and writes therefore cannot host the module without modification.

The configuration parameters are:

`STACK_SIZE` Size of the cached stack, i.e., the number of stack entries held locally. This parameter directly trades area (local storage) against the probability of a stall: a deeper cache absorbs larger swings in stack depth without touching memory.

`STACK_BURST_SIZE` Size of bursts, which are asynchronously fetched and stored. This parameter trades memory-system efficiency (longer bursts amortize the transaction overhead) against the reaction time of the refill/spill engine and against the hysteresis around the fill thresholds.

3.1.7.5 Refined architecture description

The internal architecture remains organized into four components with the same overall responsibilities. The CSR interface has been extended with additional runtime and status registers, while the general software usage model and memory-backed stack organization remain unchanged.

The architecture is internally organized into four components:

CSR logic serves requests for the CSRs of this module, which are split into configuration, runtime and status registers. Configuration registers (`stackbase`, `stackptr`, `stackend`, `stackctrl`) describe the memory-backed stack region and the enable state; runtime registers (`stackpush`, `stackpop`, `stackpoke` , and `stackpeek`) provide the operand-stack operations used during execution, while `stackstatus` reports accelerator error conditions. Stall cycles can appear on operations that require multi-cycle processing, which limits integration to cores that allow for multi-cycle CSR accesses.

Stack Stack manages a local cache of the current top elements of the stack, with a capacity of `STACK_SIZE` entries. It exposes the logical top of the stack to the CSR logic and reports its current occupancy through the `FILL` field. The fill state is also used internally by the control and memory logic to determine when backing-memory transfers are required.

Memory logic Memory logic transfers data between the local cache and backing memory in groups of up to `STACK_BURST_SIZE` entries, with the individual transfers issued through the memory-side interface. When additional stack entries are required from memory, the memory logic refills the local cache; when cached entries must be written back, it transfers them to the backing-memory region. The refill and spill mechanism is intended to reduce the frequency with which software-visible stack operations are stalled by backing-memory accesses. Background transfers may be initiated before the cache reaches its boundary in

order to hide part of the backing-memory latency.

Control logic synchronizes between the modules, handles configuration accesses, and signals wait states. It is also responsible for the ordering guarantees around the enable bit: a $1 \rightarrow 0$ transition of `stackctrl.EN` flushes the entire local cache to memory so that after the write completes, the in-memory representation of the logical stack is complete and consistent, and software may read, relocate, or hand over the stack region. `stackptr` continues to represent the logical stack position and is not changed by the flush itself. This is the mechanism that makes context switches, garbage collection, and stack inspection safe.

3.1.7.6 Interfaces

Bus/AXI4-Lite interface

The bus interface is a simple standard bus interface that only handles one outstanding memory transaction at a time. It can be mapped to AXI4-Lite and other bus protocols. Because only one transaction is in flight, the interface imposes no reordering or ID-tracking requirements on the interconnect; the burst behavior is realized as a sequence of single accesses to consecutive addresses.

CSR interface

The CSR interface is defined in the following table. All accesses are `XLEN` wide; the addresses in `stackbase`, `stackptr`, and `stackend` are byte addresses and are expected to be naturally aligned to the stack entry size.

CSR	Index	Field	Bits	Function
`stackctrl`	0x7C9	`FILL`	31..8	Current number of entries held in the local stack cache (read only)
		`EN`	0	Enabled. On transition from 1 to 0, the module flushes the stack content to memory.
`stackbase`	0x7C7	`ADDR`	XLEN-1..0	Base address of stack in memory
`stackptr`	0x7C6	`ADDR`	XLEN-1..0	byte address of the next free logical stack cell
`stackend`	0x7C8	`ADDR`	XLEN-1..0	End address of stack in memory
`stackpush`	0x7CA	`DATA`	XLEN-1..0	Push data item to stack
`stackpop`	0x7CB	`DATA`	XLEN-1..0	Pop data item from stack
`stackstatus`	0x7CC	`STATUS`	3..0	Sticky `(ERR, DIS, UNF, OVF)` accelerator status; any legal write clears the status
`stackpeek`	0x7CD	`DATA`	XLEN-1..0	Return the logical top item without modifying the stack state
`stackpoke`	0x7CE	`DATA`	XLEN-1..0	Replace the logical top item without modifying the stack state

The configuration CSRs (`stackbase`, `stackptr`, `stackend`) are only written while the module is disabled (`stackctrl.EN` = 0). While the module is enabled, `stackptr` continuously represents the byte address of the next free logical stack cell and therefore defines the logical stack depth together with `stackbase`. The `FILL` field independently reports how many of the top stack entries are currently held in the local cache. Flushing the cache does not change the logical stack position

represented by `stackptr`.

Software Interface

The bytecode virtual machine interpreter (or other stack-based software) accesses the module along the lifecycle as listed below.

```
// Configure the currently running application
csr_write(stackbase, <base-address>);
csr_write(stackptr,  <base-address>);
csr_write(stackend,  <end-address>);
csr_write(stackctrl, 1);
// During runtime: Push an element, example constants
csr_write(stackpush, 2);
csr_write(stackpush, 3);
// During runtime: Pop elements, push result, example add
p0 = csr_read(stackpop);
p1 = csr_read(stackpop);
csr_write(stackpush, p0 + p1);
// Read the current top element without changing the stack depth
p0 = csr_read(stackpeek);
// Replace the current top element without changing the stack depth
csr_write(stackpoke, 8);
// Context switch between two applications:
// disabling flushes the cached stack to memory
csr_write(stackctrl, 0);
csr_swap(stackbase, ctx0.stackbase, ctx1.stackbase);
csr_swap(stackptr,  ctx0.stackptr,  ctx1.stackptr);
csr_swap(stackend,  ctx0.stackend,  ctx1.stackend);
csr_write(stackctrl, 1);
```

Note that in the inner loop the entire operand-stack handling of a bytecode reduces to one CSR access per operand: no stack-pointer update, no address computation, and no load or store. The same disable/enable sequence used for the context switch above is also the mechanism by which a garbage collector or a debugger obtains a consistent in-memory view of the stack.

3.1.7.7 Evaluation prototype

The evaluation of the module follows two independent tracks. The *functional and performance* evaluation is carried out simulation-based and on an FPGA prototype, where the module is integrated into a CV32E40-based SoC and a real bytecode interpreter is executed against a baseline build of the same interpreter that uses conventional memory-based operand-stack handling. The cost evaluation is carried out through ASIC synthesis, quantifying the area and timing impact of adding the module to the core. In both tracks the reference point is the same software running on the same core without the module, so that the reported figures isolate the contribution of the hardware-assisted stack rather than the quality of the interpreter.

For the performance evaluation, the reference point is the same software running on the same core *without* the module, so that the resulting figures isolate the contribution of the hardware-assisted stack rather than differences in the interpreter implementation. For the cost evalua-

tion, the reference is the baseline core implemented under the same technology and design constraints.

The current evaluation prototype integrates the accelerator into CV32E40P. The memory-side interface is connected through OBI to the memory subsystem. The present implementation performs stack backing-memory accesses on demand as individual transactions; the background burst-based spill and refill mechanism described above remains an architectural extension for future implementation.

ASIC

Table 3.19: Post-layout ASIC implementation results for the baseline core and the core with the stack accelerator using the IHP 130 nm PDK.

Metric	Baseline Core	Core + Stack Accelerator	Change
Target frequency	150 MHz	150 MHz	–
Post-layout $f_{\max}$	172.57 MHz	179.49 MHz	+4.01%
Die area	1.2586 mm^2	1.4081 mm^2	+11.88%
Core area	1.1542 mm^2	1.2965 mm^2	+12.33%
Cell count	45,186	49,097	+8.66%
Leakage power	0.0465 mW	0.0519 mW	+11.61%
Static power	34.8 mW	39.0 mW	+12.07%
Dynamic power	6.03 mW	6.86 mW	+13.76%
Total power	40.88 mW	47.38 mW	+15.90%

The baseline core and the accelerator-enabled configuration were synthesized using Yosys and implemented with OpenROAD using the IHP 130 nm PDK and the `sg13g2_stdcell` standard-cell library. Both configurations were evaluated under the same target frequency of 150 MHz. Post-layout static timing analysis reports maximum frequencies of 172.57 MHz for the baseline core and 179.49 MHz for the accelerator-enabled configuration, with both designs meeting the target frequency. The addition of the stack accelerator therefore does not introduce a critical-path limitation at the 150 MHz target. The core area increases from 1.1542 mm^2 to 1.2965 mm^2, corresponding to an overhead of approximately 12.3%, while the standard-cell count increases from 45,186 to 49,097, approximately 8.7%. Total reported power increases from 40.88 mW to 47.38 mW, corresponding to approximately 15.9%.

3.1.7.8 State-of-the-Art Comparison

Arm

Arm addressed bytecode execution twice, and in both cases at the level of the *instruction set* rather than the memory subsystem. Jazelle DBX (Direct Bytecode eXecution) added a third execution state to the core in which a subset of JVM bytecodes was executed directly in hardware, with the remainder trapped to a software handler; ThumbEE (Jazelle RCT) later replaced this with a Thumb-2 variant targeted at JIT- and AOT-compiled code [14]. Both have since been deprecated and removed from the architecture: Jazelle DBX is not present in Armv8-A, and ThumbEE was deprecated in Armv7 and removed in Armv8. The reason is instructive for the present work: direct bytecode execution requires a large, format-specific decoder and a second execution state; it is

coupled to one particular bytecode format, and it was ultimately outperformed by JIT compilation on general-purpose cores, which leaves the hardware cost without a corresponding benefit.

The module described here takes the opposite position in the design space. It is deliberately *not* a bytecode decoder and adds no execution state; it accelerates only the operand-stack traffic that is common to all stack-based bytecode formats, and leaves decode and dispatch in software. This makes it an order of magnitude smaller than a Jazelle-class extension, keeps it independent of the bytecode format (JVM, WebAssembly, or a domain-specific VM), and keeps it compatible with the software ecosystem around the runtime.

For the quantitative comparison, current Arm platforms unfortunately have no bytecode-specific hardware support.

RISC-V

The RISC-V ecosystem has no ratified extension for stack-machine or bytecode execution, and the standard interpreter baseline on RV32/RV64 pays the full cost described in the purpose section: every operand access is a load or store plus stack-pointer maintenance. Work in the RISC-V space has instead concentrated on general-purpose mechanisms — cache-management operations [60] and the framework for composable custom extensions [59], the latter of which is precisely the integration path used by this module, since it is realized entirely through custom-extension CSRs and therefore does not conflict with the standard ISA or with other extensions present in the same core.

The comparison against RISC-V is consequently a comparison against the same core with and without the module, which is the methodology described in Section 3.1.7.7.

Custom Hardware-Accelerator Implementations

Dedicated stack-machine hardware has a long history, and the closest relatives of this module come from the Java processor line. picoJava-II held the top of the operand stack in a 64-entry register file and used a background *dribbler* unit to spill and fill entries to memory asynchronously so that the common case hit the local storage [58]; JOP uses a two-level stack cache with the top elements held in registers, explicitly for time-predictable execution [73, 74]. Architecturally, the caching-plus-background-transfer mechanism of this module is directly in that tradition. The decisive difference is one of integration: picoJava-II and JOP are complete Java processors, i.e., the stack cache is inseparable from a bytecode-specific datapath and from a fixed guest language, whereas the module described here is a standalone block attached to a general-purpose RISC-V core through CSRs. The host core keeps running arbitrary code, the runtime keeps running as software, and the module can be omitted from a derivative design without invalidating the software.

More recent work targets WebAssembly. Kim et al. present a hardware WebAssembly accelerator that interprets WASM bytecode directly in hardware, using a LIFO register-based data memory as the operand stack, and report speedups of up to two orders of magnitude for selected kernels on a Cyclone IV FPGA at 6,246 LUTs and 1,563 registers [46]. That design again bundles the stack with format-specific decode and execution and is attached as an off-chip SPI peripheral to an application-class host, which limits it to coarse-grained offload of whole kernels. The module presented here instead accelerates at the granularity of a single bytecode, inside the core, which is what makes it applicable to general interpreter workloads rather than to self-contained compute kernels.

3.1.7.9 Requirement Compliance

Table 3.20 provides a comparison between the obtained results and the corresponding IP requirements from WP1. The table summarizes the verification outcomes for each requirement and indicates the achieved level of compliance.

Table 3.20: Requirement Compliance Matrix BYTEC

REQ-DOC	**REQ-ID**	**REQ-TYPE**	**REQ-Description**	**Verification Method**	**Activities / Justification**	**Compliance (C / PC / NC)**
D1.4	BYTEC-MEM-01	HW_ UARCH-PR	Concept for memory tagging based on Xjpm32	Simulation and FPGA-Prototype	Concept available, but functionally not as useful as needed	PC
D1.4	Target Performance		Average Runtime Improvement of Bytecode Virtual Machines	10-25%	Improvement of dispatching	C

3.1.8 Safety-Related Traffic Injector (SafeTI) – BSC

3.1.8.1 IP Card

Basic Info		
IP name	SafeTI	
License	Open-source (MIT License)	
Repository	https://github.com/bsc-loca/SafeTI/tree/21678221e4bb9c9e51be7d135f0644213726fe7e	
Architecture		
Clock	Number of clock domains	1
	Synchronous with system	Y
	Clock generated internally	N
Ctrl Interface	ISA extension?	N
	Memory mapped?	Y
	Protocol	APB 32b
	Address Map	Base (0xfc085000) + 0x100
Initiator Interface	Protocol	APB 32b
	Cached?	N
	IOMMU?	N
Interrupts	Interrupts	N
Microarchitecture		
Parametrization	Parametric no. units?	N
	Parameteric config?	Y
Programmability	Contains programmable cores?	Y, but using its own traffic pattern descriptors
	ISA	-
Software		
Compiler	Requires specialized compiler?	N
	Compiler repository	-
Hardware Abstraction Layer	N/A	
High-level API	Is there a high-level API/SDK?	Y
	SDK repository	https://github.com/bsc-loca/SafeTI/tree/21678221e4bb9c9e51be7d135f0644213726fe7e/sw
	Is there a domain-specific compiler?	N
Integration		
IP Distribution	Manifest type (if any)	Y (Readme.md)
	Standalone simulation?	Y
	(if standalone sim) SW requirements?	QuestaSim and Xilinx Vivado 2020.2
	Integration documented / examples?	https://gitlab.com/selene-riscv-platform/selene-hardware/-/tree/ISOLDE/grlib/software/noelv/BSC_tests/injector_tests_template
Synthesis	Is the IP synthesizable?	Y
	FPGA synthesis scripts/example available?	Y
	ASIC synthesis scripts/example available?	N
Simulation	Closed-source simulation?	Y (QuestaSim)
	Open-source simulation?	N
Evaluation	PPA results available?	https://doi.org/10.1145/3703910

3.1.8.2 General Information

The SafeTI is a flexible and programmable traffic injection hardware module to enable exhaustive timing verification and validation of powerful Multiprocessor System-on-Chips (MPSoCs) for safety-critical systems. In particular, BSC's latest version comes along with an increased number of features and an improved architecture that have been contributed as part of the work in ISOLDE.

3.1.8.3 Purpose and Scope

SafeTI is designed to inject programmable traffic in on-chip interconnects. SafeTI allows programming arbitrary traffic patterns where multiple parameters can be configured, such as read/write requests, data size sent/received, burst length, inter-request delays, repetitions per request, sequence of requests, etc.

Traffic pattern programming is devised to keep the memory footprint low to reduce the internal storage needed and speed SafeTI programmability up. Moreover, traffic pattern descriptors have been devised, enabling future extensions.

SafeTI is implemented as a pipelined module to enable high injection rates without unnecessary delays between consecutive requests.

SafeTI is designed to ease its portability across different communication interfaces like AMBA AHB, AMBA AXI and others. We provide its realization for an AMBA AHB interface.

SafeTI is integrated in an FPGA-based MPSoC from Frontgrade Gaisler AB, based on RISC-V NOEL-V cores.

3.1.8.4 Place in the System

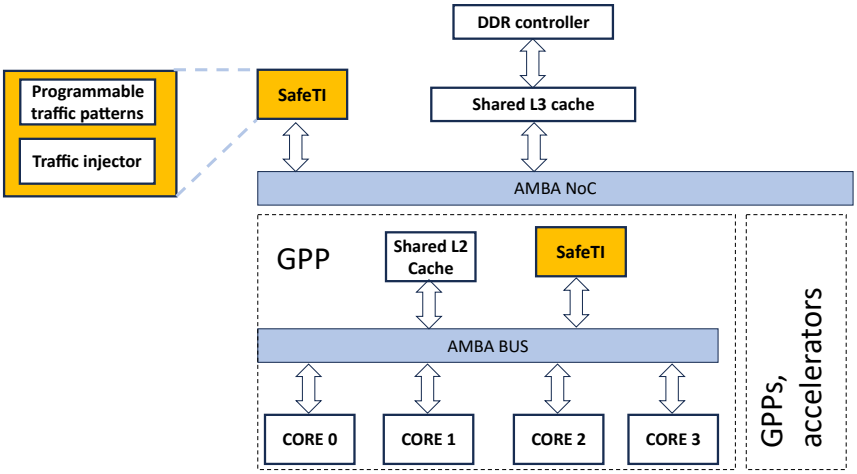


Figure 3.15: SafeTI system integration

The SafeTI is an AMBA AHB and AXI-compliant module for traffic injection. It is intended to be connected to these two types of interfaces, and it is particularly useful if those interfaces have either multiple managers or are connected to subordinates receiving requests from multiple managers. For instance, its best placement is as part of the interface connecting the cores and/or accelerators with the shared caches or memory controllers, as illustrated in the schematic in Figure 3.15. This way, the predefined traffic can be injected to test a variety of timing and functional behavior controllably. SafeTI's programming port is compliant with the AMBA Advanced Peripheral Bus (APB) and with AMBA AXI.

3.1.8.5 Architecture

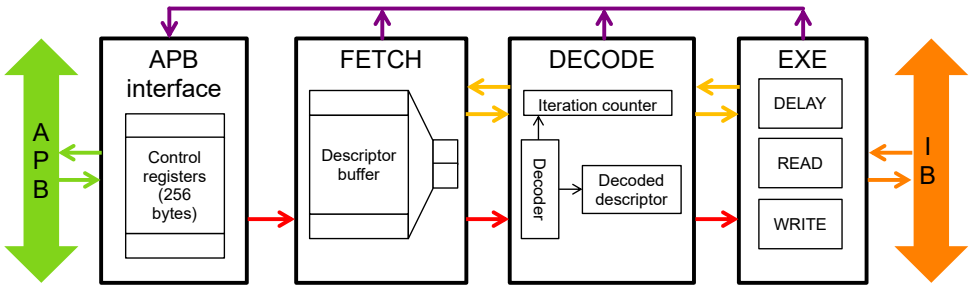


Figure 3.16: SafeTI architecture block diagram

Figure 3.16 shows SafeTI's architecture, which did not need any update since D3.2. The SafeTI has a set of control registers programmed through an APB or AXI interface. Those control registers are the ones that allow to program the descriptor buffer, which stores microprogrammed sequences of traffic patterns to be injected by the SafeTI into the injection interface (IB in Figure 3.16), namely an AXI or AHB interface.

The injection pipeline of the SafeTI works as follows: once the next descriptor is fetched (they are fetched from the descriptor buffer analogously to instructions from memory in a computing core), it is decoded. A descriptor pointer indicates the next descriptor to fetch. The iteration counter in the descriptor indicates whether the next descriptor needs to be fetched next or whether the current descriptor needs to be used again (e.g., to inject repeated traffic). Using the information of the decoded descriptor, the traffic injection stage generates the traffic to inject (read or write, with a given data transaction size, whether in burst mode or not, etc.). Note that if the descriptor is a delay descriptor, no traffic is injected until the indicated delay elapses.

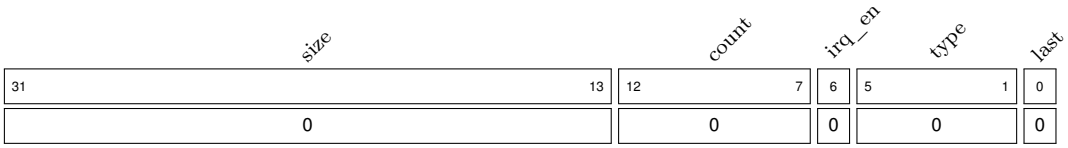


Figure 3.17: SafeTI descriptor format

The SafeTI is a microprogrammed and memory-mapped module using its own descriptor format, which we illustrate in Figure 3.17. Each descriptor type features a different word length and field encoding to accommodate the programmable parameters required by the action to be carried out. However, every descriptor shares the first descriptor word format specified in the figure, providing a compatibility layer in the descriptor format for a lighter implementation. Changes to the first descriptor word fields are considered in future descriptor type expansions. Field size and count could be modified for new descriptor types due to being action-specific, whereas fields like irq_en, type, and last are considered immutable in order to maintain the compatibility layer.

The size field encodes the number of bytes to access for READ and WRITE descriptor types or the number of clock cycles needed to wait for the DELAY descriptors. The count field encodes the number of times the descriptor's execution must repeat. Thus, the same operation is executed

(count + 1) times. The irq_en bit allows the SafeTI to send an interruption through the APB interface upon descriptor completion. Finally, the last bit is used to finalize the injection pattern at a specific descriptor completion (which disables the traffic injector if QUEUE mode is disabled) or restarts the execution from the first descriptor.

Descriptor types READ, WRITE, READ_FIX, and WRITE_FIX include a second 32-bit word used as a starting address where to perform the access operation. Should an invalid address be programmed, the traffic injector behavior depends on the network response to complete the access with an error (e.g., lack of permission, non-existing, etc.) and resumes traffic injection.

If the descriptor type is a synchronized one, a third 32-bit word is used describing the address to be observed in the interface before injecting the traffic of the descriptor. This allows synchronizing injections with specific interface events without the need of enforcing specific timing synchronization. This feature has been proven highly effective to validate, for instance, the coherence protocol in a SoC.

3.1.8.6 Interfaces

AMBA AHB/AXI interface for traffic injection: The AHB or AXI interface is a manager interface used to inject traffic. It is fully compliant with the specification of the corresponding protocol. Note that, in general, a SafeTI instance supports only one of those interfaces and injects traffic accordingly.

AMBA APB/AXI interface for module programmability:

Hardware interface

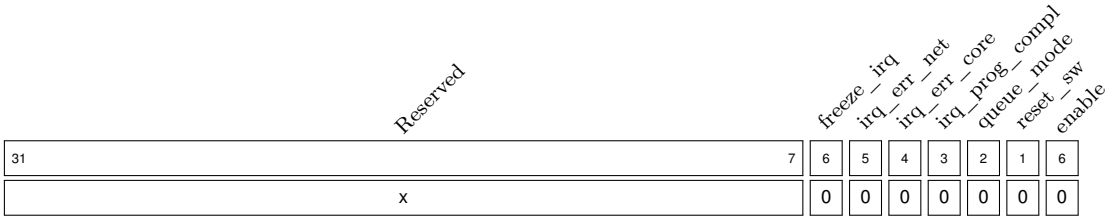


Figure 3.18: SafeTI Hardware Interface

The AMBA APB or AXI subordinate interface is used to program the control registers of the SafeTI, and to store descriptors in the internal descriptor buffer. An address space of 256 bytes is reserved for the APB or AXI interface, with such addresses being set at integration time.

The APB or AXI interface only supports single 32-bit accesses for setting the configuration register (0x00), shown in Figure 3.18, and descriptor word input feed register (0xFC) for programming the injection pattern.

The SafeTI programming process consists of word-by-word writing each descriptor, in execution order. Descriptors to be written are obtained through the APB or AXI descriptor feed register stored in the descriptor buffer, which is part of the FETCH stage. Once the desired injection patterns have been programmed, the injector may be configured and then initialized.

The reset_sw bit is asserted when a new injection pattern needs to be loaded. The current pattern is wiped out, and all circuits are reset except those related to transactions in process. This is necessary to allow for the correct termination of ongoing transactions. On the other hand, the hardware reset puts all circuits in a default state without exceptions. Yet, note that the hardware reset is a SafeTI signal not visible to the software layers.

The SafeTI module features several interruption flags that are propagated through the APB interface. These include interruptions raised due to a network error, generated when the injection network answers with an error, due to an internal error caused by an unsupported encoding, or due to injection pattern completion. Furthermore, descriptor completion can also trigger an interruption, programmed on the first descriptor word as explained before.

SafeTI also features an automatically disabling mechanism, which triggers an interruption by asserting the freeze_irq flag to notify that it has been disabled. SafeTI is disabled by means of a hardware breakpoint of the traffic pattern execution. The conditions that can trigger the interruption are configured by asserting the irq_err_net, irq_err_core and irq_prog_compl for network error, SafeTI error, or injection pattern completion, respectively.

SafeTI can be set in QUEUE mode by asserting the queue_mode flag so that the injection pattern execution loops to the first descriptor after completion. The freeze_irq flag overrides the QUEUE mode, meaning that under the right conditions, the traffic injector is disabled, even if SafeTI is configured to work in QUEUE mode.

Software Interface

The control register of the SafeTI, as well as the descriptor word input feed register used for SafeTI configuration, must be modified only by software components with appropriate privileges. To realize this, the SafeTI registers are mapped in specific physical addresses upon integration in the platform, and the hypervisor (FENTISS' XtratuM Next Generation, also known as XNG, in the particular case of the SafeTI integration in ISOLDE) is in charge of managing privileges to allow only specific partitions to update and read the SafeTI's registers.

The preferred configuration consists of allowing only a single partition to modify the SafeTI's registers, whereas the other partitions cannot access them. XNG guarantees this behavior, leveraging the MMU existing in the NOEL-V cores. This MMU also implements the RISC-V ISA hypervisor extension. Overall, the XtratuM hypervisor provides memory space isolation for the SafeTI's registers, hence achieving freedom from interference, in line with safety standards guidelines for items with integrity requirements.

For allowing a partition of the XNG hypervisor to manage the SafeTI device from a high-level perspective, a driver at the hypervisor level is being developed and integrated by FENTISS, which exposes to the partitions a new set of hypercalls. Hypercalls are services or system calls provided by the hypervisor to the partitions, and requesting them causes a privilege escalation from the partition running at user mode to the core running at system mode. These newly implemented hypercalls, specified in Table 3.21, allow managing the SafeTI device by a partition that has SafeTI access permissions.

In order to define whether or not a partition has access to the SafeTI device, a new attribute in the Xtratum Configuration Files (XCF) has been added. If a partition's XCF includes the "safeti"' attribute, that partition is granted permissions to access SafeTI registers and use the provided

hypercalls without restriction. Otherwise, if the attribute is not present, any attempt by the partition to invoke a SafeTI hypercall will result in an error code indicating an invalid configuration.

Regarding interrupt management, in order to receive from a partition the interrupts generated by the SafeTI device, the XCF of the partition needs to be configured to delegate them. It is also important to note that the partition should allow the interrupts reception, unmask the desired interrupts, and install the desired handlers to be triggered when an interrupt is received, which can be done through other hypercalls implemented within XNG.

Finally, to corroborate the correctness of this software integration, a functional validation phase was carried out. The executed test scenarios successfully demonstrated the expected behavior of the newly introduced hypercalls, ensuring proper configuration of the SafeTI via the descriptor word and control registers. Additionally, the spatial isolation enforced by the XNG hypervisor was strictly verified by asserting that access attempts from unprivileged partitions (those lacking the "safeti" attribute) are effectively blocked and appropriately return the corresponding error codes. Finally, the validation process confirmed the accurate propagation of SafeTI hardware interrupts to the designated partition, ensuring that software handlers are triggered as intended. Consequently, these tests prove that the developed driver provides a secure and interference-free interface for the SafeTI IP.

Hypercall	**Description**
`retCode XSafeTiProgramDescriptor( desc_type, size, attack_addr, count, last, int_en)`	Configures and writes an individual descriptor into the FIFO queue by writing to the descriptor memory feed register.
`retCode XSafeTiConfig( en, queue_en, int_prog_compl, int_error, int_net_error, freeze_int)`	Programs the configuration register of the SafeTI injector.
`retCode XSafeTiReset()`	Resets the injector and clears current configuration.
`retCode XSafeTiGetCounterVal( counter, *value)`	Reads a SafeTI counter (interrupts or accesses).
`retCode XSafeTiCountersReset()`	Resets the SafeTI counters.
`retCode XSafeTiIsRunning( *value)`	Returns whether or not the SafeTI module is running.

Table 3.21: Hypercall table implemented within XNG for SafeTI device support

3.1.8.7 Evaluation Prototype

FPGA

The actual operation of the SafeTI is part of the automotive use case. Hence, in this section we evaluate the overheads caused by the SafeTI in terms of hardware resources and performance costs. For that purpose, we consider a realization with 256 entries in the descriptor buffer, which is a pretty large size enabling complex traffic patterns to be programmed.

Regarding performance, the SafeTI has no impact since it is out of the critical path, hence not affecting computation per se. Moreover, it can be synthesized at the maximum operating frequency

Component	FPGA	SELENE SoC	SafeTI	% vs FPGA	% vs SoC
LUTs	1,182,240	384,211	1,352	0.11%	0.35%
Registers	2,364,480	170,645	1,088	0.05%	0.64%
CARRY8	147,780	5,598	69	0.05%	1.23%
F7 Muxes	591,120	12,712	0	0.00%	0.00%
F8 Muxes	295,560	2,546	0	0.00%	0.00%
CLB	147,780	71,021	296	0.20%	0.42%
LUT as Logic	1,182,240	380,104	1,352	0.11%	0.36%
LUT as Memory	591,840	4,107	0	0.00%	0.00%
Block RAM Tile	2,160	308	1	0.05%	0.32%

Table 3.22: SafeTI - Area overheads

allowed by the FPGA used (Xilinx VCU118), so it has no impact on the operating frequency.

In terms of resources, the SafeTI requires limited FPGA resources in absolute terms, as shown in Table 3.22. In particular, the highest overheads relate to the number of LUTs (1,352) and registers (1,088). In order to put these figures in context, we compare them against the total resources available in the FPGA, namely a Xilinx VCU118 device (see column "% vs FPGA" in the table), as well as against the total resources used by the SELENE SoC where the SafeTI is integrated (see column "% vs SoC" in the table). Focusing on the main resources, we observe that the SafeTI uses only 0.11% of the LUTs available in the FPGA and 0.35% compared to the SELENE SoC. Regarding registers, those figures are 0.05% and 0.64% respectively.

Note that the size of the SafeTI scales solely with the size of the descriptor buffer. The cost of the buffer is tiny if using a 16-entry one, which we can realize using LUTs and registers. However, larger buffers are better accommodated in Block RAM (BRAM) tiles. The 256-entry buffer requires a single BRAM tile in the VCU118 FPGA, out of the 2,160 available. This provides an indication of how large such a buffer can be made. For instance, a (huge) 4,096-entry buffer would require 16 BRAM blocks only, still below 1% of the BRAM tiles available in the FPGA.

3.1.8.8 State-of-the-Art Comparison

Arm

AMD's AXI Traffic Generator (ATG) [1] is the most relevant component against which the SafeTI can be compared. While the ATG is not an Arm component per se, it has been conceived to work solely for Arm's AXI interfaces, so we describe it within the Arm section. While collecting the hardware costs of the ATG is not a trivial task, as shown before, the SafeTI incurs very low hardware costs. Hence, although a comparison between ATG and SafeTI could show that SafeTI overheads are higher than those of ATG in relative terms for some setups, SafeTI overheads in absolute terms are anyway small.

Regarding features, we perform in this section a qualitative comparison, which we detail in Table 3.23. The main takeaway message is that, despite in several features they offer very similar capabilities, our SafeTI compares favorably due to the following main reasons:

1. The SafeTI allows some additional types of traffic injection in terms of injection upon specific events, which is key for some specific validation activities.

Feature	ATG	SafeTI
Can inject traffic of variable sizes	YES	YES
Can inject traffic in burst / no-burst modes	YES	YES
Can set delays between traffic injections	YES	YES
Includes a buffer to program traffic to be injected	YES	YES
Can repeat traffic injections	YES	YES
Notifies with an interrupt when injection is complete	YES	YES
Can inject traffic when specific traffic is observed in the interface	NO	YES
Supports multiple interfaces for injection	YES. AXI4 and AXI4-Stream	YES. AXI4 and AHB
Can be extended to other interfaces for injection	Unclear	YES
Can be programmed through multiple interfaces	NO. Only AXI4-Lite	YES. AXI4 and APB
Programmability occurs accessing a specific address range	YES	YES
Open-source permissive license.	NO. Proprietary.	YES. MIT license. Others can be offered.
Can be integrated in any device.	NO. Only AMD FPGAs with AXI4 interfaces. Unclear whether other interfaces are possible.	YES. AMBA AHB and AXI already supported. Only a wrapper needs to be changed to support any other interface.

Table 3.23: SafeTI - Comparison against ATG

2. The SafeTI has an open-source and permissive (MIT) license, whereas the ATG is proprietary.
3. The SafeTI can be deployed on any FPGA or ASIC unconstrainedly, whereas the ATG can only be used in AMD FPGAs.

3.1.8.9 Requirement Compliance

Table 3.24 provides a comparison between the obtained results and the corresponding IP requirements from WP1. The table summarizes the verification outcomes for each requirement and indicates the achieved level of compliance.

Table 3.24: Requirement Compliance Matrix SafeTI

REQ-DOC	REQ-ID	REQ-TYPE	REQ-Description	Verification Method	Activities / Justification	Compliance (C / PC / NC)
D1.4	SafeTI-01	F-HW_ARCH-STD	Support for AMBA AXI	Module programming and traffic monitoring	Traffic patterns can be programmed, and injected traffic observed in the AXI interface (e.g., by means of simulation)	C
D1.4	SafeTI-02	F-HW_ARCH-STD	Flexible traffic patterns supported (read/write, variable burst length, variable inter-request delay)	Module programming and traffic monitoring	Traffic injected in isolation matches the patterns programmed (e.g., observed in the AXI interface by means of simulation)	C
D1.4	SafeTI-03	NF-HW_UARCH-PR	Traffic injected by a given core decreases linearly with the size of the SafeTI injected bursts	Module programming and traffic monitoring	A core sending sustained requests through the AXI interface with N cycles between consecutive requests observes an inter-request delay of (B-N) cycles, where B is the minimum number of cycles needed to send a data burst of the size programmed in the SafeTI	C
D1.4	XNG-0200	F–SW_RT-FEA	XNG should allow the configuration of the SafeTI IP to the users	Testing with dedicated benchmarks	Different configuration options have been tested by using XNG hypervisor hypercalls, and SafeTI IP has been set up accordingly	C

3.1.9 Safety and Security Control Unit (SSCU) – IFX

3.1.9.1 IP Card

Basic Info		
IP name	Safety and Security Control Unit	
License	Open Source: CERN-OHL-S (strongly reciprocal)	
Repository	https://github.com/Infineon/safety-security-control-unit-ip	
Architecture		
Clock	Number of clock domains	1
	Synchronous with system	Y
	Clock generated internally	N
Ctrl Interface	ISA extension?	Custom CSRs required
	Memory mapped?	N
	Protocol	CSR based
	Address Map	Custom CSR addresses according to platform
Initiator Interface	Protocol	CSR based
	Cached?	N
	IOMMU?	N
Interrupts	Interrupts	Y
Microarchitecture		
Parametrization	Parametric no. cores?	N
	Parameteric config?	RTL from a parametrized generator
Programmability	Contains programmable cores?	N
	ISA	RV32I + Custom CSRs
Software		
Compiler	Requires specialized compiler?	N
	Compiler repository	N/A
Hardware Abstraction Layer	N/A	
High-level API	Is there a high-level API/SDK?	N
	SDK repository	N/A
	Is there a domain-specific compiler?	N
Integration		
IP Distribution	Manifest type (if any)	Bender.yml
	Standalone simulation?	Y
	(if standalone sim) SW requirements?	Vivado
	Integration documented / examples?	Y
Synthesis	Is the IP synthesizable?	Y
	FPGA synthesis example available?	N
	ASIC synthesis example available?	N
Simulation	Closed-source simulation?	Y (Xcelium or Vivado)
	Open-source simulation?	N
Evaluation	PPA results available?	Y

3.1.9.2 General Information

The Safety and Security Control Unit (SSCU) serves as a centralized control unit. Its role is to collect all error signals coming from various sources on the chip, analyze and process them, and in turn alarm the system when a critical state is reached.

This implementation of the SSCU is architected to be tightly integrated with a RISC-V CPU core, enabling a highly efficient configuration for systems with a single processing core. In such systems, where the RISC-V core serves as the sole initiator for accessing the SSCU, it becomes unnecessary to connect the SSCU to the shared bus interconnect. Unlike multi-core designs or systems with multiple initiators, there is no requirement for other components to access the SSCU's registers. Consequently, leveraging RISC-V CSRs as the primary interface for the SSCU becomes a more optimal solution.

By decoupling the SSCU from the bus interconnect, the overall complexity of the bus infrastructure is reduced. Notably, this eliminates the need for address decoding logic specific to the SSCU, thereby streamlining the datapath and reducing the critical timing path associated with bus operations. Furthermore, accessing the SSCU via CSR instructions bypasses the latency and contention typically associated with shared bus transactions, offering significantly lower access times. The RISC-V CSR mechanism inherently allows for efficient read and write operations, as they are tightly coupled to the instruction pipeline of the CPU core.

Additionally, the tight physical coupling between the SSCU and the RISC-V core introduces further architectural advantages. By situating the SSCU in close proximity to the processor core, wire lengths are minimized, which in turn reduces signal propagation delays and mitigates power dissipation associated with long interconnects. This proximity optimizes both timing and energy efficiency, contributing to improved system performance.

Figure 3.19 provides an example of one possible integration of the SSCU in a RISC-V core. The error signals that are external to the core perspective would be configured as asynchronous interrupts, while the error signals internal to the core would be configured as a custom synchronous exception to enable the quickest way to react.

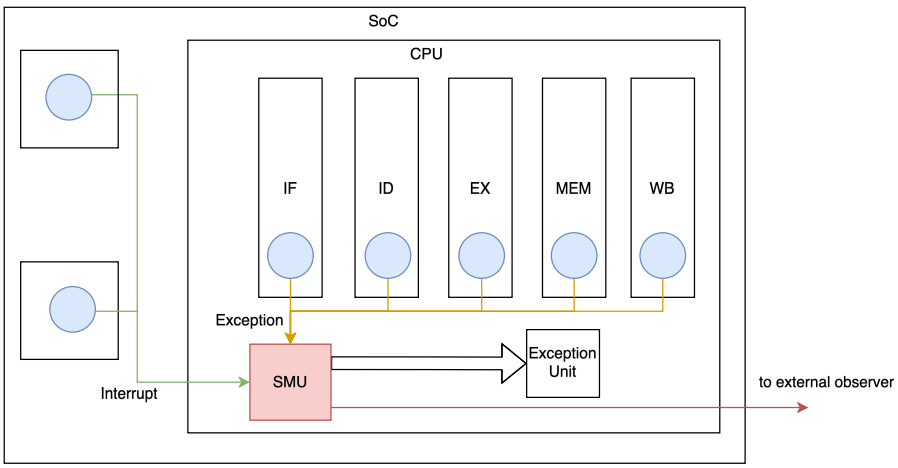


Figure 3.19: Example of SSCU integration in a 5-stage RISC-V core.

3.1.9.3 Purpose and Scope

The SSCU is designed as a universal module capable of integration into a wide range of safety-critical platforms and applications. Its primary role is to manage the handling of single-event faults,

ensuring robust system operation in environments prone to transient errors caused by radiation or high-energy particles.

The SSCU is solely dedicated to fault handling and does not directly engage in fault detection or correction. This design choice ensures that the SSCU remains flexible and independent of specific fault models, allowing it to integrate seamlessly across various platforms and applications. Operating at a higher conceptual level, the SSCU focuses exclusively on error management, regardless of the mechanisms used for fault detection and correction. When errors are detected, the SSCU triggers specific traps, prompting the core to jump to designated trap handlers. These trap handlers are specialized software components responsible for addressing the errors that initiated the traps. Meanwhile, tasks related to error detection and correction are handled by the chip's internal hardening mechanisms.

We refer to these hardening mechanisms simply as safety nodes. Each node is directly connected to the SSCU. For these nodes to be compatible with the described SSCU, they must have the following attributes:

- *identifier*: unique identifier for each node within the system.
- *function*: specific original functionality performed by the node, such as error correction or error detection.
- *error detection/correction mechanisms*: the implementation-specific method used by the node to detect or correct errors.
- *trap specifier*: an additional specifier that indicates which trap handler of the CPU is responsible for handling the particular error generated from this node.

Different errors have varying levels of severity and importance in terms of system functional safety. To address this, nodes are categorized into groups based on the severity and handling requirements of the errors they detect. Each of these groups requires a dedicated logic for its management. By centralizing all error signals and processing them in group units, the SSCU reduces the number of individual error notifications by generating a single alarm for each group. In turn, each alarm can be configured to trigger internal actions or notify external fault handling systems via a fault signaling protocol.

The supported errors can be categorized on the basis of their status in:

- *uncorrected errors*: errors detected but not corrected by the node, which will propagate in the next cycle and possibly contaminate the hardware state.
- *corrected errors*: errors detected and corrected by the node, preventing them from affecting subsequent cycles

Figure 3.20 shows an example design of a node with error detection and correction capabilities. This node has the ability to correct errors using spatial redundancy. It features an Encoder, Register Bank, and Decoder.

The Encoder processes input data *Data_in* and an enable signal (*Enable*), then forwards it to the Register Bank, which includes three protected registers (Reg0, Reg1, Reg2) implementing TMR. The Decoder uses a voter mechanism to correct errors and outputs the corrected data *Data_corrected_out*. Additionally, a non-equivalence comparator in the Decoder generates an *alarm_out* signal if discrepancies are detected. This *alarm_out* signal is wired towards the SSCU and whenever an alarm is detected, specific processing to ensure robust error management of

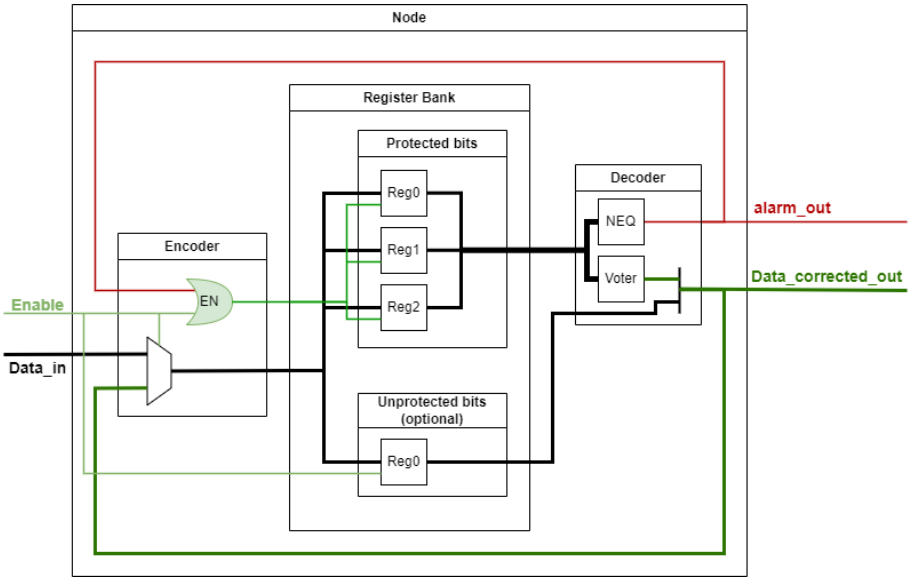


Figure 3.20: Node with Error Correction (TMR).

the error is performed.

The described logic is not the only type of logic used for error detection or correction; it is just shown for illustration. Any other mechanism used also works as long as it is able to alarm the SSCU accordingly.

3.1.9.4 Refined architecture description

The architecture of the SSCU remains largely the same as described in the previous deliverable. Minor changes have been done to fix some bugs that were found during testing.

Error Groups

The SSCU requires the incoming errors to be categorized into groups. Each group will assert a dedicated alarm.

Figure 3.21 illustrates the concept of error groups within the SSCU. Nodes are categorized into distinct groups based on the severity and/or type of errors they produce. Each group is processed independently to enable locally prioritized error handling. In the example provided, two groups are displayed. Group 1 addresses a subset of errors with specific characteristics, all of which are mapped to a common trap. The output of this group triggers alarm 1, which indicates an error condition based on the aggregated errors from the nodes within the group. Likewise, Group 2 manages a different subset of errors, potentially involving distinct levels of severity or unique handling requirements. The output from this group triggers alarm 2. The SSCU aggregates these error signals and interfaces with software, allowing for real-time monitoring and control. The software interface can read and write CSRs corresponding to each error group.

Implementing an Error Group

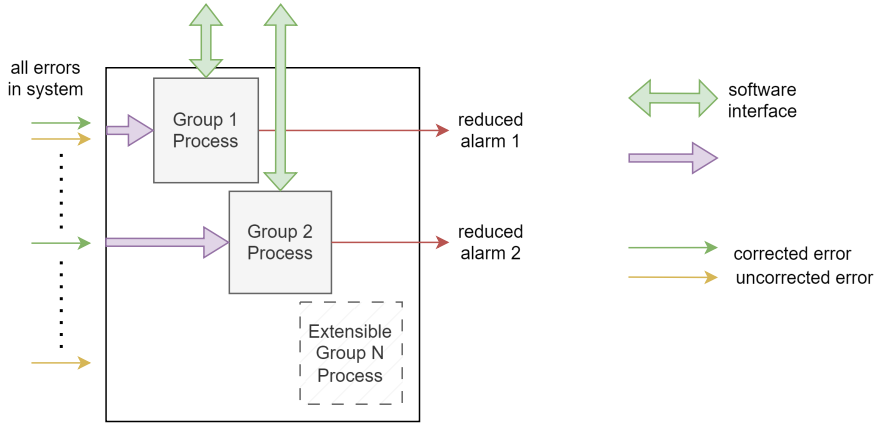


Figure 3.21: SSCU Consisting of Multiple Error Groups.

The Group Process unit is the module where most of the logic to handle errors is located and also the module that interacts with the software. This unit provides 3 key features: *error prioritization*, *alarm flood prevention,* and *diagnostic ability,* which are in turn explained below.

First, it is crucial to implement a mechanism that prioritizes different types of errors. This ensures that critical errors are addressed promptly while less severe issues are managed appropriately. Additionally, the unit must prevent alarm flooding by avoiding the continuous sending of alarms for the same cause, which can overwhelm the system and obscure significant new errors. Furthermore, the error batch processing unit needs to have mechanisms for enabling and disabling error reporting based on the system's operational requirements. This flexibility is vital for maintenance and testing purposes. Another requirement is the storage of error information for software diagnostics.

Figure 3.22 depicts the implementation details of the Group Process unit.

Alarm flood prevention is achieved through a report control mechanism and an error counter and compare mechanism, which reduces the frequency of servicing the same error. As illustrated in the image, corrected errors do not immediately trigger an alarm. Instead, they increment a counter, and only when the counter reaches a predefined threshold does the SSCU trigger an alarm to notify the system of frequent error occurrences. Although these errors are being corrected, their high recurrence may indicate potential system corruption.

Prioritization of the errors is achieved using a priority encoder. Alarms are encoded accordingly, and this value is stored in the control status register for diagnostic purposes. This allows the software to retrieve information about the type of error and the specific node where it was detected, allowing it to take appropriate corrective actions.

There are three main categories of alarms that the SSCU can generate:

- *uncorrected error alarm* : This alarm is triggered when an uncorrected error is detected. Uncorrected errors indicate serious faults that cannot be automatically resolved by the local safety mechanism, requiring system measures such as triggering a CPU exception to step in.

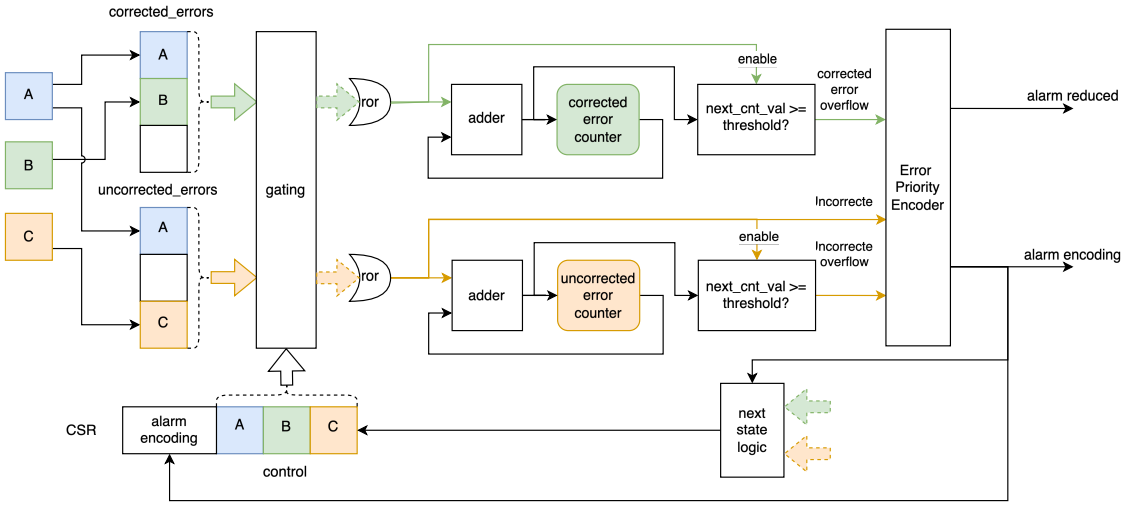


Figure 3.22: High-Level Schematic of Group Process Unit.

- *uncorrected overflow alarm* : This alarm is activated when the counter for uncorrected errors overflows. It occurs when the number of uncorrected errors exceeds the configured threshold, indicating that the system itself is experiencing catastrophic issues where external action needs to be taken.
- *corrected overflow alarm* : This alarm is triggered when the counter for corrected errors overflows. The purpose of this alarm is to notify that the system is experiencing a high rate of correctable errors, which, while not immediately critical, may indicate underlying issues that could escalate if not addressed.

Error Diagnosis

In the presence of an error, whether corrected or uncorrected, the software can diagnose the issue by leveraging the hierarchical structure of the SSCU. The diagnosis process begins with the fact that each error group will trigger a different trap. Each error group corresponds to a specific set of nodes categorized based on the severity and type of errors they generate, and by pinpointing the exact error group, the system narrows down the possible sources of the error.

Once the error group is identified, the next step is to determine the specific node within that error group that has triggered the alarm. The software reads the CSR of the batch group to further diagnose the issue. The CSR contains detailed information about the error signals and their origins, including the node error report control bits for each node within the batch. If a node error report control bit is disabled, it indicates that the corresponding node is the source of the alarm. The software can then use the alarm encoding stored in the CSR to identify the type of error—whether it was a corrected error, an uncorrected error, or an overflow condition.

3.1.9.5 Interfaces

As shown in Figure 3.19, the SSCU needs to interact with the Exception Unit of the RISC-V core. The implementations of the Exception Unit might vary from core to core, but the idea is to configure different alarms that SSCU generates as custom traps. Once these alarms are generated, the Exception Unit is responsible for forcing the core into executing the relevant trap handling routine, which in turn will diagnose the SSCU to detect the error and then take further actions.

To achieve full integration of the SSCU with a given core, the core must provide the required infrastructure that enables the SSCU to update a set of custom CSRs. The given core must also implement these CSRs in a predefined specific structure. This in turn requires a software interface capable of accessing these CSRs to perform the following operations:

- *Error Control*: which allows the system to enable or disable error reporting based on specific conditions or thresholds. It prevents the system from being overwhelmed by continuous alarms from repetitive or non-critical errors.
- *Error Diagnosis*: which allows the system to identify the root cause of the error and evaluate its criticality so it can take appropriate measures.

The addresses of the CSRs can vary from platform to platform, based on the availability of the addresses and the number of groups implemented in the SSCU. However, they must be within the ranges reserved for *Custom read/write Machine-Level CSRs*, as defined in the official RISC-V specification.

Each group must have a dedicated CSR that is used to configure the respective group. The two least significant bits of this CSR contain information on the alarm type. Since uncorrected errors have the potential to propagate and contaminate the circuit state, they need to be given higher priority. Therefore, alarm encoding is introduced to indicate the type of error that needs to be processed. The exact encoding is shown in Table 3.25.

Uncorrected Error	Uncorrected Overflow	Corrected Overflow	Encoding
1	1	X	11
1	0	X	10
0	0	1	01
0	0	0	00

Table 3.25: alarm Encoding Table.

The rest of the CSR is composed of single bitfields, with each bitfield mapped to a single error node. When such a bitfield is cleared, the error reporting for that specific error node is disabled. When SSCU generates an alarm, it automatically clears the bitfield of the node responsible for the alarm. This way, the SW can simply detect the node that raised the error by simply identifying the cleared bitfield in the CSR. This organization limits the number of nodes one group can have to a maximum of 30.

3.1.9.6 ISA specialization

This unit does not necessitate the introduction of a specialized ISA extension for its operation. Instead, it integrates seamlessly with the existing RISC-V architecture. A minimal extension is

required to the ISA, which consists of new synchronous exceptions together with their related bitfields in the standard CSRs and a set of new custom CSRs. These custom CSRs should be allocated specific addresses within the designated range reserved for user-defined CSRs, as per the RISC-V specification.

3.1.9.7 Evaluation prototype

FPGA

The SSCU is integrated into an in-house RV32I_Zicsr_Xir RISC-V core (Xir is the custom Indexable Register Extension). The core is augmented with specific hardening mechanisms applied to a set of selected registers. These hardening mechanisms are designed to improve system fault tolerance by detecting and mitigating potential errors [36]. Fault injection mechanisms are also included in the core, which introduces faults into the hardened registers at randomized intervals. When faults occur, the hardening mechanisms detects the errors and propagates the information towards the SSCU. Upon receiving this signal, SSCU initiates a controlled response by forcing the core to execute specific trap handlers. These trap handlers are responsible for executing predefined recovery actions based on the type of fault detected. This entire flow is simulated using Vivado, and checks are performed on runtime to evaluate the correct functioning of the Safety Unit.

To further evaluate the system, the design is also implemented on a basic FPGA platform for hardware-based testing [45]. Faults in the FPGA prototype are manually injected via external inputs, such as buttons, allowing for deterministic testing of fault scenarios. The system's response to these faults is visualized using LED indicators, providing a clear representation of the fault detection and recovery processes in real time.

To obtain an initial estimate of the power, performance, and area (PPA) characteristics, the open-source SSCU was synthesized as a standalone design targeting a Xilinx Artix-7 35T FPGA (xc7a35ticsg324-1L). In this context, standalone synthesis means that only the released SSCU implementation was included, without integration into a processor or SoC. The synthesized SSCU is a small-scale prototype containing only two Group Processes, each supporting three nodes. The architecture is scalable and can be extended with additional Group Processes and nodes according to the requirements of the target SoC. The resulting FPGA resource utilization of the small-scale prototype is presented in Table 3.26.

Table 3.26: FPGA resource utilization

Site Type	Used	Fixed	Prohibited	Available	Utilization (%)
Slice LUTs	14	0	0	20800	0.07
LUT as Logic	14	0	0	20800	0.07
LUT as Memory	0	0	0	9600	0.00
Slice Registers	10	0	0	41600	0.02
Register as Flip-Flop	10	0	0	41600	0.02
Register as Latch	0	0	0	41600	0.00
F7 Muxes	0	0	0	16300	0.00
F8 Muxes	0	0	0	8150	0.00

Regarding timing performance, the relatively small size of the prototype results in an estimated

maximum operating frequency of approximately 408 MHz. This value should therefore be considered preliminary, as the final achievable frequency may change after integration of the SSCU into a RISCV core, especially if it happens to be part of the critical Path.

3.1.9.8 State-of-the-Art Comparison

Performing a quantitative comparison against third-party IP that integrates functional-safety management and hardened CPUs is challenging. Many Arm-based commercial products include such subsystems—for example, Infineon's AURIX Safety Management Unit (SMU), NXP's Fault Collection and Control Unit (FCCU), and TI's Error Signaling Module (ESM). Analogous blocks exist in several commercial RISC-V–based MCUs and SoCs that aggregate faults, supervise diagnostics, and orchestrate escalation. However, these implementations are proprietary, with limited public collateral and measurements, and they vary significantly in integration, technology, and configuration.

In the open-source RISC-V space, there is no widely adopted, safety-certified, flagship IP that provides a subsystem suitable for rigorous quantitative comparison with the SSCU. Consequently, we provide a qualitative comparison between the features offered by our SSCU module and the respective extensions defined by Arm and RISC-V architectures.

Arm

Arm has introduced a Reliability, Availability, and Serviceability (RAS) extension [6] that specifies architectural features designed to enhance error management and system reliability for Arm systems. Key capabilities include defining system registers for recording and reporting errors (e.g., severity levels, error counters, and timestamps), error synchronization events for effective error isolation, and non-maskable error exceptions to facilitate error recovery. This specification supports the development of a broad spectrum of systems, ranging from basic implementations that perform a system reset upon error detection to highly mission-critical systems with advanced fault-handling mechanisms.

The extension is defined at two levels. At the Processing Element (PE) level, it specifies RAS features implemented within an execution context, such as a core or hardware thread. At the system level, it specifies how errors are detected, recorded, and reported across the broader SoC. In contrast, the SSCU (Safety and System Control Unit) operates as a centralized unit positioned near the CPU. It primarily manages errors originating from the RISC-V core while also aggregating error data detected throughout the rest of the SoC, subsequently initiating appropriate handling actions.

Comparison of error types

As the RAS specification is tailored for high-end, mission-critical systems, it provides a significantly more comprehensive error classification compared to the SSCU, which only distinguishes between corrected and uncorrected errors. The RAS error classification includes the following states:

- **Corrected Errors (CE):** These errors are detected and corrected, ensuring they are not silently propagated. Under normal circumstances, the error does not affect the system state after correction.

- **Deferred Errors (CE):** These errors are detected but not corrected. Additionally, they are deferred, meaning they do not immediately trigger an action, because the error is not directly consumed.
- **Uncorrected Errors (CE):** These errors are detected but not corrected. They trigger an immediate action. They are classified into further sub-types:
  - **Unrecoverable (UEU):** Execution cannot be recovered from the preferred return address of the exception because the error has already been consumed.
  - **Recoverable (UER):** Execution can be recovered from the preferred return address of the exception because the error has not been consumed and it is repaired by the handler.
  - **Restartable (UEO):** Execution can be recovered from the preferred return address of the exception without any repair action taken by the handler.
  - **Uncontainable (UC):** These errors cannot be isolated to an application or VM.

Comparison of handling events

For errors detected or consumed internally by a PE, the PE generates an error exception. The primary type of exception generated is an external abort. In specific cases, RAS allows SError (system error) interrupts to be fired as well. For errors that are recorded outside the PE, the respective component can trigger one of the following types of interrupts:

- Fault Handling Interrupt
- Error Recovery Interrupt
- Critical Error Interrupt
- In-Band Error Response

This behavior is comparable to the SSCU. As previously mentioned, errors detected in peripherals trigger asynchronous exceptions (interrupts), while errors detected inside the CPU itself result in synchronous exceptions. However, unlike RAS, the SSCU does not categorize interrupts into specific types. Instead, the implementation of the interrupt handler determines the nature and functionality of each interrupt.

Additionally, RAS introduces error synchronization events, such as the Error Synchronization Barrier (ESB) instruction, which is absent in the SSCU. These synchronization events ensure the CPU checks for any unrecoverable error UEU before proceeding with execution. This guarantees that any UEU caused by earlier instructions is either addressed immediately or marked as pending before subsequent code is executed. Such error synchronization events are used, for instance, at the entry and exit points of exception handlers to isolate and manage errors effectively.

Comparison of Error Recording and Diagnosing

The RAS specification incorporates a detailed and systematic approach to error recording. Upon error detection, relevant diagnostic information is immediately stored in dedicated registers, which can either be system registers or memory-mapped registers, depending on the system design. In contrast, our SSCU relies on CSRs to enable faster access to diagnostic information while being situated closer to the CPU. This approach also reduces the load on the interconnect by not requiring additional memory-mapped registers.

As for the type of information stored that helps handle the error, key distinctions are as follows:

- **Error Type:** Errors will set a specific bitfield that tells the specific type of the error.

- **Error Identification:** Every detected error must store sufficient information to enable the system to identify the originating unit. While this requirement is mandatory, RAS leaves the choice of the specific mechanism open, allowing it to be implementation-specific. In contrast, the SSCU identifies errors through a combination of the invoked trap handler—which provides information about the group of nodes—and the corresponding node ID. The node ID is determined by the index of the bitfield set in a dedicated CSR, providing a clear way to trace the error source.
- **Error Counters:** RAS provides an optional counter for tracking correctable errors, which can be enabled or disabled as needed. By comparison, the SSCU's counter is hardwired to a fixed value, is always enabled, and only resets after an overflow event.
- **Memory Address and Timestamps:** RAS supports the capture of associated memory addresses and optional timestamps indicating when the error was detected. Both these features are absent in the SSCU.
- **Error Codes and Severity Levels:** RAS records an error code to identify specific errors and includes explicit information about error severity. While SSCU implements a mechanism to encode specific error types, severity is implicitly managed through prioritization schemes, where errors with higher severity levels are assigned higher priority.

RISC-V

The RAS capabilities in RISC-V platforms are standardized through the RAS Error Recording Register Interface (RERI) specification [62]. This specification establishes rules and guidelines for recording and reporting hardware errors in RISC-V-based systems. Its key features include categorization of error severity classes and standard error codes, memory-mapped error-record registers and error record banks, corrected error counting, etc. Similar to the Arm RAS Extension, the RERI specification is designed to support a diverse range of systems, from low-end embedded devices to high-end, server-class platforms.

Since the registers for error reporting are memory-mapped, the corresponding logic implementing the RAS features is centrally located within the SoC and connected to the overall SoC interconnect. This centralized approach enables multiple components across the SOC to implement RERI-compatible error detection mechanisms and allows them to report to a central unit for error recording and handling. This design approach significantly differs from our SSCU, where the unit is closely coupled to the CPU core but is still used to collect error signals from across the SoC.

Comparison of error types

RERI also defines error classifications, which are similar to the categories used in our SSCU, encompassing both corrected and uncorrected errors. The key distinction lies in the introduction of Deferred Errors in RERI. Unlike the SSCU, where all uncorrected errors trigger immediate actions, Deferred Errors in RERI allow high-performance systems to continue operation as long as the erroneous value is not consumed or does not alter critical system states. The error types defined by RERI are summarized as follows:

- **Corrected Errors (CE):** The error was detected and corrected by the employed hardware mechanisms.
- **Uncorrected Errors Critical (UEC):** Errors that are detected without any deferring mechanism and require immediate remedial action to prevent potential system failure.
- **Uncorrected Errors Deferred (UED):** Detected errors that are not corrected but are per-

mitted to propagate to the requester of the data without triggering immediate action.

Comparison of handling events

When an error is detected and must be managed, RAS handlers must be invoked. The central logic responsible for the RAS features signals the core to invoke the correct RAS handler. The handlers are responsible for determining and executing the appropriate corrective or recovery action for the particular error. These handlers can be triggered in various forms, depending on the system design. They consist of the following:

- **Interrupts:** RAS events primarily trigger interrupts to notify the processor of the error.
- **Non-Maskable Interrupts:** Designers have the option to configure specific high-priority RAS signals to generate Non-Maskable Interrupts, ensuring these signals cannot be ignored.
- **Hardware Error Exceptions:** Certain critical uncorrected errors (UEC) may directly trigger hardware error exceptions to respond to severe faults.

Unlike the Arm RAS specification, where interrupts are categorized into distinct types by hardware, RERI treats all interrupts as the same type. It differentiates between error severities using interrupt priorities and the specific action assigned to each handler. This approach is similar to the interrupt handling methodology employed in our SSCU.

Comparison of Error Recording and Diagnosing

RERI also defines the error recording mechanism, referred to as "error reporting" in the specification. This mechanism is implemented through "error banks," which are memory-mapped components designed to store detailed information about detected errors. The concept is straightforward: the errors will change specific states stored in these error banks, and later the RAS handler will access the relevant registers to gather information about the error and determine the appropriate corrective or recovery action.

A system-on-chip (SoC) may include multiple error banks, with each bank dedicated to a specific component that has error detection capabilities, such as the CPU, memory, or other subsystems.

- **Error Class:** The error class is explicitly recorded in the register bank. Specific bits are assigned to represent the error class, such as Corrected Errors (CE), Uncorrected Errors Deferred (UED), or Uncorrected Errors Critical (UEC).
- **Error Counting:** Error counting is an optional feature that may or may not be supported by the platform. When supported, the counter can be enabled or disabled through software configuration. If the counter value surpasses a predefined threshold, an overflow signal is generated.
- **Optional Additional Information:** The error banks can store additional, optional information in dedicated registers, including the memory address where the error occurred, the type of transaction involved, and even platform-specific details that vary from one system to another.
- **Error Code:** Each recorded error is associated with an error code. The specification defines 28 standard error codes while also allowing the encoding of up to 255 custom error codes. These error codes denote specific scenarios, such as "Corrupted Data Access," "Cache Error," or "Interconnect Data Error." In contrast, our SSCU organizes similar information implicitly within the grouping of errors rather than explicitly encoding it in a dedicated register

field.

In addition, RERI includes control registers that allow enabling or disabling both error recording and error signaling. This provides flexibility in managing how errors are handled and reported within the system.

In comparison, our SSCU also allows error recording to be disabled by writing to the respective group CSR. However, in the SSCU, the same control bit simultaneously governs both error recording and error signaling. As a result, it is not possible to log errors without generating an immediate interrupt, which differs from RERI's more decoupled and configurable approach.

To conclude, the SSCU represents a lightweight and efficient approach to safety monitoring in RISC-V systems. In contrast to RERI, which is designed as a comprehensive SoC-wide RAS framework, the SSCU is tightly coupled to the processor and focuses on the functionality required for fault detection, reporting, diagnosis, and handling. This architecture provides shorter communication paths between the core and the safety logic, reducing latency and enabling operation at higher frequencies. At the same time, the reduced feature set results in a smaller silicon footprint and lower implementation cost compared to a full RERI-based solution. Therefore, the SSCU achieves a favorable trade-off between functionality, performance, and hardware overhead while retaining the essential capabilities required for safety-critical error management.

3.1.9.9 Requirement Compliance

Table 3.27 provides a comparison between the obtained results and the corresponding IP requirements from WP1. The table summarizes the verification outcomes for each requirement and indicates the achieved level of compliance.

Table 3.27: Requirement Compliance Matrix SSCU

REQ-DOC	REQ-ID	REQ-TYPE	REQ-Description	Verification Method	Activities / Justification	Compliance (C / PC / NC)
D1.4	SCU-01	F-HW_ARCH-FEA	Combine the error signals from different units, assembled those and provides the result of the error assembly to a higher-level control unit	Simulations on different abstraction levels (VHDL, RTL)	Verified using Formal Verification and Simulation ; Validated via FPGA demonstrator	C
D1.4	SCU-02	F-HW_ARCH-FEA	Make the upper limit of errors reported configurable	Review	Flow allows to configurable error Threshold	C
D1.4	SCU-03	NF-HW_UARCH-FEA	Harden the design against single bit faults	Fault Simulation	Verfied using Formal Verification and Simulation ; Validated via FPGA demonstrator	C
D1.4	SCU-04	F-HW_ARCH-FEA	Make a minimum threshold for error reporting software programmable	Simulations on different abstraction levels (VHDL, RTL)	Threshold is static; Verfied using Formal Verification and Simulation ; Validated via FPGA demonstrator	NC

3.1.10 Safety Island - Interface Definition – UzL

UzL reports a deviation from the original work plan due to the discontinuation of Task 3.1 (Safety Island – Interface Definition), which was assessed as obsolete following the evolution of the project's objectives. Consequently, the activities originally planned under this task were no longer pursued. The associated activities were redirected to Tasks T3.2 and T4.3 to support the project-wide focus on floating-point unit (FPU) analysis and fault-injection activities. The related FPU development and analysis activities are reported in Deliverable D3.5.

Furthermore, the originally intended Intel HDFIT fault-injection framework was discontinued. To ensure the availability of an open-source fault-injection solution for the RISC-V ecosystem, UzL designed and implemented NAIL[75], an open-source[3] fault-injection framework for RISC-V processors. NAIL enables systematic fault-injection campaigns and supports the analysis of hardware resilience against faults in RISC-V-based designs.

[3] https://gitlab.iti.uni-luebeck.de/soct/nail

3.1.11 Root-of-Trust Unit (RoT) – UNIBO

3.1.11.1 IP Card

Basic Info		
IP name	Root-of-Trust (RoT)	
License	Open-source (SolderPad Hardware License v0.51)	
Repository	https://github.com/pulp-platform/opentitan/tree/mc/astral	
Architecture		
Clock	Number of clock domains	1
	Synchronous with system	Y
	Clock generated internally	N
Ctrl Interface	ISA extension?	N
	Memory mapped?	N
	Protocol	SCMI
	Address Map	N.A.
Initiator Interface	Protocol	AXI4
	Cached?	N
	IOMMU?	N
Interrupts	Interrupts	Y
Microarchitecture		
Parametrization	Parametric no. cores?	N (1)
	Parametric config?	Y
Programmability	Contains programmable cores?	Y
	ISA	RISC-V (CV32E)
Software		
Compiler	Requires specialized compiler?	N
Hardware Abstraction Layer	N/A	
High-level API	Is there a high-level API/SDK?	Y
	SDK repository	sw subfolder of main repository
	Is there a domain-specific compiler?	N
Integration		
IP Distribution	Manifest type (if any)	Bender.yml
	Standalone simulation?	Y
	(if standalone sim) SW requirements?	Bazel + Python + QuestaSim
	Integration documented / examples?	Example in tree/lg/isolde
Synthesis	Is the IP synthesizable?	Y
	FPGA synthesis example available?	Y
	ASIC synthesis example available?	N
Simulation	Closed-source simulation?	Y (QuestaSim)
	Open-source simulation?	N
Evaluation	PPA results available?	N

3.1.11.2 General Information

Silicon Root-of-Trust (RoT) units represent the state-of-the-art in terms of trusted computing and system integrity, as they establish an isolated silicon region with security features for data and code protection. It includes a secure microcontroller, on-board private memory, cryptographic

hardware accelerators, and communication I/O interfaces. It is based on lowRISC's OpenTitan Earl Grey architecture, which UNIBO refactored for SoC integration (TLUL-to-AXI4 initiator port, SCMI mailbox towards the host, no host-accessible target port) and extended with hardware acceleration, turning it into a silicon secure element easily integrable within larger designs.

3.1.11.3 Purpose and Scope

The Root-of-Trust provided by UNIBO within ISOLDE is based on lowRISC's OpenTitan, the first open-source RISC-V-based RoT design, protecting sensitive data and systems against hardware attacks, tampering, and counterfeiting. It includes acceleration units for the Secure Hash Algorithm (SHA), enabling cryptographic hashing (SHA-256 and SHA-3), message authentication (Hash-based Message Authentication Code - HMAC, KECCAK Message Authentication Code - KMAC) and symmetric encryption (Advanced Encryption Standard - AES). It enables a multi-stage secure boot process by ensuring each component is authenticated by its predecessor, leveraging hardware features and cryptographic checks to protect against unauthorized modifications or executions. UNIBO's RoT is meant to be a ready-to-integrate silicon IP able to act as a RoT.

3.1.11.4 Refined architecture

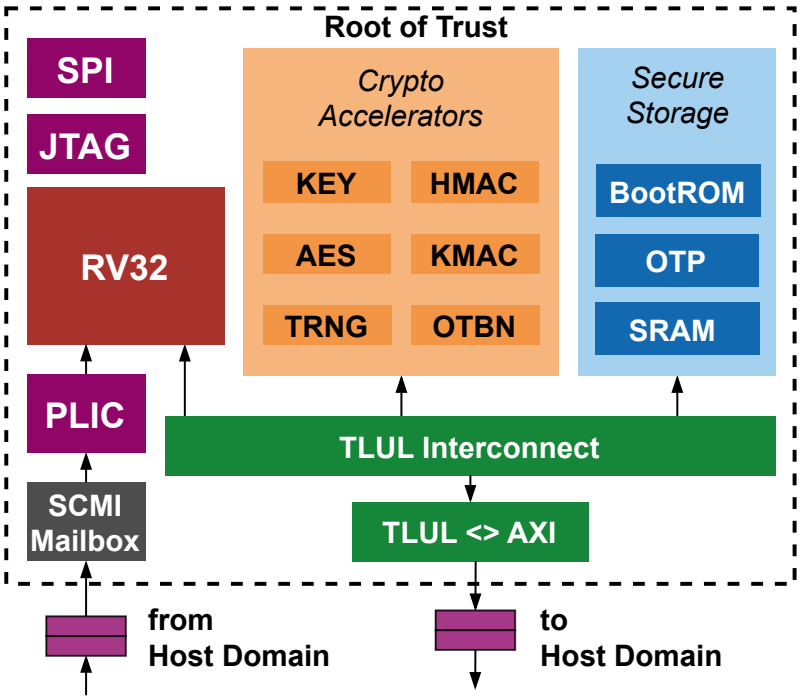


Figure 3.23: Root-of-Trust architecture.

The architecture of the Root-of-Trust (RoT) unit is depicted in Figure 3.23. It is centered around a 32-bit RISC-V core, specifically designed to support embedded and power-efficient applications,

with an emphasis on minimal area usage. At the system level, interconnection is facilitated through the TileLink Uncached Lightweight (TLUL) protocol, ensuring streamlined and secure communication between internal components. With respect to Earl Grey, the UNIBO modifications concern primarily the host interface (Section Interfaces) and the DMA, as well as improvement of the embedded accelerators.

To enhance cryptographic performance, the RoT unit incorporates dedicated hardware accelerators (*Crypto Accelerators*) capable of executing essential cryptographic algorithms. These include AES, SHA-256, SHA-3, HMAC, and KMAC, which are fundamental to the silicon Root-of-Trust's security features. In addition, the OpenTitan Big Number (OTBN) accelerator provides hardware support for asymmetric encryption algorithms that underpin key exchange protocols and digital signature schemes.

The memory subsystem (*Secure Storage*) of RoT comprises a scratchpad SRAM and an embedded ROM, each managed by dedicated controllers to optimize performance and reliability. Critical cryptographic and scrambling keys are generated by specialized hardware accelerators and securely stored within a tamper-proof one-time-programmable (OTP) memory region. This secure memory pairs with the key manager, which governs the handling of hardware identities and root keys, while safeguarding sensitive assets against potential software-based threats.

To enable the application domain to request specific services from the Root-of-Trust (RoT)—such as Trusted Execution Environment (TEE) primitives or cryptographic operations—a secure communication channel has been established between the host and the unit. This channel avoids exposing any direct target port that could be driven by the host, thereby maintaining the integrity of the RoT boundary.

The communication mechanism is implemented using a shared mailbox, which adheres to the Arm System Control and Management Interface (SCMI) standard—a protocol widely adopted in heterogeneous architectures. Interactions between the primary processor and the secure module are restricted to message-based exchanges, conducted through an interrupt-driven mailbox system. In accordance with the SCMI specification, this mailbox provides two dedicated interrupt lines, each connected to the Platform Level Interrupt Controllers (PLICs) of the application processor and RoT, respectively. Notably, the external interrupt generated via this mechanism remains the sole pathway through which the host can directly interact with OpenTitan, ensuring strict control over the communication interface and preserving the security guarantees of the RoT.

Finally, it includes a peripheral subsystem that enables external communication through standard interfaces such as SPI and JTAG, thereby facilitating integration into a wide range of embedded environments.

3.1.11.5 Interfaces

To enable integration of the Root-of-Trust (RoT) unit into broader SoC architectures, an initiator port from OpenTitan's TLUL interconnect is exposed and connected via a dedicated TLUL-to-AXI4 bridge, ensuring compatibility with host platforms using the AXI4 protocol.

This setup allows OpenTitan to access the host's memory map, which is essential for delivering TEE primitives and performing real-time integrity checks. The initiator port connects directly to the SoC's main interconnect, granting the RoT full system visibility while preserving security boundaries.

To safeguard sensitive information within secure subsystems and prevent unauthorized host-core modifications, OpenTitan is deliberately integrated without exposing a target port access to the host. Consequently, interaction from the host to OpenTitan is strictly limited to external interrupts through mailboxes, ensuring rigorous control of the communication interface and upholding robust security guarantees inherent to the RoT.

Finally, the RoT is architecturally designed to interface with the external environment via standard off-chip communication peripherals, such as SPI and JTAG interfaces, thereby ensuring compatibility and facilitating external interaction and debugging capabilities.

3.1.11.6 ISA specialization

The Root-of-Trust unit has no impact on the ISA.

3.1.11.7 Evaluation prototype

ASIC

We implemented the RoT unit in synthesizable SystemVerilog HDL. We targeted GlobalFoundries GF12LP+ 12nm technology to evaluate the RoT in terms of area and performance, using Synopsys Design Compiler for synthesis and Cadence Innovus for place-and-route with a frequency target of 300 MHz. The RoT occupies 0.63mm^2 of silicon, of which the Secure Storage subsystem is 58%, the Crypto Accelerators cores are 22%, the RISC-V core is 6%, and the peripherals are 5%.

FPGA

The design has also been evaluated with an FPGA implementation targeting the AMD Xilinx UltraScale+ VCU118 board with a 20 MHz target frequency, using AMD Xilinx Vivado 2020.1. The RoT occupies approximately 250,000 lookup tables (LUTs), 147,000 flip-flops (FFs), 40 36kb blocks of block RAM (BRAM) and 16 units of UltraRAM (URAM), corresponding to 21%, 6%, 2%, and 2% of the board's available resources, respectively.

3.1.11.8 State-of-the-Art Comparison

RISC-V

The performance of the proposed OpenTitan-based Root of Trust (RoT) secure element was benchmarked against a pure software implementation running on the CVA6 application processor using the BearSSL cryptographic library. Three cryptographic algorithms were evaluated: SHA-256, HMAC, and AES-256 (CBC mode), each tested with payload sizes ranging from 64 bytes to 4 KB.

The software baseline utilized BearSSL's performance-optimized (non-constant-time) implementations executing on the CVA6 64-bit RISC-V application processor at 900 MHz, representing an upper bound for software-only cryptographic performance without hardware acceleration. This configuration requires the application processor to handle all cryptographic operations through sequential instruction execution, resulting in significant computational overhead for hash operations and block ciphers.

Table 3.28 presents the measured performance in cycles per byte (cycle/B) for 4KB payloads. Two memory configurations were evaluated to assess the impact of the memory hierarchy: L1 refers to data residing within the RoT's private scratchpad memory (internal to the secure element), while L3 represents data stored in external system RAM accessed through the platform's memory hierarchy, including a 512KB last-level cache and HyperRAM controller.

Table 3.28: Cryptographic performance: RoT vs CVA6 (4KB payload)

Algorithm	**L1 (RoT Internal)**		**L3 (External RAM)**		**Nominal BW**
	CVA6 [cycle/B]	**RoT** [cycle/B]	**CVA6** [cycle/B]	**RoT** [cycle/B]	[cycle/B]
SHA-256	87.1	1.3	87.1	3.4	1.3
HMAC	93.7	1.4	93.7	3.1	1.3
AES-256	203.7	4.6	203.7	7.8	4.5

Note: Lower cycle/B indicates better performance. CVA6 operates at 900 MHz, RoT at 500 MHz.
Nominal BW represents theoretical accelerator limits from lowRISC specifications.
L3 performance includes DMA overhead for data transfer across RoT perimeter.

3.1.11.9 Requirement Compliance

Table 3.29 provides a comparison between the obtained results and the corresponding IP requirements from WP1. The table summarizes the verification outcomes for each requirement and indicates the achieved level of compliance.

Table 3.29: Requirement Compliance Matrix TitanCFI

REQ-DOC	REQ-ID	REQ-TYPE	REQ-Description	Verification Method	Activities / Justification	Compliance (C / PC / NC)
D1.4	TitanCFI-01	F-HW_ARCH-STD	OpenTitan integration and communication with CVA6 core using a mailbox on the AXI port and interrupt line for mailbox interface with OpenTitan	Functional validation	SCMI mailbox with two interrupt lines to the CVA6 and RoT PLICs; TLUL-to-AXI4 initiator port and no host-facing target port. Validated in RTL simulation and on the VCU118 prototype.	C
D1.4	TitanCFI-02	F-HW_ARCH-PR	Operating frequency ratio CVA6/OT = 1/1	Simulation and FPGA prototype	Single clock domain, synchronous with the system: the 1/1 ratio holds by construction and is demonstrated on the VCU118 prototype.	C
D1.4	TitanCFI-03	F-HW_ARCH-PR	CFI overhead	Simulation and FPGA prototype	Offloading to the RoT reduces the worst-case overhead from 72× in software on CVA6 to +8% with data in the RoT scratchpad and 2.4× with data in external RAM.	C

3.1.12 High-Performance Cache Analysis (CACHE) – SYSGO

3.1.12.1 IP Card

We include an IP card, although in this case the IP was developed by CEA in TRISTAN. We are just using the IP as a component under analysis; here we have answered the questions to the best of our knowledge.

Basic Info		
IP name	CV-Hpdcache	
License	Open-source (Solderpad Hardware License version 2.1)	
Repository	https://github.com/openhwgroup/cv-hpdcache	
Architecture		
Clock	Number of clock domains	1
	Synchronous with system	Y
	Clock generated internally	N
Ctrl Interface	ISA extension?	Y
	Memory mapped?	Y
	Protocol	N
	Address Map	Y
Initiator Interface	Protocol	Cache-Memory Interface (CMI)
	Cached?	Y
	IOMMU?	N
Interrupts	Interrupts	N
Microarchitecture		
Parametrization	Parametric no. cores?	Y
	Parameteric config?	Y
Programmability	Contains programmable cores?	N
	ISA	RISC-V
Software		
Compiler	Requires specialized compiler?	N
Hardware Abstraction Layer	N/A	
High-level API	Is there a high-level API/SDK?	N
	SDK repository	N
	Is there a domain-specific compiler?	N
Integration		
IP Distribution	Manifest type (if any)	Bender.yml
	Standalone simulation?	Y
	(if standalone sim) SW requirements?	Verilator
	Integration documented / examples?	Integration Examples of the HPDCache
Synthesis	Is the IP synthesizable?	Y
	FPGA synthesis example available?	Y
	ASIC synthesis example available?	N
Simulation	Closed-source simulation?	N
	Open-source simulation?	Y
Evaluation	PPA results available?	N

3.1.12.2 General Information

Within the framework of the TRISTAN project for the ISOLDE demonstrator, we performed a comparative analysis between the baseline CVA6 cache architecture and the High-Performance Data (HPD) cache provided by CEA. This analysis focuses on quantifying the throughput gains and latency-hiding capabilities offered by the HPDcache in a real-world system environment.

3.1.12.3 Purpose and Scope

The primary objective of this task is to advance the CVA6 RISC-V ecosystem by integrating and validating high-performance caching architectures. In the context of modern embedded systems, effective cache management is a critical pillar for achieving both safety and security. By integrating advanced caching mechanisms. This study aims to establish a foundation for future analyses (not part of ISOLDE) on cache partitioning.

3.1.12.4 System Integration and Architecture

Caches serve as the high-speed intermediary between the CPU pipeline and the relatively high-latency main memory. In high-performance application settings, the efficiency of this layer determines the overall system throughput and responsiveness.

The architecture of the HPDcache, as illustrated in Figure 3.24, is designed to handle multiple concurrent memory requests. Central to its operation is an arbiter that manages request ports and interfaces with the core cache logic. The core comprises several critical components:

- Data Directory: Manages tag lookups and data storage.
- Miss Status Holding Register (MSHR): Tracks and manages outstanding read misses.
- Replay Table (RTAB): Facilitates out-of-order processing by managing blocked requests and ensures that they are re-executed when conditions allow.
- Write Buffer: Decouples the CPU from the store latency of the underlying bus.

Experimental Setup and Evaluation Methodology

The evaluation was conducted on an FPGA-based CVA6 platform using two distinct bitstream configurations: one integrated with the HPDcache and the other with the standard Write-Through (WTD) cache. These bitstreams were generated using Xilinx Vivado, with specific RTL parameters tuned to highlight the architectural differences in memory-level parallelism.

A detailed breakdown of these hardware configurations is provided in Table 3.30. It is important to note that the differing design parameters, specifically the load buffer depth and AXI transaction ID width (`DcacheIdWidth`) are not arbitrary experimental variables but intrinsic architectural boundaries of the respective RTL implementations. The baseline `WTDcache` relies on a static "Hit-Under-Miss" state machine that cannot structurally utilize more than two load buffer entries or generate concurrent AXI read misses. Conversely, the `HPDcache` requires an expanded 8-entry buffer and an ID width of 3 ($2^3 = 8$ IDs) to support its Replay Table (RTAB) and Miss Status Holding Registers (MSHRs), which actively manage up to eight out-of-order in-flight transactions.

This dual-bitstream approach allows for a direct comparison of how each architecture handles high-frequency memory requests and DRAM latency under a realistic, single-core Linux-managed memory environment.

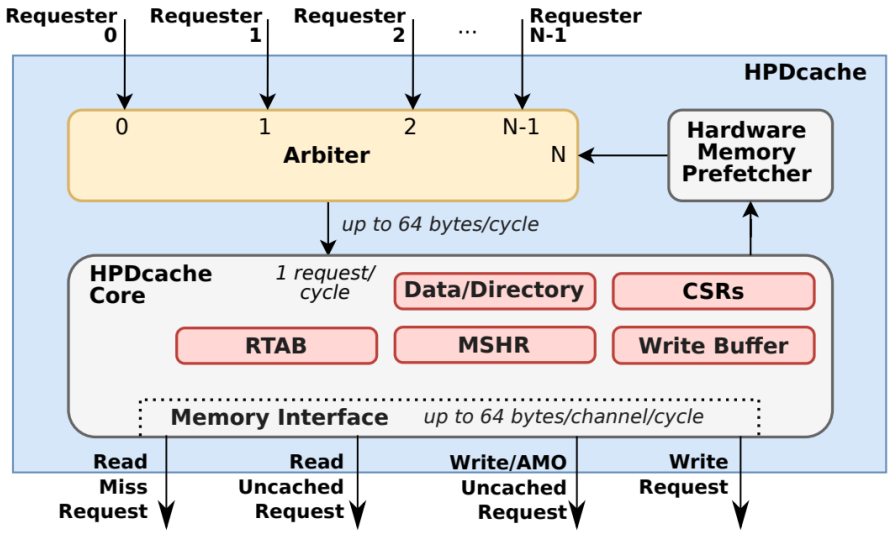


Figure 3.24: High-performance cache block diagram [34]

Table 3.30: RTL Configuration Comparison: WTDcache vs. HPDcache Subsystems

Feature	WTDcache (Baseline)	HPDcache (WT)	Architectural Impact / Limitation
Cache Type	`config_pkg::WT`	`HPDCACHE_WT`	Transition from blocking to non-blocking logic.
Load Buffer Entries	2 (Max Supported)	8	Baseline is structurally limited by single-miss logic; HPD scales to track 8 in-flight misses via MSHRs.
DcacheIdWidth	1 (2^1 IDs)	3 (2^3 IDs)	Baseline only issues 1 active miss; HPD utilizes up to 8 concurrent AXI bus transactions.
Line Width	128-bit	128-bit	Identical spatial granularity for fair comparison.
Write Buffer Depth	8	8	Consistent store-handling capacity across both.
Scoreboard Entries	8	8	Identical instruction tracking in the core pipeline.

To quantify the performance differences, a custom high-pressure throughput benchmark was developed and executed as a Linux ELF binary. The test uses RISC-V Control and Status Registers (`csrr`) to capture exact `cycle` and `instret` counts. To evaluate both load-latency hiding and store-handling capabilities, the benchmark executes a mixed read-write workload with an 8:1 read-to-write ratio (8 byte-wide loads per single byte-wide store) across three distinct memory access patterns:

- `L1_HIT` (Stride 0): Targets a constant address to measure baseline pipeline performance

under ideal hit conditions while steadily exercising the write buffer.
- `LINEAR` (Stride 1): Measures spatial locality and hardware streaming efficiency.
- `RANDOM` (Stride 64): Forces continuous read misses by using a stride that significantly exceeds the 16-byte (128-bit) cache line width configured in the RTL. This 64-byte stride guarantees that every subsequent load instruction lands on a completely new cache line, ensuring a 100% cache miss rate to test the non-blocking MSHR capabilities of the HPDcache under concurrent store traffic.

3.1.12.5 Evaluation Results and Discussion

Performance analysis reveals a substantial advantage for the HPDcache architecture during high-latency memory transactions. As shown in Table 3.31, HPDcache demonstrates superior Memory-Level Parallelism (MLP) in the `RANDOM` test, achieving a 16.6% cycle reduction and an 8.6% reduction of CPI (Cycles per Instruction). While both the baseline `WTDcache` and the `HPDcache` were evaluated in Write-Through (WT) mode, the HPDcache significantly outperforms the baseline in miss-heavy scenarios due to its support for Hit-Under-Multiple-Misses. The baseline `WTDcache` is limited to "Hit-Under-Miss" logic, meaning that the pipeline stalls upon encountering a second concurrent miss. In contrast, the HPDcache leverages its 8-entry load buffer and `DcacheIdWidth` of 3 to track up to eight parallel in-flight misses, effectively hiding DRAM latency by overlapping multiple fetches.

In hit-intensive scenarios (`L1_HIT`), the HPDcache shows a marginal overhead (-1.5% in cycles); because both cache instantiations share identical 8-entry write buffers and Write-Through policies, this minor difference is not a result of write buffer capacity. Instead, it stems from the microarchitectural complexity of the HPDcache. Specifically, routing requests through the HPDcache's multi-port Arbiter and non-blocking scheduling logic (managing MSHRs, RTAB lookups, and store drains) introduces a slight control overhead during tight, continuous hit-and-store loops compared to the simpler, direct hit path of the baseline blocking controller.

Table 3.31: Comparative Analysis of CVA6 Cache Architectures (WTDcache vs. HPD)

Configuration	**L1_HIT** (Stride 0)		**LINEAR** (Stride 1)		**RANDOM** (Stride 64)	
	Cycles	**CPI**	**Cycles**	**CPI**	**Cycles**	**CPI**
WTDcache	27,751,652	2.2630	27,796,927	2.2651	37,941,655	2.7965
HPDcache	28,173,013	2.2880	27,938,575	2.2757	**31,643,377**	**2.5556**
Improvement	-1.5%	-1.1%	-0.5%	-0.4%	**+16.6%**	**+8.6%**

3.1.12.6 State-of-the-Art Comparison

To contextualize CVA6 cache performance against an industry standard, the same high-pressure throughput benchmark was ported to the ARM64 architecture and executed on a Xilinx Kria KR260 platform. This system features a commercial-grade hardened quad-core Arm Cortex-A53 application processor running Ubuntu Linux. Hardware performance counters were accessed di-

rectly via the Linux `perf_event_open` API to ensure an accurate, user-space-only measurement of hardware cycles and retired instructions.

Table 3.32: Cross-Architecture Performance: CVA6 vs. Arm Cortex-A53 (Hardened Silicon)

Architecture	**L1_HIT** (Stride 0)		**LINEAR** (Stride 1)		**RANDOM** (Stride 64)	
	Cycles	**CPI**	**Cycles**	**CPI**	**Cycles**	**CPI**
CVA6 WTDcache (WT)	27,751,652	2.2630	27,796,927	2.2651	37,941,655	2.7965
CVA6 HPDcache (WT)	28,173,013	2.2880	27,938,575	2.2757	31,643,377	2.5556
Arm Cortex-A53 (Quad-Core)	**15,771,471**	**1.5401**	**15,771,265**	**1.5401**	**15,858,349**	**1.5486**

Note: The Arm Cortex-A53 results reflect a dual-issue superscalar pipeline utilizing a Write-Back (WB) L1/L2 cache hierarchy and proactive hardware prefetching, which provides a flat latency profile across all memory access patterns.

As summarized in Table 3.32, the Cortex-A53 achieves a highly consistent CPI of approximately 1.54 in all access patterns, effectively neutralizing the memory penalty in the `RANDOM` test. This performance gap comes down to three fundamental architectural differences.

First, the baseline CPI difference (1.54 vs. 2.28) is driven by the pipeline design. The Cortex-A53 uses a dual-issue superscalar pipeline, allowing instruction-level parallelism. The CVA6 configuration evaluated here is a single-issue, in-order core, establishing a higher hard limit on baseline throughput.

Second, the system topologies differ drastically. The CVA6 is implemented as a single-core system, forcing the Linux OS and the benchmark to share the same pipeline and L1 cache. This inevitably introduces context-switching noise and cache pollution. The Kria's quad-core architecture allows the Linux scheduler to offload background OS tasks to secondary cores, isolating the benchmark thread and providing a largely uninterrupted execution environment.

Third, the memory subsystems utilize completely different latency-handling strategies. The Arm processor relies on proactive memory prediction. The Kria KR260 features a Write-Back (WB) L1 data cache backed by a shared L2 cache and aggressive hardware prefetchers. These prefetchers actively detect stride patterns and load data into the cache before the CPU pipeline explicitly requests it, preventing stalls. Furthermore, the Kria SoC utilizes a hardened DDR4 memory controller in silicon, whereas the CVA6 Genesys-2 setup relies on a soft memory controller (MIG) instantiated in the FPGA fabric to access DDR3 memory.

In contrast to the Arm's proactive miss prevention, the RISC-V implementation relies on a single L1 cache layer and a reactive latency-hiding strategy. Because modern workloads will inevitably defeat predictive prefetchers under heavy memory pressure, state-of-the-art processors must also rely on deep Miss Status Holding Registers (MSHRs) to sustain throughput. By integrating the HPDcache and its "Hit-Under-Multiple-Misses" capabilities into the CVA6, this work actively closes the architectural gap between open-source RISC-V cores and established commercial silicon, providing the foundational non-blocking mechanisms required for deterministic performance.

3.1.12.7 Requirement Compliance

Table 3.33 provides a comparison between the obtained results and the corresponding IP requirements from WP1. The table summarizes the verification outcomes for each requirement and indicates the achieved level of compliance.

Table 3.33: Requirement Compliance Matrix CACHE

REQ-DOC	REQ-ID	REQ-TYPE	REQ-Description	Verification Method	Activities / Justification	Compliance (C / PC / NC)
D1.4	CACHE-01	F-HW_UARCH-PR	The possible performance gain of the use of the CEA high performance cache for CVA6 shall be analysed in a typical application payload scenario.	Observation	Tests developed, but without proof it is a typical application payload scenario (shifted to WP5).	PC
D1.4	Target Performance		As a working hypothesis, we envisioned that using the CEA caches on CVA6 increase the performance by 5%.	Observation	We achieved performance increases larger than 5% for some test cases with larger strides (see Table 3.31).	C

3.2 Monitoring Infrastructure

Task 3.3 of WP3 has focused on the development of components and methodologies that provide monitoring support for multiple purposes, ranging from performance monitoring in a safety-specific context to power and energy monitoring, via on-line hardware-software monitoring infrastructures that enable therefore the optimization of the overall system both at design time and at run time.

A multicore statistics unit (BSC) is integrated as part of the safety infrastructure, while context-aware performance monitoring counters are extended with software context filtering capabilities to further strengthen the monitoring of the safety infrastructure (TRT) and a configurable and programmable co-processor dedicated to monitoring time contracts (OFFIS) can observe application-specific hardware and software timing properties.

Finally, a dedicated methodology can deliver an on-line power monitoring infrastructure (POLIMI) while considering the accuracy, area overhead, and side-channel information leakage metrics as constraints in the power model identification phase.

IP	Lead Partner	Dependencies	Licensing
CA-PMC-IF (3.2.1)	TRT	CA-CORE (WP2), CA-BUS (WP2), CA-PMC (3.1.6)	Permissive open-source
RTPM (3.2.2)	POLIMI	Monitored IP	Proprietary
SafeSU (3.2.3)	BSC	None	Permissive open-source
TCCP (3.2.4)	OFFIS	None	Permissive open-source

Table 3.34: Overview of Task 3.3 contributions

In general, the solutions presented in this chapter extend the monitoring, observability, and runtime analysis capabilities of RISC-V-based systems beyond the current state-of-the-art in both Arm and RISC-V ecosystems. The proposed components either (i) introduce novel capabilities not currently available in existing platforms, (ii) provide broader or more flexible feature sets targeting multi-core and multi-application systems, or (iii) achieve comparable functionality with lower overhead, improved granularity, or open-source availability.

In more detail, each module comparison against state-of-the-art is as follows:

- The Context-Aware Monitoring framework, of which the Context-Aware Performance Monitoring Counter Interface (CA-PMC-IF) is developed in this task, provides features also delivered by Arm alternatives, but with a much simpler unit, and enabling multiple entities to manage monitoring.
- The Run-Time Power Monitoring (RTPM) infrastructure provides significantly finer temporal and spatial granularity than Arm-based solutions, which typically rely on external sensors and coarse-grained software aggregation. By operating at RTL level, RTPM enables low-overhead, design-agnostic deployment on FPGA and ASIC platforms, while also supporting security-aware monitoring by excluding signals correlated with sensitive data.

- The Safety-Related Statistics Unit (SafeSU) compares favorably against Arm alternatives, providing a superset of monitoring capabilities and introducing quota enforcement mechanisms. In addition, SafeSU is open source and platform-agnostic, enabling transparent and flexible deployment across FPGA and ASIC targets. On the RISC-V side, SafeSU complements CA-PMC, with the former specifically targeting per-context monitoring of any type of event while SafeSU focuses on monitoring and quota control of multi-core events and combinations of events.
- The Time Contract Monitoring Co-Processor (TCCP) introduces a novel hardware-based approach to run-time monitoring based on formal contract evaluation, for which no direct equivalent exists in either Arm or RISC-V ecosystems. Compared to Arm alternatives, which focus on trace collection with software-based analysis, TCCP enables non-intrusive, real-time monitoring directly in hardware. Relative to existing RISC-V approaches, it provides deterministic, decentralized, and resource-efficient monitoring of both hardware and software behaviors.

3.2.1 Context-Aware PMC Interface (CA-PMC-IF) – TRT

3.2.1.1 IP Card

Basic Info		
IP name	Context-Aware PMC Interface (CA-PMC-IF)	
License	Open-source (SolderPad Hardware License v0.51)	
Repository	https://github.com/ThalesGroup/cva6-context-aware-monitoring.git	
Architecture		
Clock	Number of clock domains	1
	Synchronous with system	Y
	Clock generated internally	N
Ctrl Interface	ISA extension?	N
	Memory mapped?	Y
	Protocol	AXI4
	Address Map	Y (see Section 3.2.1.5)
Initiator Interface	Protocol	N/A
	Cached?	N
	IOMMU?	N
Interrupts	Interrupts	Y (generates 1 interrupt)
Microarchitecture		
Parametrization	Parametric no. cores?	N
	Parameteric config?	Y
Programmability	Contains programmable cores?	N/A
	N/A	
Software		
Compiler	Requires specialized compiler?	N
	Compiler repository	-
Hardware Abstraction Layer	N/A	
High-level API	Is there a high-level API/SDK?	N
	SDK repository	N
	Is there a domain-specific compiler?	N
Integration		
IP Distribution	Manifest type (if any)	N
	Standalone simulation?	Y
	(if standalone sim) SW requirements?	Verilator
	Integration documented / examples?	Y
Synthesis	Is the IP synthesizable?	Y
	FPGA synthesis example available?	Y (as component in CVA6 based design)
	ASIC synthesis example available?	N
Simulation	Closed-source simulation?	N
	Open-source simulation?	Y (Verilator)
Evaluation	PPA results available?	N

3.2.1.2 General Information

The CA-PMC-IF, short name for Context-Aware Performance Monitoring Counter Interface, IP is part of the Context-Aware Monitoring framework developed by TRT. For an overview of the CA-PMC-IF IP and its function in the Context Aware Monitoring framework, refer to Section 3.1.6.2 on

page 67.

3.2.1.3 Purpose and Scope

The purpose of the CA-PMC-IF IP is to provide a way to program/control instantiations of CA-PMC modules, e.g., the CA-PMC module of the CVA6 [84, 80, 43] RISC-V core, the CA-PMC module of the cache level 2, etc. As such, the CA-PMC-IF is to be instantiated by CA-PMC modules. The CA-PMC-IF module then provides and manages the counters of the CA-PMC module, and it also provides the interfaces to access and configure these counters, leaving the CA-PMC module the task to provide the events that will be counted.

The CA-PMC-IF module currently provides AXI 4 or AXI 4 Lite interfaces; which one is used depends on the targeted design. Other interfaces might be provided/implemented if required. The AXI interface provides an easy and standard way to connect the CA-PMC-IF (and thus the CA-PMC) to an AXI bus so it can be accessed by cores connected to it, e.g., a supervision core. The CA-PMC-IF address map provides means to configure and retrieve the different counters the CA-PMC instantiating it provides.

Finally, the CA-PMC-IF also provides mechanisms to generate and manage interrupts when the counters overflow. A single interrupt output signal is provided. This signal is to be connected to an interrupt controller, e.g., PLIC.

3.2.1.4 Refined architecture description

An initial architecture description of the CA-PMC-IF hardware component was presented in Section 3.3.1 of Deliverable D3.1 [19]. A first refinement of the architecture (CA-PMC-IF version 1) was presented in Deliverable D3.2 [18]. This document presents the latest evolution of the CA-PMC-IF: CA-PMC-IF version 2. As for CA-PMC-IF version 1, the architecture of the CA-PMC-IF version 2 architecture integrates the counter logic as depicted in Figure 3.25, leaving only the event generation to the CA-PMC. This simplifies the creation of new CA-PMC modules, the CA-PMC-IF taking care of the programming interface, the counters management, and the interrupt generation.

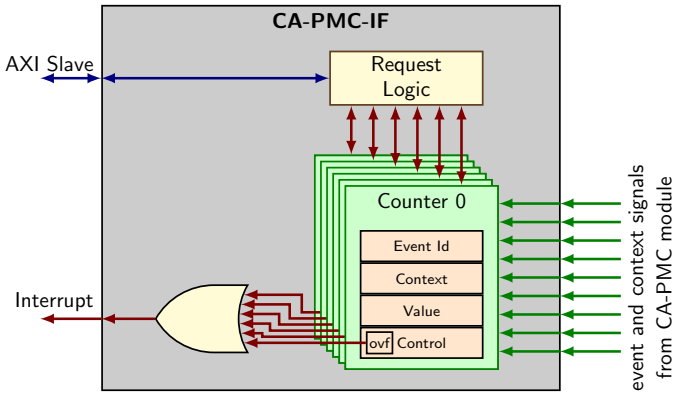


Figure 3.25: CA-PMC-IF architecture

At the core of the CA-PMC-IF are the counters. Each counter contains a register to select the event the counter has to count and the value of the counter, i.e., the number of events that have occurred. Moreover, to control the context information, a register with the current context is stored in the counter. The value field will be only updated when the traced event occurs in the stored counter context. Each counter can generate one interrupt when the value register overflows; this information is stored in a dedicated one-bit field of the counter control register (labelled as "ovf" in Figure 3.25). The control register is used to enable/disable the counter, indicate if the counter should consider or not context information, to enable/disable the generation of interrupts when the value register in the counter overflows, and check if the counter produced the overflow interrupt and clean it.

The number of counters in a CA-PMC-IF instantiation is parameterizable at design time, with a limit of 256 in the current design. The overflow interrupt generated by each of the counters is combined into a single output signal by an OR gate. To feed the counters with events and the context information, the CA-PMC-IF simply forwards the information received through a dedicated interface from the associated CA-PMC module to the counters.

Finally, the CA-PMC-IF provides an AXI interface to programmatically control the counters by writing and reading them. Section 3.2.1.5 describes the address map provided by CA-PMC-IF for that purpose. The CA-PMC-IF processes the AXI requests through a dedicated logic, depicted by the "*Request Logic*" box in Figure 3.25, and reads/writes the counters registers accordingly. This dedicated logic and the associated memory map are where the main difference between version 1 and version 2 of the CA-PMC-IF lies.

3.2.1.5 Memory-Mapped Interface

This section describes the memory-mapped interface of CA-PMC-IF version 2; for a description of the CA-PMC-IF version 1 memory-mapped interface, refer to Deliverable D3.2 [18]. As described in Section 3.2.1.4 the main purpose of the CA-PMC-IF module is to provide a unified counter infrastructure for CA-PMC modules and provide its memory-mapped interface to control and read the CA-PMC counters.

Global CA-PMC-IF memory layout

Table 3.35 presents the memory layout of the CA-PMC-IF memory-mapped interface.

The CA-PMC-IF specific memory regions are decomposed into:

- **capabilities** register: This register provides information on the capabilities provided by the specific instance of the CA-PMC-IF IP, such as the supported context width, the available number of counters and so on.
- **ovfmap** register: This register maps the CA-PMC-IF counters **control** register overflow bit (**counter.control[ovf]**).
- **ctenmap** register: This register maps the CA-PMC-IF counters **control** register counter enable bit **counter.control[cnten]**.
- **ovfenmap** register: This register maps the CA-PMC-IF counters **control** register overflow enable bit **counter.control[ovfen]**.
- **ctxenmap** register: This register maps the CA-PMC-IF counters **control** register context enable bit **counter.control[ctxen]**.

Address	Name	Size	Description
BASE + 0x00	capabilities	16 Bytes	Register describing CA-PMC-IF instance features
BASE + 0x10	*reserved*	16 Bytes	Reserved for future usage
BASE + 0x20	ovfmap	32 Bytes	Register mapping the counters overflow status bit
BASE + 0x40	ctenmap	32 Bytes	Register mapping the counters counter enable bit
BASE + 0x60	ovfenmap	32 Bytes	Register mapping the counters overflow enable bit
BASE + 0x80	ctxenmap	32 Bytes	Register mapping the counters context enable bit
BASE + 0x0a0	*reserved*	96 Bytes	Reserved for future usage
BASE + 0x100	counters	Up to 8192B	Up to 256 memory areas of 32 Byte for each counter

Table 3.35: CA-PMC-IF memory-mapped interface

- **counters**: This memory area holds memory areas for each counter provided by the CA-PMC-IF instance.

More details about each memory section are presented below.

capabilities register layout

The **capabilities** register provides read-only information about the CA-PMC-IF instance. Table 3.36 shows the layout of this register.

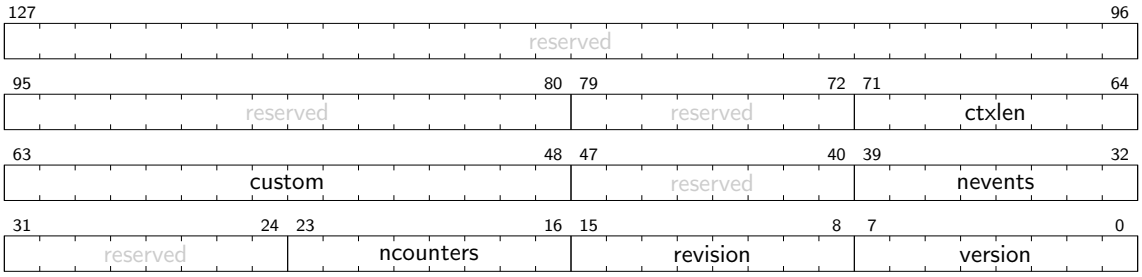


Table 3.36: CA-PMC-IF capabilities memory layout

The whole register is read only, any write is ignored. The register fields are defined as follows:

- **version** (bits 0 to 7): The first byte provides the CA-PMC-IF specification version the CA-PMC-IF instance follows. This document describes version 2 of such specifications, i.e. **version** is equal to 2.
- **revision** (bits 8 to 15): This field provides the CA-PMC-IF specification revision the CA-PMC-IF instance follows. Revisions of the same version of a CA-PMC-IF instance should

be backwards compatible, e.g. a CA-PMC-IF instance with version 1 and revision 2 provides an interface compatible with software written for version 1 revision 1 or 0. This document describes revision 0 of the CA-PMC-IF version 2 specification, i.e. **revision** is equal to 0.

- **ncounters** (bits 16 to 23): This field indicates the number of counters supported by the IP.
- **nevents** (bits 32 to 39): This field indicates the number of events supported by this IP. This field also limits the event to count that can be set on the counters **event** register (**counter.event**), i.e. the **counter.event** register can be set from 0 to **nevents**-1.
- **custom** (bits 48 to 63): Reserved for custom extensions of the CA-PMC-IF interface. Different than zero if used, otherwise all bit should be set to zero.
- **ctxlen** (bits 64 to 71): Width of the software context value supported by the IP. The maximum value this field can have in the current CA-PMC-IF specification is 32, for a 32-bit wide software context.

Unused bits are reserved and should read as 0, and writes on them are ignored.

ovfmap register

This 32-byte, i.e. 256 bits, register maps the overflow bit field of the counters **control** memory region (**counter.control[overflow]**) provided by the IP. Table 3.37 shows its layout. Bit 0 of this register maps counter 0 control register overflow bit (eighth bit of the counter **control** register, **counter.control[overflow]**), bit 1 counter 1 overflow bit, bit 2 counter 2, and so on, up to **capabilities.ncounters**. If **capabilities.ncounters** is smaller than 256, bits **capabilities.ncounters** to 255 of the **overflow** register are mapped to the value 0 and can not be modified.

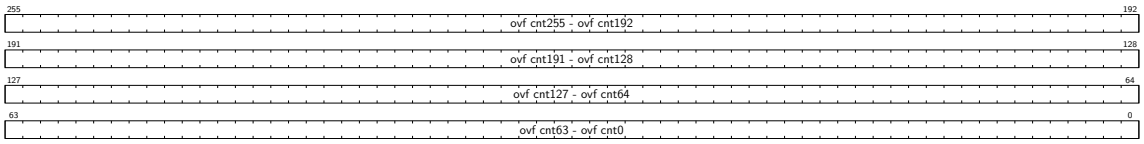


Table 3.37: CA-PMC-IF **ovfmap** register layout

Reading any bit of the **ovfmap** register gets the current value of the corresponding counter overflow bit (**counter.control[overflow]**), or 0 if it is not mapped to any counter. Writing a 0 to any bit of the **ovfmap** register does not modify the corresponding counter overflow bit, and is ignored if the bit is not mapped to any counter. Writing a 1 to any bit of the **ovfmap** register does set the corresponding counter overflow bit to 0, and is ignored if the bit is not mapped to any counter.

cntenmap register

This 32-byte, i.e. 256 bits, register maps the counter enable bit field of the counters **control** memory region (**counter.control[cnten]**) provided by the IP. Table 3.38 shows its layout. Bit 0 of this register maps counter 0 control register counter enable bit (first bit of the counter **control** register, **counter.control[cnten]**), bit 1 counter 1 counter enable bit, bit 2 counter 2, and so on, up to **capabilities.ncounters**. If **capabilities.ncounters** is smaller than 256, bits **capabilities.ncounters** to 255 of the **cntenmap** register are mapped to the value 0 and can not be modified.

Reading any bit of the **cntenmap** register gets the current value of the corresponding counter's counter enable bit (**counter.control[cnten]**), or 0 if it is not mapped to any counter. Writing a 0 to

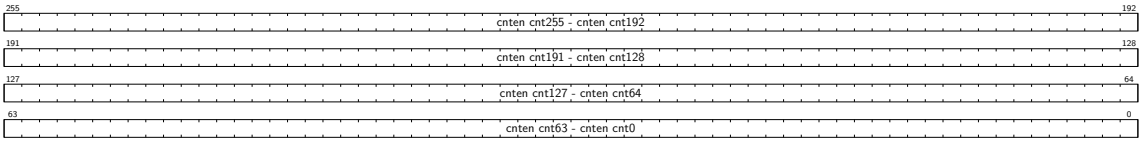


Table 3.38: CA-PMC-IF **cntenmap** register layout

any bit of the **cntenmap** register writes a 0 to the corresponding counter's counter enable bit and disables counting on the corresponding counter if not already disabled. Writing a 1 to any bit of the **cntenmap** register writes a 1 to the corresponding counter's counter enable bit and enables counting on the corresponding counter if not already enabled. Writing to an index not mapped to any counter is ignored.

ovfenmap register

This 32-byte, i.e. 256 bits, register maps the overflow enable bit field of the counters **control** memory region (**counter.control[ovfen]**) provided by the IP. Table 3.39 shows its layout. Bit 0 of this register maps counter 0 control register overflow enable bit (second bit of the counter **control** register, **counter.control[ovfen]**), bit 1 counter 1 overflow enable bit, bit 2 counter 2, and so on, up to **capabilities.ncounters**. If **capabilities.ncounters** is smaller than 256, bits **capabilities.ncounters** to 255 of the **ovfenmap** register are mapped to the value 0 and can not be modified.

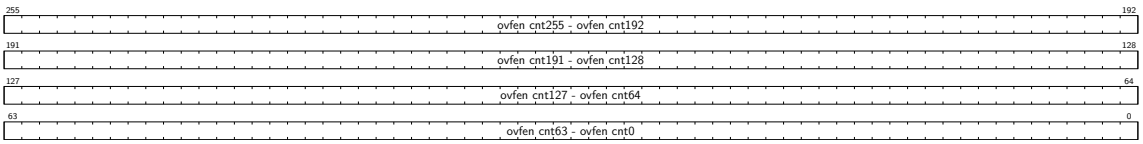


Table 3.39: CA-PMC-IF **ovfenmap** register layout

Reading any bit of the **ovfenmap** register gets the current value of the corresponding counter overflow enable bit (**counter.control[ovfen]**), or 0 if it is not mapped to any counter. Writing a 0 to any bit of the **ovfenmap** register writes a 0 to the corresponding counter overflow enable bit and disables interrupt generation when the corresponding counter overflows. Writing a 1 to any bit of the **ovfenmap** register writes a 1 to the corresponding counter overflow enable bit and enables interrupt generation when the corresponding counter overflows. Writing to an index not mapped to any counter is ignored.

ctxenmap register

This 32-byte, i.e. 256 bits, register maps the context enable bit field of the counters **control** memory region (**counter.control[ctxen]**) provided by the IP. Table 3.40 shows its layout. Bit 0 of this register maps counter 0 control register context enable bit (third bit of the counter **control** register, **counter.control[ctxen]**), bit 1 counter 1 context enable bit, bit 2 counter 2, and so on, up to **capabilities.ncounters**. If **capabilities.ncounters** is smaller than 256, bits **capabilities.ncounters**

to 255 of the **ctxenmap** register are mapped to the value 0 and can not be modified.

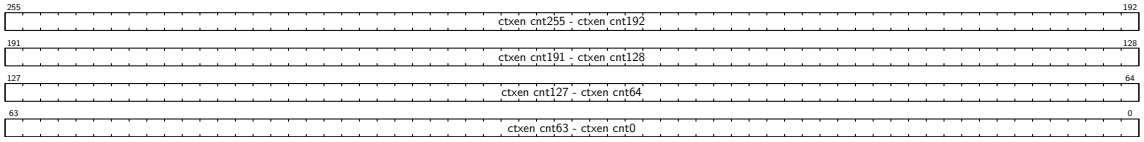


Table 3.40: CA-PMC-IF **ctxenmap** register layout

Reading any bit of the **ctxenmap** register gets the current value of the corresponding counter context enable bit (**counter.control[ctxen]**), or 0 if it is not mapped to any counter. Writing a 0 to any bit of the **ctxenmap** register writes a 0 to the corresponding counter context enable bit and disables context filtering on the corresponding counter, i.e. counts all the counter events. Writing a 1 to any bit of the **ctxenmap** register writes a 1 to the corresponding counter context enable bit and enables context filtering on the corresponding counter based on its context value. Writing to an index not mapped to any counter is ignored.

Counters memory layout

The **counters** memory region provides the registers of each of the performance counters provided by the CA-PMC-IF, as defined by **capabilities.ncounters**. Each performance counter, **counter**, memory area is organized as shown in Table 3.41.

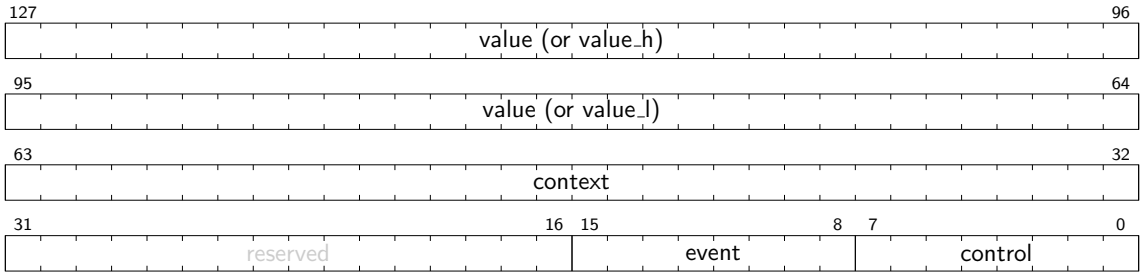


Table 3.41: CA-PMC-IF **counter** memory layout

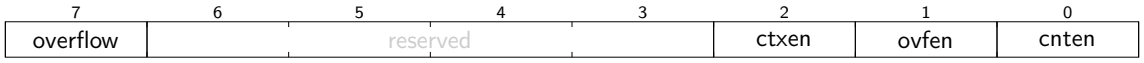


Table 3.42: CA-PMC-IF **counter.control** memory layout

The following are the descriptions of the fields of each counter register:

- **control** register: A R/W register to control performance counter behavior, enabling/disabling counting, overflow or context, and to check and clear the overflow interrupt status. Table 3.42 shows the **counter.control** register layout:

  - **cnten** field (bit 0): Enables (1) or disables (0) event counting.
  - **ovfen** field (bit 1): Enables (1) or disables (0) the generation of overflow interrupt on overflow of the **counter.value** field.
  - **ctxen** field (bit 2): Enables (1) or disables (0) the filtering of events to count based on the context.
  - **overflow** field (bit 7): This a status bit indicating if the counter has produced an overflow interrupt. The counter interrupt signal maps the value in this bit. An overflow interrupt is generated when increasing **counter.value** because of the ocurrence of the monitored event and resulting **counter.value** is smaller than before. This bit remains set to 1 once an interrupt has been generated. To set this bit to 0, i.e. to acknowledge the interrupt, a write request with a value of 1 on this bit needs to be issued. Writing a 0 to this bit does not modify its status.
- **event** register: A R/W 8-bit register specifying the hardware event to be monitored. Writing a value equal or bigger than **capabilities.nevents** sets this field to an implementation defined value smaller than **capabilities.nevents**.
- **context**: A R/W 32-bit register specifying the context value used restrict hardware event counting to specific software contexts.
- **value**: A R/W 64-bit register increased each time the monitored event (**counter.event**) occurs. This register can be accessed with a 64-bit request or with two 32-bit requests to its subparts: **counter.value_h** for the higher 32-bits and **counter.value_l** for the lower 32-bits. Writing this field is not recommended when the counter is enabled (**counter.control.cnten** equals to 1), specially when writing using 32-bit write requests. When using 64-bit read requests the **counter.value** register can be read on a single 64-bit read request. When using 32-bit read requests the **counter.value_h** and **counter.value_l** registers should be read when the counter is disabled (**counter.control.cnten** equals to 0) or if enabled using the following sequence to ensure the correctness of the read 64-bit value:
  1. read **counter.control.value_h**,
  2. read **counter.control.value_l**,
  3. read **counter.control.value_h**,
  4. if value read in step 1 is different than the value in step 3 then repeat the sequence (i.e. go back to step 1), otherwise the read of the performance counter value is finished.

Reserved bits are set to zero, and write accesses are ignored.

3.2.1.6 Hardware interfaces

The CA-PMC-IF provides three interfaces:

1. An AXI or AXI-Lite bus slave interface (slave): This interface allows access to the CA-PMC-IF and the counters it provides programmatically using the memory map described in Section 3.2.1.5. This interface is parameterizable at design time, providing a full AXI interface or a simplified AXI-Lite, and to define the bus characteristics, like address and data width.
2. An output interrupt signal (output): This interface provides a single bit signal indicating that at least one of the CA-PMC-IF counters has produced an overflow interrupt; see Section 3.2.1.4.
3. Monitored module event and context signals (input): This interface allows the monitored CA-PMC module (e.g., a core, a cache module, etc.) to provide to the CA-PMC-IF module the events produced at each cycle and their context. The number of input events the CA-

PMC-IF is parameterizable, and each event signal is one width wide, i.e., the event occurred during a cycle or not. The context signal is currently shared by all the events, i.e., there is a single context signal. The width of the context signal should match the **capabilities.ctxlen** value (see Section 3.2.1.5 *Global CA-PMC-IF memory layout* on page 128).

3.2.1.7 ISA specialization

The CA-PMC-IF IP does not require any modification of the ISA. However, as described in Section 3.1.6.6, to exploit the context monitoring capability, IPs generating events should be capable to pass the context information (through the context signal presented in Section 3.2.1.6). Generators of this context information might need ISA modifications to generate it, as proposed by the CA-CORE extension developed in Task T2.3.

3.2.1.8 Evaluation prototype

FPGA

The CA-PMC-IF version 2 IP has been evaluated in a dedicated Verilog testbench to verify its operation. Furthermore, it has been integrated into a CVA6 CA-PMC module to evaluate its integration, synthesis, and operation. For further information on this integration, see Section 3.1.6.7 (page 71).

3.2.1.9 State-of-the-Art Comparison

Refer to Section 3.1.6.8 for a state-of-the-art comparison of the CA-PMC and CA-PMC-IF modules.

3.2.1.10 Requirement Compliance

Table 3.18 from Section 3.1.6.9 provides a comparison between the obtained results and the corresponding IP requirements from WP1 for both the CA-PMC and the CA-PMC-IF hardware modules.

3.2.2 Run-Time Power Monitoring Instrumentation (RTPM) – POLIMI

3.2.2.1 IP Card

Basic Info		
IP name	Run-Time Power Monitoring Instrumentation	
License	Closed-source proprietary	
Repository		
Architecture		
Clock	Number of clock domains	1
	Synchronous with system	Y
	Clock generated internally	N
Ctrl Interface	ISA extension?	N
	Memory mapped?	Y
	Protocol	AXI
	Address Map	N.A.
Initiator Interface	Protocol	AXI
	Cached?	N
	IOMMU?	N
Interrupts	Interrupts	No
Microarchitecture		
Parametrization	Parametric no. cores?	N
	Parameteric config?	Y
Programmability	Contains programmable cores?	N
	ISA	None
Software		
Compiler	Requires specialized compiler?	N
	Compiler repository	N.A.
Hardware Abstraction Layer	N/A	
High-level API	Is there a high-level API/SDK?	N
	SDK repository	N.A.
	Is there a domain-specific compiler?	N
Integration		
IP Distribution	Manifest type (if any)	N.A.
	Standalone simulation?	Y
	(if standalone sim) SW requirements?	AMD Vivado
	Integration documented / examples?	N.A.
Synthesis	Is the IP synthesizable?	Y
	FPGA synthesis example available?	Y
	ASIC synthesis example available?	N
Simulation	Closed-source simulation?	Y (AMD Vivado)
	Open-source simulation?	N
Evaluation	PPA results available?	Y

3.2.2.2 General Information

The automatic generation of the run-time power monitoring infrastructure delivers the capability to provide periodic power estimates from the switching activity of a set of signals in the monitored components.

3.2.2.3 Purpose and Scope

The effectiveness of run-time optimization techniques that aim to improve the energy efficiency of a target computing platform is strongly tied to the quality of the measurements or estimates of power consumption provided by a run-time power monitoring infrastructure. The latter can perform indirect estimation of the dynamic power consumption of the target computing platform by analyzing its run-time statistics, such as the switching activity of microarchitectural signals, monitored through dedicated hardware counters.

The identification of the power model and the instrumentation of the corresponding power monitoring infrastructure can optionally take into account side-channel leakage, avoiding using, as inputs to the model, the signals whose switching activity correlates with the secret key or the plaintexts and ciphertexts being processed by the target computing platform.

With respect to Deliverable D3.2, in this deliverable we expand on the experimental evaluation and on the comparison against the state of the art. Moreover, we expand the proposed methodology to take into account the avoidance of side-channel leakage by the instrumented run-time power monitoring infrastructure.

3.2.2.4 Refined Architecture Description

The monitoring infrastructure consists of a set of switching activity counters attached to corresponding inputs and output signals of the system components of which it is required to monitor dynamic power consumption. The power estimate value obtained by aggregating the counter values is exposed through a hardware register.

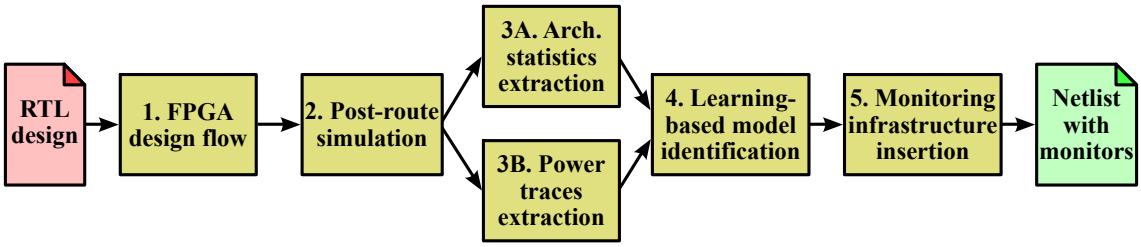


Figure 3.26: Flowchart diagram of the flow for run-time power monitoring.

The automatic instrumentation of a run-time power monitoring infrastructure is carried out by taking the RTL description of the target computing platform and its corresponding testbench as inputs and producing a gate-level netlist enhanced with power monitoring capabilities. This process, depicted in Figure 3.26, can be divided into five sequential phases, each contributing to the final implementation of the monitoring infrastructure.

1. **Target implementation** – The first phase utilizes the standard hardware design flow to process the RTL description of the target platform. This includes synthesis, placement, and routing steps, resulting in a gate-level netlist of the design. This netlist serves as the foundation for all subsequent stages of the framework.
2. **Design simulation** – In the second phase, the post-route gate-level netlist is subjected to a detailed simulation that exercises all parts of the design using a comprehensive input dataset. The goal is to stress the entire computing platform to ensure thorough coverage of its functionality. This phase outputs a value change dump (VCD) file, which contains detailed records of the switching activity of the design, enabling the calculation of power consumption traces. However, the time required for simulation grows significantly with the complexity of the target platform.
3. **Data extraction** – The third phase processes the VCD file to extract relevant information about the switching activity and to compute the power consumption traces associated with the target design. This step is computationally intensive, particularly for large platforms or when higher temporal resolution is required for the power traces. The resulting power traces and switching activity data form the basis for model identification in the next phase.
4. **Model identification** – The fourth phase involves creating a power model tailored for run-time monitoring. This model identifies a subset of signals from the target platform and assigns them corresponding weights by analyzing the correlation between the extracted switching activity and the computed power traces. The objective is to minimize the difference between the actual power trace and the one generated by the identified power model while also satisfying constraints such as limiting the number of signals used as model inputs, minimizing the area and power overheads of the corresponding monitors [30], and avoiding side-channel leakage [85].
5. **Monitor implementation** – In the fifth and final phase, the framework automatically instruments the identified power model into the original design. The signals selected by the model are wrapped in additional hardware that periodically samples their switching activity and calculates the corresponding power consumption. This calculated power consumption is made accessible through an architectural register, enabling real-time power monitoring without significant interference with the platform's primary functionality.

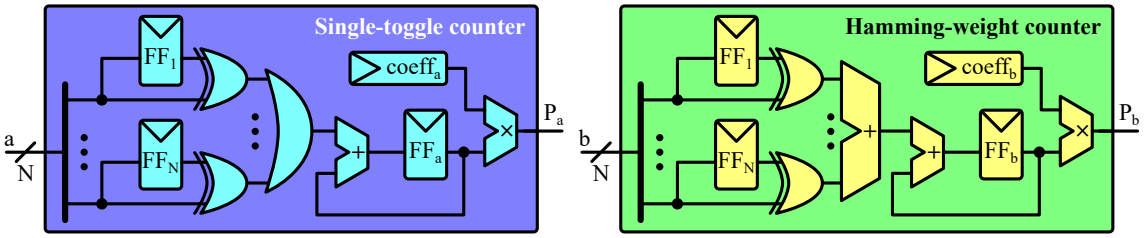


Figure 3.27: Architecture of single-toggle and Hamming-weight counters measuring the switching activity of a multi-bit signal.

The architecture of the toggling activity counters can be implemented in two different ways, depending on whether they measure single-toggle or Hamming-weight counts, as shown in Figure 3.27.

A single-toggle counter counts whether there is at least one change in the signal on the monitored physical wire. It takes the multi-bit input signal and combines a XOR-tree network with an OR gate to output 1 or 0 depending on whether at least one bit of the signal toggled, and the produced value is added to the cumulative switching activity.

Conversely, a Hamming-weight counter counts instead the number of bits in the signal that switched their values. It takes the multi-bit input signal and combines a XOR-tree network with an adder to output the number of switching bits, adding the latter value to the cumulative switching activity.

Concerning area overhead, the number of flip-flops for Hamming-weight counters increases with the size of the monitored signal and the number of clock cycles in the time window, while in the case of single-toggle counters it depends only on the time window, since for each clock cycle they perform at most a unitary increment. Hamming-weight counters also show a higher power overhead due to their higher switching activity.

The run-time monitoring infrastructure instantiated by the automatic generation flow may contain both single-toggle and Hamming-weight counters. Regardless of the type of such counters, their switching activity values are multiplied by their respective model weights and aggregated to obtain the estimated power consumption at each time interval.

In addition, the model identification can also take into account side-channel leakage by filtering out of the candidate power model inputs the signals that are vulnerable, i.e., whose switching activity is a function of the secret key, plaintext, or ciphertext. Such RTL signals are filtered out by checking the correlation between their switching activity and the secret key, plaintext, or ciphertext of the cryptosystem under execution by means of Welch's t-test.

3.2.2.5 Memory-Mapped Interface

An AXI memory-mapped register provides, at each time interval, the estimate of power consumption computed by the run-time power monitoring infrastructure. This register holds the estimated power consumption value provided by the run-time power monitoring infrastructure at each time interval. The estimate is periodically updated by the monitoring infrastructure, according to the selected time resolution. The register is read-only and can be accessed by the rest of the system through a read operation at the corresponding memory-mapped address.

3.2.2.6 ISA Specialization

No ISA specialization is required, as the IP can be accessed via a standard AXI memory-mapped read operation.

3.2.2.7 Evaluation Prototype

We validated the RTPM methodology by applying it to a set of SystemVerilog-described RTL designs, both HLS-generated and human-designed, evaluating the effectiveness of the automatic instrumentation of a run-time power monitoring according to a variety of quality metrics. The quality of the instantiated monitoring infrastructures is measured through their area and power overheads as well as the root-mean-square error (RMSE) of their estimations of run-time power consumption.

Vivado 2022.1 is employed for the synthesis, implementation, simulation, collection of switching activity, and estimation of power consumption of the target computing architectures.

In the following, we first characterize the target designs and then analyze the accuracy and overheads achieved by the obtained monitoring infrastructure, also when applying the leakage-resistant protection.

Table 3.43: Resource utilization of designs targeted within the experimental evaluation. Refers to post-implementation netlists on a Virtex UltraScale+ VCU118 Evaluation Platform (`xcvu9p-flga2104-2L-e`) FPGA. Clock frequency expressed in MHz.

Source	Design	Clock	LUT	FF	BRAM	DSP
CHStone	adpcm	100	10393	7090	16	137
	aes	100	4961	4425	8	0
	blowfish	100	3748	6063	9	0
	dfadd	100	4706	1281	0	0
	dfdiv	100	3723	3422	0	16
	dfmul	100	2221	1234	0	16
	gsm	100	3145	2784	3	45
	mips	100	1330	362	1	8
	motion	100	4574	2853	5	0
ISOLDE	ACC-BIKE	125	274290	147977	408	94
	FPMIX	256	2194	573	0	10

FPGA

The target HLS-generated and SystemVerilog RTL accelerators are implemented on the VCU118 FPGA with the clock frequency and resource utilization listed in Table 3.43. The HLS-generated designs include nine different accelerators from the CHStone [42] benchmark suite for C-based HLS. The AMD Vivado toolchain was employed for HLS, RTL synthesis and implementation, bitstream generation, FPGA programming, and simulation. The other targets of the experimental campaign include two designs developed within the ISOLDE project, namely, the FPMIX floating-point unit, described in Section 3.1.1 of Deliverable D3.5, and the ACC-BIKE accelerator for the BIKE post-quantum cryptosystem, described in Section 3.3.1 of D3.5. The FPMIX component is human-designed and written in SystemVerilog, while ACC-BIKE is HLS-generated.

The designs operate at clock frequencies ranging from 100 MHz to 256 MHz and consume between few mW up to 3.4W on average for the ACC-BIKE accelerator.

The identification of the power model and the evaluation of power estimates obtained by the monitoring infrastructure are carried out by splitting the collected dataset of power traces and microachitectural statistics according to a 80:20 ratio.

The first-order linear model takes as its inputs the switching activity of a selected subset of input and output signals of the modules in the design hierarchy and outputs the estimated power consumption. The hardware counters that monitor the selected signals are either single-toggle ones, counting any change in the target multi-bit signal, or Hamming-weight counters, that count the number of bits that toggled their values.

In our experiments, without loss of generality, we targeted the generation of run-time monitoring

Table 3.44: Temporal resolution, estimation error, and overheads of the monitoring infrastructure applied to the designs considered within the experimental evaluation. Refers to post-implementation netlists on a Virtex UltraScale+ VCU118 Evaluation Platform (`xcvu9p-flga2104-2L-e`) FPGA. Clock frequency expressed in MHz. Legend: **ST** single-toggle counters, **HW** Hamming-weight counters, **Temp. Res.** temporal resolution (in microseconds), **RMSE** root mean-square error (expressed as a percentage).

					Overheads			
Design	**ST**	**HW**	**Temp. Res. (us)**	**RMSE (%)**	**LUT**	**FF**	**DSP**	**Power (W)**
adpcm	1	0	0.250	1.92	91	49	3	0.002
aes	7	8	0.003	4.32	2170	610	15	0.035
blowfish	3	2	2.500	2.16	585	200	6	0.01
dfadd	1	5	0.250	3.88	834	445	9	0.015
dfdiv	0	1	2.500	1.37	184	235	0	0.004
dfmul	1	3	0.250	3.44	409	349	6	0.006
gsm	4	5	0.030	4.86	1306	383	6	0.018
mips	0	1	0.025	3.38	92	43	3	0.002
motion	0	1	0.050	1.33	239	43	0	0.003
ACC-BIKE	11	4	125	4.16	1752	410	6	0.033
FPMIX	3	3	256	2.64	1737	819	0	0.056

infrastructures with an estimation error below 5% and with different temporal resolutions, on par with the run-time power monitoring solutions from the open literature [86]. Table 3.44 lists, for the various HLS and RTL designs, the achieved estimation error and corresponding overheads, together with the target temporal resolution and the number of instantiated counters split between the single-toggle and Hamming-weight ones.

We also evaluated the capability of avoiding leakage from estimated power consumption, by filtering signals related to keys, plaintexts and ciphertexts from those fed as inputs to the power model. In particular, we applied such approach to the AES and Blowfish HLS-generated accelerators from the CHStone suite and the ACC-BIKE developed within ISOLDE.

Table 3.45: Temporal resolution, estimation error, and overheads of the SCA-protected monitoring infrastructures. Refers to post-implementation netlists on a Virtex UltraScale+ VCU118 Evaluation Platform (`xcvu9p-flga2104-2L-e`) FPGA. Clock frequency expressed in MHz. Legend: **ST** single-toggle counters, **HW** Hamming-weight counters, **Temp. Res.** temporal resolution (in microseconds), **RMSE** root mean-square error (expressed as a percentage).

					Overheads			
Design	**ST**	**HW**	**Temp. Res. (us)**	**RMSE (%)**	**LUT**	**FF**	**DSP**	**Power (W)**
aes	8	7	0.003	4.91	1910	605	21	0.03
blowfish	4	0	2.500	2.56	491	152	3	0.007
ACC-BIKE	16	14	125	4.97	8941	2904	6	0.106

As shown in Table 3.45, the RTPM methodology is still able to deliver an estimation error below the target 5% threshold at the same temporal resolution as the non-protected power monitors. The RMSE increases, respectively, by 0.59%, 0.40%, and 0.81%, still achieving the 5% target, by

changing the monitored signals, as can be seen by the number of single-toggle and Hamming-weight having changed from Table 3.44.

3.2.2.8 State-of-the-Art Comparison

Arm

Commercial Arm-based platforms typically expose power information through board- or SoC-level sensors rather than synthesized on-chip monitors. For instance, the Arm Energy Probe (AEP), a widely used accessory integrated in the DS-5/Streamline toolchain, samples up to 10 kSa/s (i.e., a 100 µs nominal time resolution) over three external rails with a 10-bit ADC and a specified measurement error of $\pm 5\%$ for shunt voltages above 20 mV [4]. Similarly, many Arm development boards (e.g., those based on Samsung Exynos SoCs) integrate current and power monitors such as the TI INA231 on the supply rails of the big and little CPU clusters, GPU and DRAM. At the sensor level, the INA231 supports per-sample conversion times down to 140 µs (and up to 8.244 ms) with a 16-bit ADC and a gain error below 0.5% across the operating temperature range [78]. However, the corresponding Linux drivers typically expose these readings at much coarser granularity: for example, on an Odroid-XU3 board the I^2C driver is configured with an update period of 200 ms, and power values are further averaged to obtain one sample per second for each rail [41].

Software profilers on Arm SoCs, such as ALEA, combine these on-board sensors with statistical sampling of the program counter to attribute energy at basic-block granularity. On an Exynos-based ODROID-XU+E platform, ALEA operates with a 10 ms sampling period and achieves mean energy-estimation errors between 1.9% (whole-program and coarse basic blocks) and 3.5% (fine-grain basic blocks) when validated against direct measurements [53]. In contrast, the proposed RTPM infrastructure is instantiated directly in the RTL of the target design and can therefore provide power estimates with significantly finer temporal resolution (down to sub-microsecond windows) and module-level spatial granularity, while maintaining an RMSE below 5% and incurring limited area and power overheads. Moreover, our flow is applicable to arbitrary SystemVerilog/HLS designs targeting FPGAs or ASICs, and can selectively exclude signals that correlate with secret data, unlike vendor-specific power monitors that are tightly coupled to a given Arm SoC and do not address side-channel leakage.

3.2.2.9 Requirement Compliance

Table 3.46 provides a comparison between the obtained results and the corresponding IP requirements from WP1. The table summarizes the verification outcomes for each requirement and indicates the achieved level of compliance.

Table 3.46: Requirement Compliance Matrix RTPM

REQ-DOC	REQ-ID	REQ-TYPE	REQ-Description	Verification Method	Activities / Justification	Compliance (C / PC / NC)
D1.4	RTPM-01	F-HW_UARCH-S	Instrumented monitoring infrastructure satisfies defined area overhead constraint	Synthesis and implementation on FPGA target	Verified in synthesis and implementation	C
D1.4	RTPM-02	F-HW_UARCH-PR	Instrumented monitoring infrastructure satisfies accuracy constraint	Simulation	Verified in simulation	C
D1.4	RTPM-03	F-HW_UARCH-FEA	Instrumented monitoring infrastructure does not enable side-channel attacks	Simulation	Verified in simulation	C
D1.4	RTPM-04	NF-HW_UARCH-PR	Instrumented monitoring infrastructure delivers power estimation at temporal resolution not higher than 1ms	Simulation, system integration	Verified in simulation and system integration	C
D1.4	Target Performance		Frequency of synth. and impl. on target FPGA $\geq$ 50MHz	Synthesis and implementation reports	Verified in synthesis and implementation	C
D1.4	Target Performance		Temporal resolution of power estimation $\leq$ 1ms	Simulation, system integration	Verified in simulation and system integration	C

3.2.3 Safety-Related Statistics Unit (SafeSU) – BSC

3.2.3.1 IP Card

Basic Info		
IP name	SafeSU	
License	Open-source (MIT License)	
Repository	https://github.com/bsc-loca/SafeSU - rev. 9b7f520	
Architecture		
Clock	Number of clock domains Synchronous with system Clock generated internally	1 Y N
Ctrl Interface	ISA extension? Memory mapped? Protocol Address Map	N Y APB 32b Base (0x80100000)
Initiator Interface	Protocol Cached? IOMMU?	APB 32b N N
Interrupts	Interrupts	Y
Microarchitecture		
Parametrization	Parametric no. units? Parameteric config?	N Y
Programmability	Contains programmable cores? ISA	N -
Software		
Compiler	Requires specialized compiler? Compiler repository	N -
Hardware Abstraction Layer	N/A	
High-level API	Is there a high-level API/SDK? SDK repository Is there a domain-specific compiler?	Y github.com/bsc-loca/SafeSU/tree/main/drivers N
Integration		
IP Distribution	Manifest type (if any) Standalone simulation? (if standalone sim) SW requirements? Integration documented / examples?	Y (Readme.md) Y QuestaSim, Vivado 2020.2, Verilator -
Synthesis	Is the IP synthesizable? FPGA synthesis scripts/example available? ASIC synthesis scripts/example available?	Y Y N
Simulation	Closed-source simulation? Open-source simulation?	Y (QuestaSim) Y
Evaluation	PPA results available?	DOI 10.1109/ETS50041.2021.9465444

3.2.3.2 General Information

The SafeSU is a modular and scalable Performance Monitor Unit (PMU) that can be connected to any on-chip interconnect and allows multicore interference observability and controllability.

3.2.3.3 Purpose and Scope

The SafeSU builds on a number of components, namely, the Contention-Cycle Stack (CCS), the Request Duration Counter (RDC) and the Maximum-Contention Control Unit (MCCU).

- The CCS offers observability features by providing multicore time-interference breakdown.
- The RDC provides end users with an observability channel to monitor high-watermark latencies per event and core, as needed for interference bounding (e.g., during worst-case execution time estimation).
- The MCCU offers controllability capabilities with interference quota monitoring and enforcement, alerting the user when allocated quotas are exceeded.

3.2.3.4 Place in the System

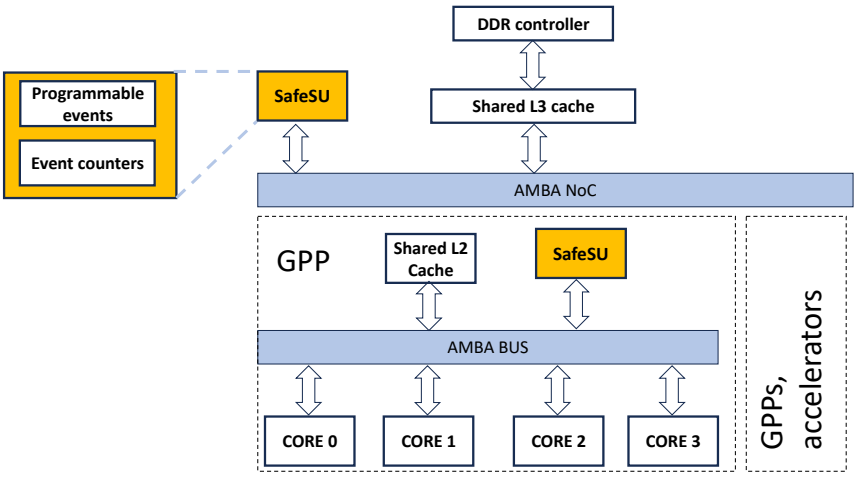


Figure 3.28: SafeSU system integration

The monitoring interface of the SafeSU depicted in Figure 3.28, is AMBA AHB and AXI compliant. The SafeSU is intended to be connected to those types of interfaces, and it is particularly useful if those interfaces have either multiple managers or are connected to subordinates receiving requests from multiple managers. For instance, its best location is normally connected to the interface used by the cores and/or accelerators to access shared caches or memory controllers so that ongoing traffic can be monitored and eventually compared to predefined quotas to ensure that no manager abuses the use of relevant shared resources.

SafeSU's programming port is compliant with AMBA APB and has been recently extended to also support AMBA AXI.

3.2.3.5 Architecture

The main components of the SafeSU, which did not need any update since D3.2, are described next and can be seen in the block diagram of Figure 3.29:

- Self-test: configures the counters' inputs to a fixed value, bypassing the crossbar and ignoring the SoC inputs. This mode allows for tests of the software and the unit under known conditions.
- Crossbar: routes any input event, or combination of events thereof, to any counter.
- Counters: A group of simple counters with settable initial values and a general control register.
- Overflow: Detects counters' overflow. It can raise interrupts upon overflow with its dedicated interruption vector and per-counter interrupt enable.
- MCCU (Maximum Contention Control Unit): Contention control measures for each core for the particular event type (or combination of events) that has been programmed to be monitored. It can raise an interrupt if a contention threshold is exceeded. It accepts real contention signals or estimation through weights.
- RDC (Request Duration Counters): Provides measures of the pulse length of a given input signal (watermark). It can be used to determine maximum latency and cycles of uninterrupted contentions. Each of the counters can trigger an interrupt at a user-defined threshold.

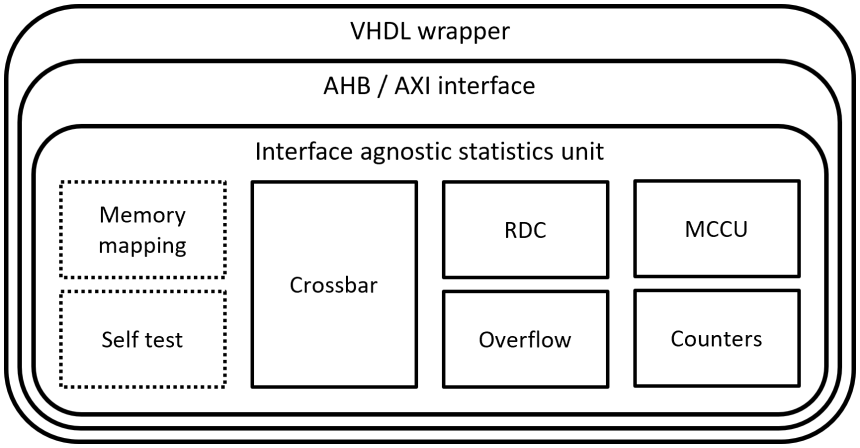


Figure 3.29: SafeSU architecture block diagram

3.2.3.6 Interfaces

AMBA AHB/AXI interface:

The AHB or AXI interface is a subordinate interface used to snoop traffic. It is fully compliant with the specification of the corresponding protocol. Note that, in general, a SafeSU instance supports only one of those interfaces.

AMBA APB interface

The AMBA APB subordinate interface is used to program the control registers of the SafeSU. The control registers are as follows:

Main configuration and self-test

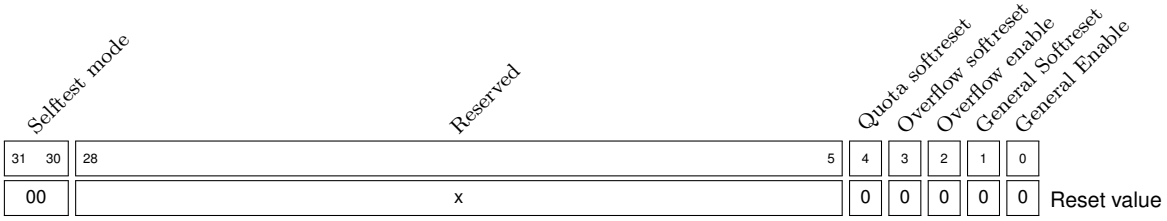


Figure 3.30: SafeSU - Base Configuration Register (0x000)

Reset and enable overflow, quota, and regular counters' operations can be performed with the Base Configuration Register shown in Figure 3.30. All signals are active high.

Self-test mode allows bypassing the input events from the crossbar and instead using a specific input pattern where signals are constant. This mode can be used for debugging. After the addition of the crossbar and debug inputs, there is a certain overlap. The same results can be achieved with the correct crossbar configuration. Nevertheless, it has been included in this release for compatibility.

These are the self-test modes for each configuration value of the field Selftest mode part of the register shown in Figure 3.30:

- 0b00: Events depend on the crossbar. Self-test is disabled.
- 0b01: All signals are set to 1.
- 0b10: All signals are set to 0.
- 0b11: Signal 0 is set to 1. The remaining signals are set to 0.

Crossbar

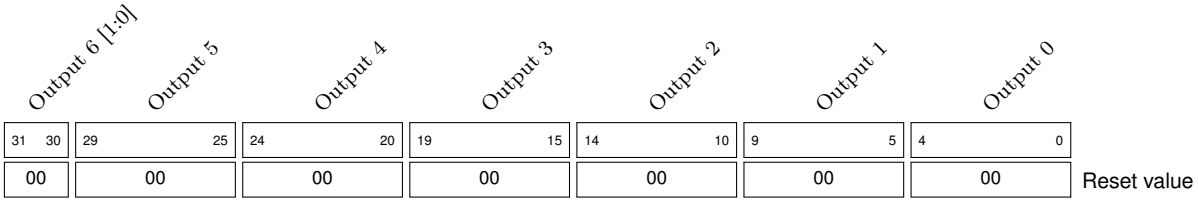


Figure 3.31: SafeSU - Crossbar Configuration Register 0 (0x0AC)

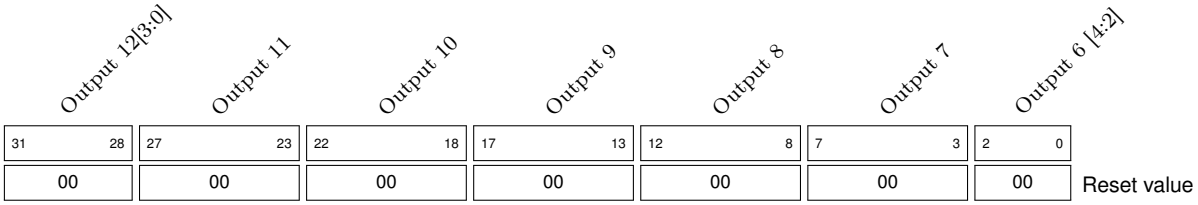


Figure 3.32: SafeSU - Crossbar Configuration Register 1 (0x0B0)

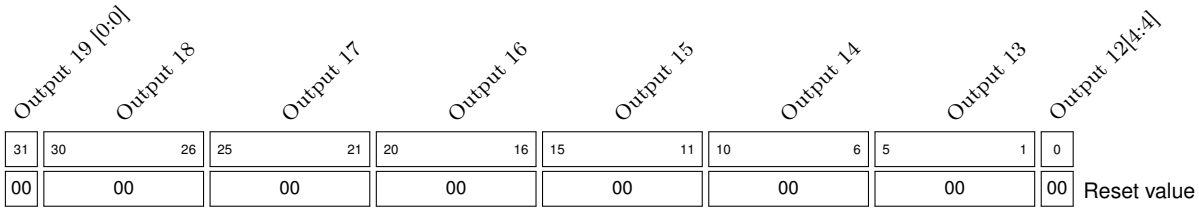


Figure 3.33: SafeSU - Crossbar Configuration Register 2 (0x0B4)

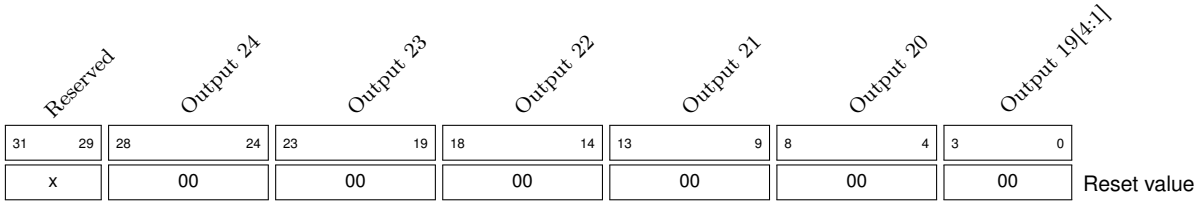


Figure 3.34: SafeSU - Crossbar Configuration Register 3 (0x0B8)

Output	Counters	Overflow	MCCU	RDC
0	Yes	Yes	Core 0	Yes
1	Yes	Yes	Core 0	Yes
2	Yes	Yes	Core 1	Yes
3	Yes	Yes	Core 1	Yes
4	Yes	Yes	Core 2	Yes
5	Yes	Yes	Core 2	Yes
6	Yes	Yes	Core 3	Yes
7	Yes	Yes	Core 3	Yes
8	Yes	Yes	No	No
9	Yes	Yes	No	No
10	Yes	Yes	No	No
11	Yes	Yes	No	No
12	Yes	Yes	No	No
13	Yes	Yes	No	No
14	Yes	Yes	No	No
15	Yes	Yes	No	No
16	Yes	Yes	No	No
17	Yes	Yes	No	No
18	Yes	Yes	No	No
19	Yes	Yes	No	No
20	Yes	Yes	No	No
21	Yes	Yes	No	No
22	Yes	Yes	No	No
23	Yes	Yes	No	No

Table 3.47: SafeSU - Crossbar outputs and SafeSU capabilities

This feature allows routing any of the input signals of the SafeSU into any of the 24 counters of the SafeSU (see Table 3.47). Each one of the counters has a 5-bit configuration value. These values are stored in the registers shown in Figures 3.31, 3.32, 3.33 and 3.34. All the configuration values are consecutive. Thus, some values may have configuration bits in two consecutive memory addresses. Examples of this are outputs 6, 12, and 19 in our current configuration. As a consequence, the previous outputs may require two writes to configure the desired input signal.

Configuration fields match one-to-one with the internal counters. So, the field Output 0 matches with counter 0, Output 1 with counter 1, etc.

As a usage example, suppose the user wants to route the signal pmu_events(0).icnt(0) to the internal counter 0. The field Output 0 of the register in Figure 3.31 shall match the index of the signal in the table of inputs. In this case, the index is 2. After this configuration, the event count will be recorded in counter 0. The addresses for counter values range between 0x04 and 0x60.

Overflow

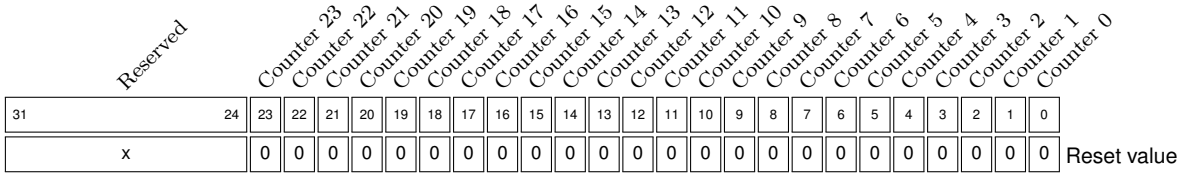


Figure 3.35: SafeSU - Overflow Interrupt Enable Mask (0x064)

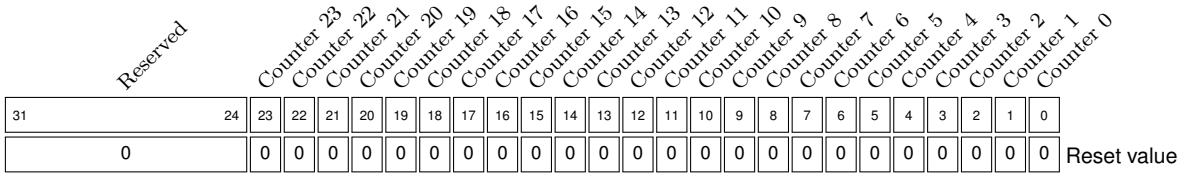


Figure 3.36: SafeSU - Overflow Interrupt Vector (0x068)

The user can enable overflow detection for each of the counters in the SafeSU. Enables are active high and individual for each counter, as indicated in the Overflow Interrupt Enable Mask register depicted in Figure 3.35. If a counter with overflow detection active wraps over the maximum value, the corresponding bit of the Overflow Interrupt Vector register depicted in Figure 3.36 will become 1, and AHB interrupt number 6 will become active. The default AHB interrupt mapping can be modified within the file ahb_wrapper.vhd.

MCCU

The MCCU allows monitoring for a subset of the input events and tracking the approximate contention that they will cause. Currently, events assigned to counters 0 to 7 can be used as inputs of the MCCU. Thanks to the crossbar, any of the 32 SoC signals can be used by the MCCU. Figure 3.37 shows the internal elements required to monitor the quota consumption of one core, given that there are four input events. When the events become active, they pass the value assigned in the weight register depicted in Figure 3.38 for the given signal to a series of adders.

The addition is subtracted from the corresponding quota register, mapped to addresses 0x088 to 0x094. If the remaining quota is smaller than the cycle contention, an interrupt is triggered.

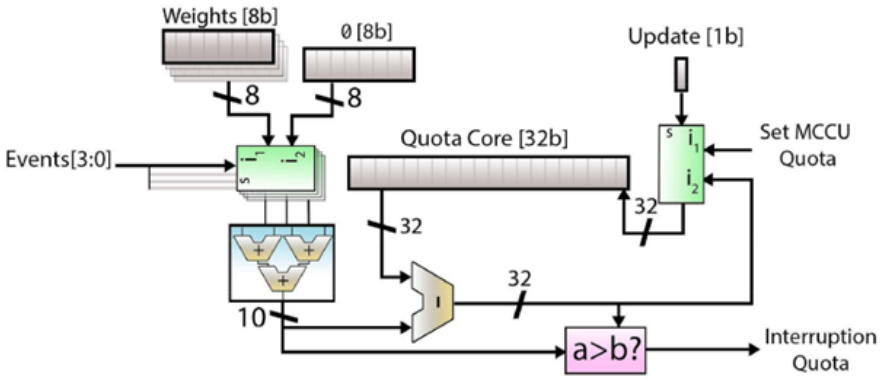
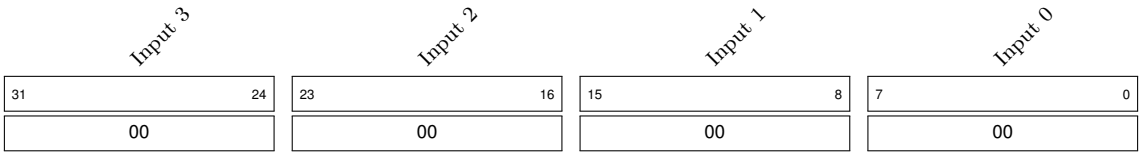


Figure 3.37: SafeSU - Block diagram of the MCCU mechanism for one core



Reset value

Figure 3.38: SafeSU - MCCU Event Weights Register 0 (shared with RDC; 0x098)

In the current release, the MCCU can be reset and activated with the respective fields of the MCCU Main Configuration register depicted in Figure 3.39. The fields labelled as Update Quota Core x are used to update the available quota of each core (addresses 0x088 to 0x094). While Update Quota Core x is high, the content of the corresponding quota register (addresses 0x088 to 0x094) is assigned to the available quota, as configured in registers 0x078 to 0x084. Once released (low), the available quota can start to decrease if the MCCU is active. The current quota can be read while the unit is active.

In the current release, each core can monitor two input events. The MCCU module is parametric. More events can be provided in future releases. Table 3.47 listing the outputs shows the available features for each crossbar output. Under the column MCCU, you can see towards which core quota the event will be computed. The unit provides one interrupt for each of the monitored cores. Quota exhaustion for cores 3, 2, 1, and 0 is mapped to AHB interrupts 10, 9, 8, and 7, respectively.

Weights for each monitored event are registered in the MCCU Event Weights Register x registers depicted in Figures 3.38 and 3.40. Currently, each weight is an 8-bit field. Each input of the MCCU maps directly to the outputs of the crossbar. Thus, the weight for the MCCU input 0 corresponds to the signal in crossbar output 0.

RDC

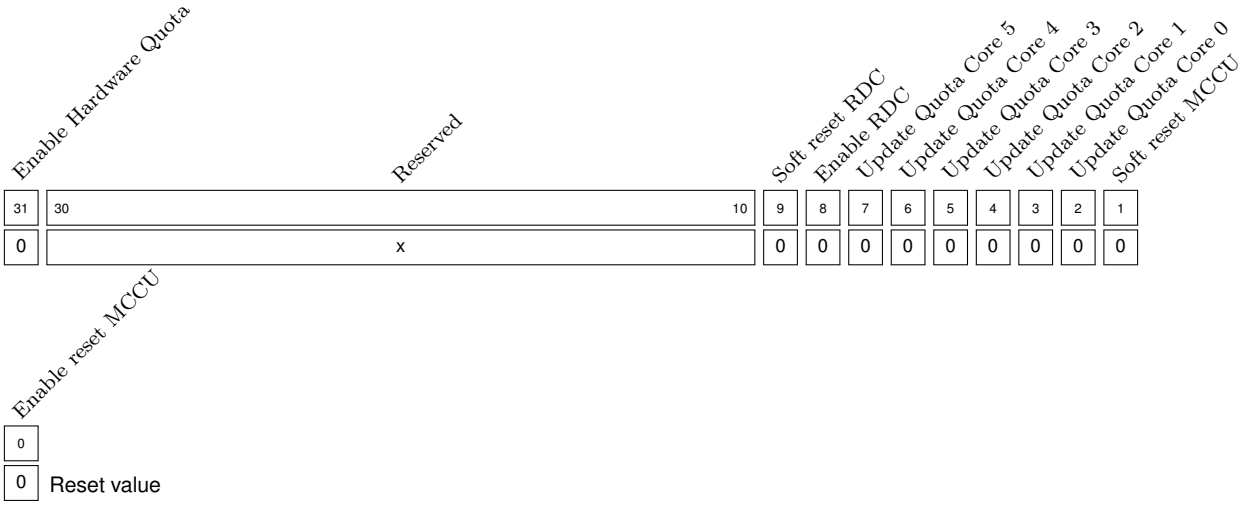


Figure 3.39: SafeSU - MCCU Main Configuration (0x074)

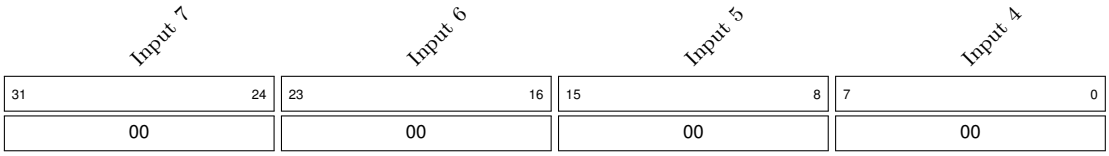


Figure 3.40: SafeSU - MCCU Event Weights Register 1 (shared with RDC; 0x09c)

The Request Duration Counter or RDC depicted in Figure 3.41 is comprised of a set of 8-bit counters and comparators that allow monitoring the length of a CCS signal, recording the number of clock cycles of the longest pulse and comparing this number with the defined weight.

The current release provides monitoring for crossbar outputs 0 to 7. The weights for each signal are shared with the MCCU and are stored in the RDC Event Weights Register x registers depicted in Figure 3.43. Weights are 8-bit fields. Counters have overflow protection, preventing the count from wrapping over the maximum value. The maximum value for each event (watermarks), is stored in the RDC Watermark Register x registers depicted in Figure 3.44. The RDC shares the main configuration register with the MCCU (Figure 3.39). Through this register, the unit can be reset and enabled through the corresponding fields. Such fields are active high signals. The unit does provide access to the internal interrupt vector (Figure 3.42), but such information is redundant and may be removed in future releases. Given the current watermarks and assigned weights, the events responsible for the interrupt can be identified. The RDC interrupt has been routed to AHB interrupt 11.

Software interface

The control registers of the SafeSU, as well as the counters by the SafeSU monitoring the events must be accessed (modified and/or read) only by software components with appropriate privileges. To realize this, the SafeSU registers are mapped to specific physical addresses upon

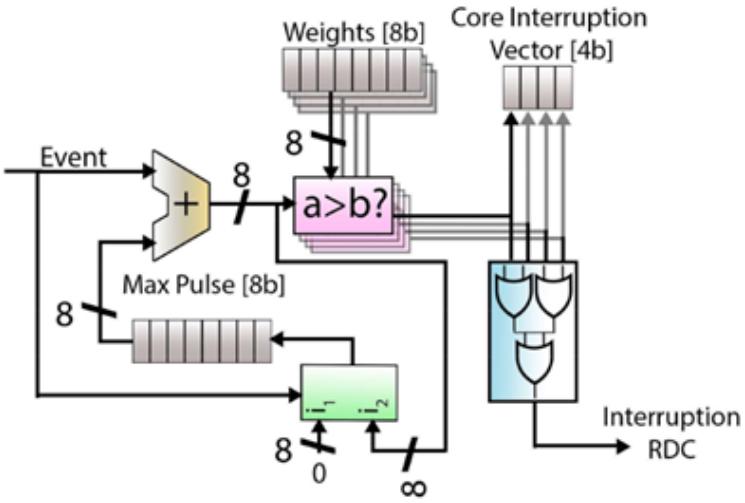


Figure 3.41: SafeSU - Block diagram of the RDC mechanism

Reserved		Core 3	Core 2	Core 1	Core 0
31 – 4		3	2	1	0
x		00	00	00	00

Figure 3.42: SafeSU - RDC Interrupt Vector (0x0A0)

Input 3	Input 2	Input 1	Input 0
31 – 24	23 – 16	15 – 8	7 – 0
00	00	00	00

Figure 3.43: SafeSU - RDC Event Weights Registers 0 and 1 (shared with MCCU; 0x098, 0x09C)

Input 7	Input 6	Input 5	Input 4
31 – 24	23 – 16	15 – 8	7 – 0
00	00	00	00

Figure 3.44: SafeSU - RDC Watermark Registers 0 and 1 (0x0A4, 0x0A8)

integration into the platform. The hypervisor (XtratuM Next Generation, also known as XNG, in the particular case of the SafeSU integration in ISOLDE) is in charge of managing privileges, allowing only specific partitions to be updated, in accordance with SafeSU's registers.

The preferred configuration consists of allowing only a single partition to modify SafeSU's configu-

ration registers and read SafeSU's counters, whereas the other partitions would not be allowed to access those registers. XNG guarantees this behavior, building on the MMU existing in the NOEL-V cores, which also realizes the RISC-V ISA hypervisor extension. Overall, the XNG hypervisor provides space isolation for the SafeSU's registers, hence achieving freedom from interference. This is in line with safety standards guidelines for items with integrity requirements.

In order to allow a partition to manage the SafeSU device from a high-level perspective, a driver at the hypervisor level has been developed and integrated by FENTISS within XNG, implementing a set of hypercalls. The new hypercalls defined to manage the SafeSU device are specified in Table 3.48, in which a description is included, and can be used by a partition with SafeSU access permissions.

For defining if a partition has access to the SafeSU device, a new attribute in the XtratuM Configuration Files (XCF) has been added. In the case that the "safesu" attribute is present in the XCF of a partition, that partition would be allowed to access SafeSU registers and the implemented hypercalls could be freely used. Otherwise, if the attribute is not set, if the partition tries to use any of the SafeSU hypercalls, an error code meaning invalid configuration will be returned.

Regarding interrupt management, all the interrupts generated by the SafeSU device need to be delegated to the partition in the XCF, so the partition can receive them. It is also important to note that the partition should allow the interrupts reception, unmask the desired interrupts, and install the desired handlers to be triggered when an interrupt is received. The management of these interrupts can be done through the use of other XNG's hypercalls.

To ensure the reliability of this software interface, a dedicated validation testsuite was developed and executed. The tests verified the full functionality of the driver within the XNG hypervisor environment, including the correct execution of all defined hypercalls to configure and monitor the SafeSU registers. Furthermore, the privilege mechanism was rigorously validated by simulating unauthorized access attempts from non-privileged partitions, confirming that the hypervisor successfully blocks them and returns the expected error codes. Finally, the end-to-end interrupt management—from the SafeSU hardware generation to the correct routing and triggering of the authorized partition's handler—was successfully tested. Also, it is important to remark that an overhead was observed between the generation of the maximum quota reached interrupt and the execution of its associated handler. This overhead is an expected behavior caused by the necessary privilege mode transitions, as the CPU must first capture and manage the physical interrupt at the hypervisor level before transferring control to the partition's handler. Overall, the validation campaign confirms a fully operational SW/HW integration.

3.2.3.7 Evaluation Prototype

FPGA

The actual operation of the SafeSU is part of the automotive use case. Hence, in this section we evaluate the overheads caused by the SafeSU in terms of hardware resources and performance costs.

Regarding performance, the SafeSU has no impact since it is out of the critical path, hence not affecting computation per se. Moreover, it can be synthesized at the maximum operating frequency allowed by the FPGA used (Xilinx VCU118), so it has no impact on the operating frequency.

Hypercall	Description
`retCode XSafeSuAssignEvent2Counter( counter, eventId)`	Assigns a specific event to a specific counter.
`retCode XSafeSuCountersEnable()`	Enables all SafeSU counters.
`retCode XSafeSuCountersDisable()`	Disables all SafeSU counters.
`retCode XSafeSuCountersReset()`	Resets all SafeSU counters.
`retCode XSafeSuGetCounterVal( counter, *value)`	Gets the current value of a specific counter.
`retCode XSafeSuRdcEnable()`	Enables the monitoring of the events' duration.
`retCode XSafeSuRdcDisable()`	Disables the monitoring of the events' duration.
`retCode XSafeSuRdcReset()`	Resets the status of the RDC.
`retCode XSafeSuRdcSetMaxDuration( counter, duration)`	Sets the maximum allowed duration for a given counter (compatible with RDC monitoring).
`retCode XSafeSuRdcReadWatermark( counter, *value)`	Reads the current maximum value for a given counter.
`retCode XSafeSuMccuEnable()`	Enables contention control.
`retCode XSafeSuMccuDisable()`	Disables contention control.
`retCode XSafeSuMccuReset()`	Resets the status of the MCCU.
`retCode XSafeSuMccuSetQuotaLimit( coreId, quota)`	Sets the maximum contention quota for a specific core.
`retCode XSafeSuMccuGetQuotaRemaining( coreId, *value)`	Gets the remaining contention quota for a specific core.
`retCode XSafeSuMccuSetEventWeights( counter, weight)`	Assigns weights to MCCU events for a given counter.
`retCode XSafeSuOverflowEnable()`	Enables counters overflow detection.
`retCode XSafeSuOverflowDisable()`	Disables counters overflow detection.
`retCode XSafeSuOverflowReset()`	Resets the status of the overflow submodule.
`retCode XSafeSuOverflowSetIrqMask( mask)`	Configures the overflow interrupt enable mask, enabling the overflow detection for the counters set in the mask.
`retCode XSafeSuOverflowGetIrqStatus( *mask)`	Gets the status of the active interrupts in the overflow submodule.

Table 3.48: Hypercall table implemented within XNG for SafeSU device support

In terms of resources, the SafeSU requires limited FPGA resources in absolute terms, as shown in Table 3.49. In particular, the highest overheads relate to the number of LUTs (7,386) and registers (3,155). In order to put these figures in context, we compare them against the total resources available in the FPGA, namely a Xilinx VCU118 device (see column "% vs FPGA" in the table), as well as against the total resources used by the SELENE SoC where the SafeSU is integrated (see column "% vs SoC" in the table). Focusing on the main resources, we observe

Component	FPGA	SELENE SoC	SafeSU	% vs FPGA	% vs SoC
LUTs	1,182,240	384,211	7,386	0.62%	1.92%
Registers	2,364,480	170,645	3,155	0.13%	1.85%
CARRY8	147,780	5,598	309	0.21%	5.52%
F7 Muxes	591,120	12,712	288	0.05%	2.27%
F8 Muxes	295,560	2,546	144	0.05%	5.66%
CLB	147,780	71,021	1,370	0.93%	1.93%
LUT as Logic	1,182,240	380,104	7,370	0.62%	1.94%
LUT as Memory	591,840	4,107	16	0.00%	0.39%
Block RAM Tile	2,160	308	0	0.00%	0.00%

Table 3.49: SafeSU - Area overheads

that the SafeSU uses only 0.62% of the LUTs available in the FPGA and 1.92% compared to the SELENE SoC. Regarding registers, those figures are 0.13% and 1.85% respectively.

Note that the size of the SafeSU scales with the number of events monitored and the number of counters used. Hence, using larger SoCs would not have any impact on the resources used by the SafeSU. Instead, its cost would only grow if we wanted to monitor more than 24 different events using more than the 24 counters included in the version integrated in the SELENE SoC.

3.2.3.8 State-of-the-Art Comparison

Arm

Arm's Memory System Resource Partitioning and Monitoring (MPAM) [5] is the main (and only) relevant component against which the SafeSU can be compared. Given the proprietary nature of the MPAM component, we cannot compare their overheads and hence, the only statement we can do in this direction is that SafeSU overheads are very low as shown before. Hence, although a comparison between MPAM and SafeSU could show that SafeSU overheads are higher than those of MPAM in relative terms for some setups, SafeSU overheads in absolute terms are anyway small.

Regarding features, we perform in this section a qualitative comparison, which we detail in Table 3.50. The main takeaway messages is that our SafeSU compares very favorably due to the following main reasons:

1. The SafeSU monitors many more events and casuistics (a superset of those of the MPAM)
2. The SafeSU provides quota monitoring capabilities, whereas the MPAM does not.
3. The SafeSU has an open source and permissive (MIT) license, whereas the MPAM is proprietary.
4. The SafeSU can be deployed on any FPGA or ASIC unconstrainedly, whereas it is unclear if MPAM can be integrated with non-Arm SoCs.

RISC-V

The only relevant component with a similar functionality to SafeSU in the context of RISC-V is the CA-PMC/CA-PMC-IF presented before in this document. For a comparison between both components, the CA-PMC/CA-PMC-IF and the SafeSU, we refer the reader to Section 3.1.6.8 where such comparison is already provided.

Feature	MPAM	SafeSU
Can monitor individual events	YES	YES
Can monitor combinations of events	Depends. It can monitor individual events in a per-partition basis. Hence, combinations of an event for a given partition can be monitored.	YES. Any combination of the events routed to the SafeSU.
Can monitor any type of event on its own	NO. Requires support in other components (cores, caches, etc.) to monitor their events.	YES. Relevant events only need to be routed to the SafeSU.
Can track event quotas	NO	YES. For any individual event or combination of events.
Can trigger interrupts	YES. Upon an error or a counter overflow.	YES. Upon a quota violation or a counter overflow. Interrupts are fully programmable.
Provides partitioning capabilities on its own	Partially. Partitioning is provided as long as the relevant components (cores, caches, etc.) also provide specific support.	NO. However, multicore interference can be limited building on SafeSU's quotas, hence compensating for the lack of explicit partitioning capabilities.
Programmability occurs accessing a specific address range	YES	YES
Open-source permissive license.	NO. Proprietary.	YES. MIT license. Others can be offered.
Can be integrated in any device.	NO. Only AMBA interfaces. Unclear whether other interfaces are possible.	YES. AMBA AHB and AXI already supported. Only a wrapper needs to be changed to support any other interface.

Table 3.50: SafeSU - Comparison against MPAM

3.2.3.9 Requirement Compliance

Table 3.51 provides a comparison between the obtained results and the corresponding IP requirements from WP1. The table summarizes the verification outcomes for each requirement and indicates the achieved level of compliance.

Table 3.51: Requirement Compliance Matrix SafeSU

REQ-DOC	REQ-ID	REQ-TYPE	REQ-Description	Verification Method	Activities / Justification	Compliance (C / PC / NC)
D1.4	SafeSU-01	F-HW_ARCH-STD	Support for AMBA	Statistics analysis	AHB and AXI statistics for different types of traffic traversing the corresponding AHB or AXI interface are accurately captured (e.g., by means of simulation). Such traffic can be injected with the SafeTI in T3.1	C
D1.4	SafeSU-02	F-HW_ARCH-STD	Traffic quotas can be set for specific devices	Module programming and statistics analysis	Quotas can be programmed in the SafeSU, properly monitored, and upon exhaustion, the corresponding interrupt raised (e.g., observing the SafeSU and interconnect behavior during simulation)	C
D1.4	SafeSU-03	NF-HW_UARCH-PR	Quota violation is reported within 1,000 clock cycles to the system software	Interrupt observation	No more than 1,000 cycles elapse since the time the quota is exhausted until the software interrupt routine starts its execution	C
D1.4	XNG-0190	F-SW_RT-FEA	XNG should allow the configuration of the SafeSU IP to the users	Testing with dedicated benchmarks	Different configuration options tested by using XNG hypercalls; SafeSU IP verified to set up accordingly	C
D1.4	XNG-0180	F–SW_RT-FEA	XNG should provide services to get the desired statistical information provided by SafeSU	Testing	XNG dedicated hypercalls used to retrieve different statistical information	C

3.2.4 Time Contract Monitoring Co-Processor (TCCP) – OFFIS

3.2.4.1 IP Card

Basic Info		
IP name	Time Contract Monitoring Co-Processor (TCCP)	
License	Apache V2.0	
Repository	https://github.com/offis/isolde-tccp	
Architecture		
Clock	Number of clock domains	2(+)
	Synchronous with system	N
	Clock generated internally	N
Ctrl Interface	ISA extension?	N
	Memory mapped?	Y
	Protocol	(AXI)
	Address Map	N.A.
Initiator Interface	Protocol	(AXI)
	Cached?	N
	IOMMU?	N
Interrupts	Interrupts	(Y) could generate interrupts if necessary
Microarchitecture		
Parametrization	Parametric no. cores?	N/A
	Parameteric config?	Y
Programmability	Contains programmable cores?	N
	ISA	-
Software		
Compiler	Requires specialized compiler?	Y
	Compiler repository	WP4
Hardware Abstraction Layer	N/A	
High-level API	Is there a high-level API/SDK?	N
	SDK repository	-
	Is there a domain-specific compiler?	Y
Integration		
IP Distribution	Manifest type (if any)	-
	Standalone simulation?	Y
	(if standalone sim) SW requirements?	N
	Integration documented / examples?	(Y)
Synthesis	Is the IP synthesizable?	N
	FPGA synthesis example available?	N
	ASIC synthesis example available?	N
Simulation	Closed-source simulation?	N
	Open-source simulation?	Y (SystemC TCCP simulation)
Evaluation	PPA results available?	Y

3.2.4.2 General Information

This module serves as a modular and composable time contract monitoring co-processor. The monitoring approach builds upon previous work from the VE-VIDES project [32, 21], a German-funded initiative, and extends earlier research [79] conducted in EU-funded projects such as Pro-

ductive4.0. Parts of the following content were already published as part of the ISOLDE publication strategy under [52] or will be published [51].

This reconfigurable co-processor is designed to support a formal Contract-Based Design (CBD) language [71]. It must handle fundamental timing properties, including aging, event occurrence, and reaction. Additionally, it should facilitate the monitoring of non-temporal properties, such as power consumption or ensuring that a specific controller maintains a parameter within a defined range over a given time frame. Examples include running closed-loop motor control in parallel with a computationally intensive task or verifying that a complex AI algorithm consistently completes its execution within a specified period.

Moreover, TCCP must be capable of monitoring multiple properties simultaneously. The functional and performance requirements of TCCP are detailed in Deliverable D1.2 (Section 3.3.2).

As shown in Figure 3.45, the contract-based runtime monitoring approach consists of three interacting components: At its core (1), the Time Contract Co-Processor (TCCP) connects to two other types of components, the TCCP-Compiler (2) and different observer interfaces (3). The TCCP executes contract-based specifications in hardware, it observes various event sources via an observer interface. The observers are minimalistic adapters to source data, like a RISC-V trace port to observe computational progress or a memory content observer. The TCCP monitors events according to its programmable configuration derived from contract specifications. These specifications are processed by the TCCP compiler, which generates a configuration program for the TCCP. The TCCP compiler is build as part of WP4 in Task T4.3, while the target architecture is part of the automotive demonstrator in WP5, Task T5.2.

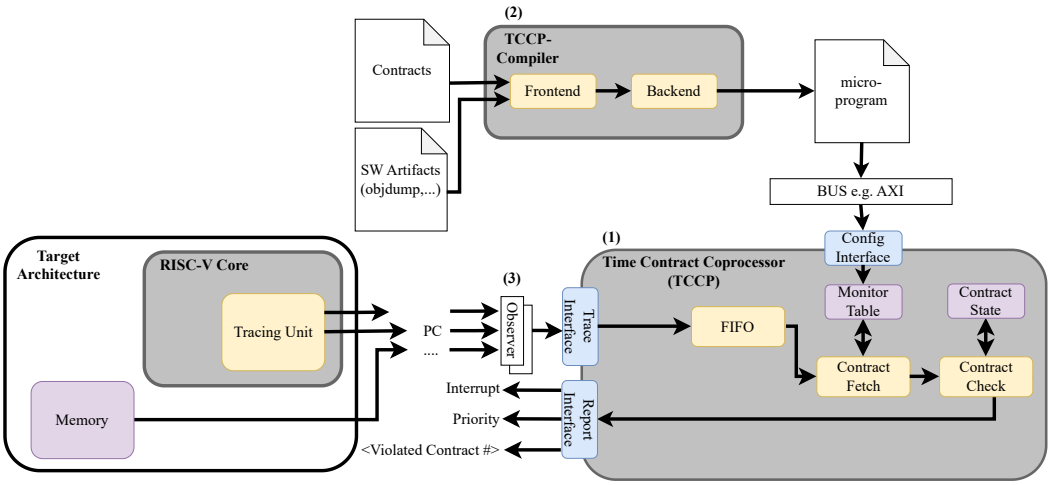


Figure 3.45: Time Contract Co-Processor (TCCP).

3.2.4.3 Purpose and Scope

The purpose of the TCCP is to advance the RISC-V ecosystem by developing high-performance architectures that incorporate the safety concept of contract-based monitoring. It is designed to keep the interference on the processor itself to a minimum. The scope includes multiple monitors for various properties, each with multiple interfaces, all operating simultaneously.

3.2.4.4 Refined architecture description

The refined architecture of the Monitoring Unit in Figure 3.46, illustrates the configuration of the TCCP and the processing flow from the trace interfaces to the report interface. These modules may operate in different clock domains, indicated by color coding: light blue for the monitor clock domain and orange for each of the observer clock domains.

Compared to the architecture described in Deliverable D3.2 in section 3.2.4, the design has been refined by introducing decentralized watchdogs as the key architectural change, while the contract fetch mechanism has been further optimized. Furthermore, instead of a single asynchronous FIFO buffering all incoming traces, the new architecture implements a separate FIFO for each trace interface, which allows to observe and monitor properties in different clock domains in parallel. A FIFO demultiplexer has been introduced to order incoming events chronologically based on their timestamps and to ensure that the watchdogs are triggered at least with a fixed, regular cycle interval to maintain a minimum monitoring frequency. Additionally, a comprehensive test suite has been introduced to replace the minimal simulation testbench, supporting multiple trace configurations and use cases.

The TCCP is configured by an DP-RAM inheriting each contract. Before active monitoring, each of the `N` contracts is fetched into one watchdog.

The trace interfaces collect incoming streams of traces from observers, each of which monitors different properties at various points in the architecture. For each trace interface exists one FIFO. The trace interface transmits an event to the FIFO, consisting of an observer ID, location, data value, and timestamp. The observer ID identifies the monitored property, the location points to a program counter or other observed data, and the timestamp is generated using the TCCP clock. The FIFO buffers incoming events from one trace sink. It has an beforehand adjustable size and reports any FIFO overruns to the report interface. Each FIFO outputs the oldest incoming event in the queue until it gets selected by the FIFO demultiplexer. The demultiplexer compares the timestamp of each incoming trace and selects the oldest by signaling the selection with an outgoing flag to the FIFO, which can therefore pop the oldest event. In addition to ordering and accessing the FIFO traces, the module ensures that the watchdogs are triggered at least every `Y` cycles.

The watchdogs are triggered through the demultiplexer, either by one specific trace or by a general trigger; every `Y` cycles to ensure a minimum monitoring frequency. Then, based on its current status, the watchdog checks if it needs to be activated or checked. If a contract is violated, the violation will be reported through the report interface, including the specific contract identification.

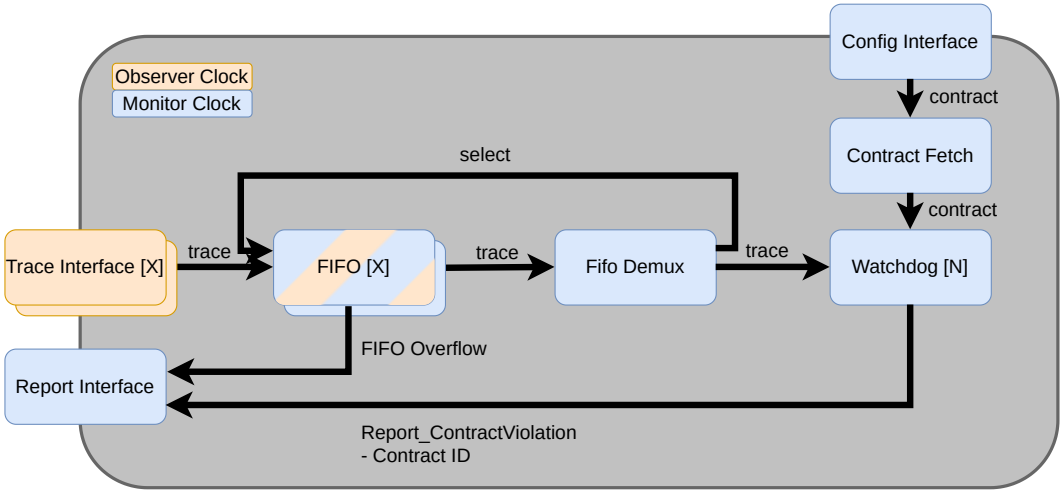


Figure 3.46: Refined Architecture of Time Contract Co-Processor (TCCP).

3.2.4.5 Interfaces

As illustrated in Figure 3.45, the TCCP features three interfaces: a Configuration Interface (Config Interface), a Trace Interface, and a Report Interface.

The Config Interface is used to transmit the monitoring information, derived from requirements and specifications, to the co-processor.

The Trace Interface connects the TCCP to the targeted modules through multiple Observers. Each Observer directly connects to gather information about a specific property, such as the current program counter or the power usage of a component. One evaluated interface is the ITI (Instruction Trace Interface) [77].

The Report Interface collects information about violated contracts or monitors and provides details about the violated contract, including its possible priority. It can also optionally trigger an interrupt.

3.2.4.6 Evaluation prototype

The evaluation concept is divided into three different stages.

SystemC Simulation

In the initial stage, SystemC is used to test the system using benchmarks derived from specific scenarios, simulating the system's behavior under various conditions to ensure it meets the required specifications and behaves as intended. The testbenches for the Instruction Trace Interface (ITI) and the Trace Ingress Port (TIP) utilize instruction and trace logs from target processors. The camera testbench emulates a driver observation brightness sensor in an automotive use case, outputting digital lux values to evaluate the IP based on the automotive use case in Task T5.2. Finally, the Autonomous Mobile Robot (AMR) testbench co-simulates a radar-based SLAM pipeline

based on execution traces from a Jetson board and a real-time motor controller with probabilistic fault injection [51].

SystemVerilog Simulation

The second stage focuses on developing a custom testbench for the SystemVerilog simulation environment with Verilator. This testbench is designed to verify the functionality of the system within a minimal simulation environment.

FPGA

The last stage involves testing two distinct scenarios to evaluate the system's capabilities in real-world applications. The first scenario tests the capabilities of the TCCP with the Instruction Trace Interface on a CVA6 architecture, developed by SYSGO and UNIBO. The second scenario tests the capabilities within the automotive demo use case of Task T5.2. Here, the TCCP is integrated into a NOEL-V architecture and coupled with a custom interface to an infrared LED supervisor.

Results

The SystemC simulation allowed to successfully evaluate the functionality of the design using the complete test suite. The offset in cycles between a staged event and a processed event in the watchdog can be calculated with $C_{emptyStack} = \lceil \frac{f_{TCCPclk}}{f_{Observerclk}} \rceil + 3$ for an empty FIFO. For multiple filled FIFOs, the offset is given by $C_{fullStack} = \lceil \frac{f_{TCCPclk}}{f_{Observerclk}} \rceil + FifoCount \cdot FifoQueueSize + 2$.

3.2.4.7 State-of-the-Art Comparison

The TCCP is compared qualitatively against state-of-the-art runtime monitoring mechanisms available in both the Arm and RISC-V ecosystems. A quantitative comparison against runtime monitoring co-processors from Arm or RISC-V ecosystems is not feasible at this stage, as no directly equivalent contract-based hardware monitoring co-processor exists in either ecosystem that would allow for a like-for-like benchmarking of area, latency, or throughput.

Arm ecosystem

In the Arm domain, **CoreSight** [2] provides a mature, standardized debug and trace infrastructure. However, CoreSight is primarily designed for debugging or tracing purposes, where monitoring means trace extraction and external delivery. The actual analysis of these traces must be performed in software, which, in contrast to the hardware-based TCCP, is intrusive and introduces computational overhead on the target processor. Furthermore, the CoreSight architecture is proprietary.

RISC-V ecosystem

Within the RISC-V ecosystem, several monitoring approaches have been proposed:

- **PHMon (Programmable Hardware Monitor)** [31]: PHMon is directly coupled to the CPU pipeline. Its primary focus is on security use cases such as shadow stacks, fuzzing, information leakage detection, and debugging, rather than real-time timing or behavioral contract monitoring.
- **Timing Instructions by Nanjundaswamy et al. (2023)** [54]: This approach extends the RISC-V ISA with custom instructions to support time-triggered architectures. However, the

timing evaluation is performed in software, making the monitoring approach intrusive and introducing runtime overhead on the main processor.

- **IMS (Intelligent Hardware Monitoring System)** [33]: IMS monitors system behavior at the bus level by observing the AXI interconnect using machine learning. It focuses on security rather than safety-critical timing and behavioral contracts, and uses a probabilistic classifier instead of deterministic evaluation.
- **Predecessor Project (VE-VIDES)** [32, 21]: The predecessor monitoring architecture is implemented as an on-chip solution within a Trusted Execution Environment (TEE) and has been demonstrated monitoring a RISC-V core. Both TCCP and this predecessor generate monitors from formal specifications using contract-based design, support runtime updates, and monitor both software and hardware domains. However, the predecessor is centralized and focuses primarily on detecting security threats, whereas the TCCP is decentralized, making it better suited for monitoring distributed systems. Additionally, the TCCP prioritizes low resource usage and high power efficiency under a completely different target architecture.

3.2.4.8 Requirement Compliance

Table 3.52 provides a comparison between the obtained results and and the corresponding IP requirements and performance targets from WP1. The table summarizes the verification outcomes for each requirement and indicates the achieved level of compliance.

REQ-DOC	REQ-ID	REQ-TYPE	REQ-Description	Verification Method	Activities / Justification	Compliance (C / PC / NC)
D1.4	TCCP-01	F-HW-ARCH-PR	The TCCP should be able to monitor different properties simultaneously	RTL testbench	REQ met, but verified in system integration and simulation instead of a RTL testbench.	C
D1.4	TCCP-02	F-HW-ARCH-FEA	The TCCP should be able to store and update timing specification	RTL testbench & system integration	REQ met, but verified in system integration and simulation instead of a RTL testbench.	C
D1.4	TCCP-03	F-HW-ARCH-FEA	The TCCP should be able to process data captured by safety infrastructure	System integration	Verified with ITI instead of Safety-Island infrastructure, since Safety-Island infrastructur was removed. TCCP was able to process the ITI output, captured through TCCP-ITI-Observer.	PC

Continued on the next page

D1.4	TCCP-04	F-HW-ARCH-FEA	The results of TCCP should be available for action in HW and SW (as interrupt and MMIO)	RTL testbench & system integration	Verified in system integration and simulation.	C
D1.4	TCCP-05	NF-HW-UARCH-PR	Monitoring per sec. > (No. of events per message * input traffic rate)	System integration & Simulation	Verified in system integration and simulation.	C
D1.4	TCCP-06	NF-HW-ARCH-PR	Support monitoring at least 100 contract-expressions at full monitoring rate	System Integration & Simulation	Verified in system integration and simulation.	C
D1.4	Target Performance		Monitoring per sec. > No of events per message · input traffic rate	System integration & Simulation	Verified in system integration and simulation.	C
D1.4	Target Performance		100 contract-expressions at full monitoring rate, a design space exploration is planned	System integration & Simulation & RTL testbench	Target performance verified in system integration and simulation but not with RTL testbench. Design Space Exploration was not carried out.	PC

Table 3.52: Requirement Compliance Matrix TCCP

4 Conclusion

This deliverable presents completed IP implementations developed within Work Package 3 of the ISOLDE project. The implemented IPs cover safety and security related hardware modules and extensions that have been developed within WP3 (Accelerators and Extensions) of the ISOLDE project.

A central objective of this deliverable is to provide a consistent evaluation and positioning of the developed IPs with respect to the state of the art, enabling comparisons across the different architectures and application domains. Where applicable, the results are compared against implementations based on Arm and RISC-V processors, as well as against relevant results reported in the literature. Overall, the emphasis on performance on the average is a bit less than in D3.5 with its more heavy focus on accelerators, as safety and security solutions are often less heavily performance-bound.

In addition to performance evaluation, the compliance of the developed IPs with the requirements established in the preceding deliverables D1.3 and D1.4 is documented. The corresponding requirements are mapped to the implemented functionality and summarized in dedicated tables, providing a traceable assessment of the extent to which the developed IPs fulfill the previously defined technical and architectural objectives.

Together, the implementation results, state-of-the-art comparisons, and requirements-compliance analyses provide the consolidated technical assessment of the WP3 IP portfolio, paving the way to RISC-V-based safe & secure high-performance embedded computing systems.

Acronyms and Definitions

Acronym	Description
ACC-BIKE	ACCelerator for post-quantum key encapsulation mechanism BIKE
ADC	Analog-to-Digital Converter
AES	Advanced Encryption Standard
AHB	Advanced High-performance Bus
AI	Artificial Intelligence
ALU	Arithmetic Logic Unit
AMA	AI/ML Accelerator
AMBA	Advanced Microcontroller Bus Architecture
APB	Advanced Peripheral Bus
ASCON	Lightweight Authenticated Block Cipher
ASIC	Application-Specific Integrated Circuit
ASIP	Application-Specific Instruction Set Processor
ASLR	Address Space Layout Randomization
AXI	Advanced eXtensible Interface
AXI-MM	AXI Memory Mapped
AXIS	AXI Stream
BCFI	Backward-Edge Control Flow Integrity
BRAM	Block RAM
BS	Base Station
CA-PMC	Context-Aware Performance Monitor Counter
CA-PMC-IF	Context-Aware PMC Interface
CBD	Contract Based Design
CCS	Contention Cycles Stack
CE	Computing Element
CFI	Control Flow Integrity
CNN	Convolutional Neural Network
CORDIC	Coordinate Rotation Digital Computer
COP	Call-Oriented Programming
CPU	Central Processing Unit
CPS	Cyber-Physical Systems
CSR	Control and Status Register
CTM	Cryptographically Tagged Memory
CV-X-IF	Core-V eXtension Interface
DBB	Digital Base Band
DDR	Double Data Rate Synchronous Dynamic Random Access Memory
DES	Data Encryption Standard
DFT	Discrete Fourier Transform
DFU	Decoder Functional Units

DMA	Direct Memory Access
DMR	Dual Modular Redundancy
DSP	Digital Signal Processor
DSS	Digital Signature Schemes
DVS	Dynamic Vision Sensor
ECC	Error Correction Code
EMI	Enclave Memory Isolation
ECNNA	Event-based CNN Accelerator
EXP	EXtension Platform
FCFI	Forward-edge Control Flow Integrity
FFT	Fast Fourier Transform
FP	Floating Point
FPGA	Field Programmable Gate Array
FSM	Finite State Machine
FIFO	First-In-First-Out
FIR	Finite Impulse Response
FMA	Fused-Multiply-Add
FPMIX	FPU for MIXed-precision computing
FPU	Floating Point Unit
GE	Gate Equivalent
GEMM	GEneral Matrix Multiply
GPIO	General Purpose Input/Output
HARQ	Hybrid Automatic Repeat Request
HCI	Heterogeneous Cluster Interconnect
HDK	Hardware Development Kit
HLS	High Level Synthesis
HLS-PQC	HLS-based Post-Quantum Cryptographic accelerator
HMAC	Hash-based Message Authentication Code
IEE	Inline Encryption Engine
IEE-RV	Inline Encryption Engine RISC-V ISA extension
INET	Interconnection NETwork
INTT	Inverse Number Theoretic Transform
IP	Intellectual Property
ISA	Instruction Set Architecture
ISE	Instruction Set Extension
IUHF	Inverse Universal Hash Function
JOP	Jump-Oriented Programming
KEM	Key Encapsulation Mechanism
KMAC	KECCAK Message Authentication Code
LDPC	Low Density Parity Check Decoder
LIF	Leaky Integrate and Fire (neuron model)
LSW	Least Significant Word
LLR	Log Likelihood Ratio
M	Machine Mode

MAC	Multiply-Accumulate
MC	Memory Controller
MCCU	Maximum Contention Control Unit
MDPC	Moderate-Density Parity-Check
ML	Machine Learning
ML-DSA	Module-Lattice-based – Digital Signature Standard
ML-KEM	Module-Lattice-based – Key Encapsulation Mechanism
MMIO	Memory Mapped Input/Output
MMU	Memory Management Unit
MPSoC	Multiprocessor System on a Chip
MSW	Most Significant Word
NIST	National Institute of Standards and Technology
NoC	Network on Chip
NR	New Radio
NTT	Number Theoretic Transform
ONNX	Open Neural Network eXchange
OVI	Open Vector Interface
PC	Program Counter
PCA	Parallel Computing Accelerator
PE	Processing Engine
PMP	Physical Memory Protection
PMU	Performance Monitor Unit
POR	Power-On Reset
PPA	Power, Performance, and Area
PQC	Post-Quantum Cryptography
PQC-MA	Post-Quantum Crypto Accelerator
PRF	Polymorphic Register File
PRINCE	Low-latency block cipher
PRNG	Pseudorandom Number Generator
QC	Quasi-Cyclic
QARMAv2	Lightweight tweakable block cipher
RDC	Request Duration Counter
ReO	Rectangle Only
ReRo	Rectangle Row
ReTr	Rectangle Transposed
RF	Radio Frequency
RFOG	Register File Organization Table
RoCo	Row Column
ROM	Read-Only-Memory
ROP	Return Oriented Programming
RoT	Root-of-Trust
RSA	Rivest–Shamir–Adleman
RTL	Register Transfer Level
RTPM	Run-Time Power Monitoring instrumentation

RV32	32-bit RISC-V processor model
RVV	RISC-V Vector extension
S	Supervisor Mode
SafeSU	Safety-related Statistics Unit
SafeTI	Safety-related Traffic Injector
SCA	Shared Correlation Accelerator
SCH	SCHeduler
SCMI	System Control and Management Interface
SDK	Software Development Kit
SDRAM	Synchronous Dynamic Random Access Memory
SEC	SECured RISC-V processor with cryptographic accelerators
SHA	Secure Hash Algorithms
SIMD	Single Instruction Multiple Data
SLDU	SLiDe Unit
SLH-DSA	Stateless Hash-Based Digital Signature Standard
SM	Security Monitor
SNN	Spiking Neutral Networks
SoA	State of the Art
SoC	System on a Chip
SPI	Serial Peripheral Interface
SRAM	Static Random-Access Memory
SSCU	Safety and Security Control Unit
TBS	Transport Block Sizes
TCCP	Time Contract monitoring Co-Processor
TCCP-CO	Time Contract monitoring Co-Processor Compiler
TCDM	Tightly-Coupled Data Memory
TI	Tweak Input
TLUL	TileLink Uncached Lightweight bus
TMR	Triple Modular Redundancy
TPU	Tensor Processing Unit
U	User Mode
UE	User Equipment
UHF	Universal Hash Function
IUHF	Inverse Universal Hash Function
VLSI	Very Large Scale Integration
VMFPU	Vector Multiplier and Floating-Point Unit
VPU	Vector Processing Unit
VRF	Vector Register File
WCET	Worst-Case Execution Time
XCF	XtratuM Configuration Files
XIF	eXtension InterFace
XNG	XtratuM Next Generation

Bibliography

[1] AMD (formerly Xilinx). *AXI Traffic Generator v3.0. LogiCORE IP Product Guide*. AMD, 2023.

[2] Arm. *Arm® CoreSight™ Architecture Performance Monitoring Unit Architecture*. Arm Limited.

[3] Arm. Where is the MTE Tag stored and checked (KA005620)? online. Version 1.0.

[4] Arm. *ARM Energy Probe*, 2011. Order Number AEP03-KT-0311A.

[5] Arm. *Arm Architecture Reference Manual Supplement. Memory System Resource Partitioning and Monitoring (MPAM), for Armv8-A*. Arm, 2018.

[6] Arm. Arm® Reliability, Availability, and Serviceability (RAS) Specification. online, July 2019.

[7] Arm. Armv8.1-M PACBTI Extensions. online, 2024. Version 1.0.

[8] Arm. Whitepaper: Armv8.5-A Memory Tagging Extension. online, 2024. Version 1.0.

[9] Arm. *Arm Architecture Reference Manual for A-profile architecture*. Arm, 2025.

[10] Arm. Arm Realm Management Extension (RME) System Architecture. online, December 2025. Version C.a.

[11] Arm. Learn the architecture - Introducing Arm Confidential Compute Architecture. online, February 2025. Version 4.0.

[12] Arm. Learn the architecture - Realm Management Extension. online, February 2025. Version 1.1.

[13] Arm. *Armv8-M Architecture Reference Manual*, ddi0553b.z edition, 2026.

[14] Arm Ltd. Jazelle DBX and ThumbEE (Jazelle RCT): Java acceleration in the Arm architecture, 2011. Jazelle DBX is not present in Armv8-A; ThumbEE was deprecated in Armv7 and removed in Armv8. See the Arm Architecture Reference Manual for the respective architecture versions.

[15] Roberto Avanzi, Subhadeep Banik, Orr Dunkelman, Maria Eichlseder, Shibam Ghosh, Marcel Nageler, and Francesco Regazzoni. The QARMAv2 family of tweakable block ciphers. *Cryptology ePrint Archive*, 2023.

[16] Yanis Belkheyar, Joan Daemen, Christoph Dobraunig, Santosh Ghosh, and Shahram Rasoolzadeh. Bipbip: A low-latency tweakable block cipher with small dimensions. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 326–368, 2023.

[17] Yanis Belkheyar, Patrick Derbez, Shibam Ghosh, Gregor Leander, Silvia Mella, Léo Perrin, Shahram Rasoolzadeh, Lukas Stennes, Siwei Sun, Gilles Van Assche, et al. Chilow and chichi: New constructions for code encryption. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 212–243. Springer, 2025.

[18] Marcus Borrmann, Erik Kraft, Marcel Medwed, Daniel Gracia Pérez, Sylvain Girbal, Stefan Wallentowitz, Holger Blasum, Darshak Sheladiya, Jaume Abella, Francisco Fuentes, Sergi Alcaide, Francisco J. Cazorla, Ramon Canal, Feng Chang, Juan Carlos Rodríguez, Juan Antonio Rodríguez, Xavier Carril, Esther Soriano, Wolfgang Cebrián, Daniel Ecker, Endri Kaja, Ares Tahiraga, Jad Al Halabi, Mladen Berekovic, Rainer Buchty, Amrit Sharma Poudel, Nguyen-Hoang-Hai Pham, Saleh Mulhem, Luigi Ghionda, Emanuele Parisi, Yvan Tortorella, Andrea Galimberti, D. Zoni, William Fornaciari, Alexandru Pușcașu, Cătălin Ciobanu, Mihai Gologanu, Sven Mehlhop, Jörg Walter, Mihai Munteanu, Honorius Galmeanu, Diego Gigena Ivanovich, Ambily Suresh, Andrew Wilson, Manuel Freiberger, Maurizio Martina, Gianvito Urgese, Matteo Perotti, Andrei Stan, Cristian-Tiberius Axinte, George Uleru, Michael Gautschi, Reza Ghanaatian, and Stefan Lippuner. Deliverable D3.2: Safety & Security modules prototype implementation. ISOLDE project deliverable, April 2025.

[19] Marcus Borrmann, Erik Kraft, Marcel Medwed, Daniel Gracia Pérez, Sylvain Girbal, Stefan Wallentowitz, Matthias Rupp, Holger Blasum, Jaume Abella, Francisco Fuentes, Sergi Alcaide, Francisco J. Cazorla, Ramon Canal, Feng Chang, Juan Carlos Rodríguez, Juan Antonio Rodríguez, Xavier Carril, Esther Soriano, Vicente Nicolau, Wolfgang Ecker, Endri Kaja, Jad Al Halabi, Mladen Berekovic, Christopher Blochwitz, Emanuele Parisi, Francesco Conti, Yvan Tortorella, Andrea Galimberti, Alexandru Pușcașu, Cătălin Ciobanu, Mihai Gologanu, Behnam Razi Perjikolaei, Mihai Munteanu, Honorius Galmeanu, Diego Gigena Ivanovich, Ambily Suresh, Andrew Wilson, Manuel Freiberger, Maurizio Martina, Gianvito Urgese, Matteo Perotti, Cristian-Tiberius Stan, Andrei amd Axinte, George Uleru, Mari-Anais Sachian, Kejsi Koci, Cristina Tudor, George Suciu, Michael Gautschi, Reza Ghanaatian, Stefan Lippuner, and Jan Kaštil. Deliverable D3.1: Initial Architecture Definition. ISOLDE project deliverable, May 2024.

[20] Dušan Božilov, Maria Eichlseder, Miroslav Knežević, Baptiste Lambin, Gregor Leander, Thorben Moos, Ventzislav Nikov, Shahram Rasoolzadeh, Yosuke Todo, and Friedrich Wiemer. PRINCEv2: more security for (almost) no overhead. In *International Conference on Selected Areas in Cryptography*, pages 483–511. Springer, 2020.

[21] Friederike Bruns, Georg Gläser, Florian Kögler, Jonas Lienke, Nithin Nanjundaswamy, Gregor Nitsche, Behnam Perjikolaei, and Jörg Walter. Trust is Good, Monitoring is Better: FPGA- & TEE-Based Monitoring for Malware-Detection. In *13th IMA International Conference on Modelling in Industrial Maintenance and Reliability*, 07 2025.

[22] Robert Buhren, Hans-Niklas Jacob, Thilo Krachenfels, and Jean-Pierre Seifert. One Glitch to Rule Them All: Fault Injection Attacks Against AMD's Secure Encrypted Virtualization. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, CCS '21, page 2875–2889, New York, NY, USA, 2021. Association for Computing Machinery.

[23] Federico Canale, Tim Güneysu, Gregor Leander, Jan Philipp Thoma, Yosuke Todo, and Rei Ueno. SCARF – a Low-Latency block cipher for secure Cache-Randomization. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 1937–1954, Anaheim, CA, August 2023. USENIX Association.

[24] Stephen Cass. Top Programming Languages 2022. online, August 2022.

[25] Thomas Chamelot, Damien Couroussé, and Karine Heydemann. SCI-FI: Control Signal, Code, and Control Flow Integrity against Fault Injection Attacks. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 556–559, 2022.

[26] Thomas Chamelot, Damien Couroussé, and Karine Heydemann. MAFIA: Protecting the Microarchitecture of Embedded Systems Against Fault Injection Attacks. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 42(12):4555–4568, 2023.

[27] Catalin Bogdan Ciobanu et al. Forging Robust Building Blocks for Next-Gen RISC-V Chips: Virtual Repository and IP Card. In *29th Euromicro Conference Series on Digital System Design (DSD)*, 2026.

[28] André Cirne, Patrícia R. Sousa, João S. Resende, and Luís Antunes. RunPBA – Runtime attestation for microcontrollers with PACBTI, 2025.

[29] Francesco Cosimi, Fabrizio Tronci, Sergio Saponara, and Paolo Gai. Analysis, Hardware Specification and Design of a Programmable Performance Monitoring Unit (PPMU) for RISC-V ECUs. In *2022 IEEE International Conference on Smart Computing (SMARTCOMP)*, pages 213–218, 2022.

[30] Luca Cremona, William Fornaciari, and Davide Zoni. Automatic identification and hardware implementation of a resource-constrained power model for embedded systems. *Sustainable Computing: Informatics and Systems*, 29:100467, 2021.

[31] Leila Delshadtehrani, Sadullah Canakci, Boyou Zhou, Schuyler Eldridge, Ajay Joshi, and Manuel Egele. PHMon: A Programmable Hardware Monitor and Its Security Use Cases. In *USENIX Security '20*, pages 807–824, 2020.

[32] Elektronik- und Mikrosysteme VDI/VDE Innovation + Technik GmbH. VE-VIDES Project. https://www.elektronikforschung.de/projekte/ve-vides, 2024. Accessed: 2025-03-31.

[33] Wadid Foudhaily, Aykut Rencber, Anouar Nechi, Rainer Buchty, Mladen Berekovic, Andres Gomez, and Saleh Mulhem. IMS: Intelligent Hardware Monitoring System for Secure SoCs, January 2026.

[34] César Fuguet. HPDcache: Open-Source High-Performance L1 Data Cache for RISC-V Cores. In *Proceedings of the 20th ACM International Conference on Computing Frontiers*, pages 377–378, Bologna, Italy, 2023. ACM.

[35] Simon Gerhalter, Samir Hodžić, Marcel Medwed, Marcel Nageler, Artur Folwarczny, Ventzi Nikov, Jan Hoogerbrugge, Tobias Schneider, Gary McConville, and Maria Eichlseder. BEANIE–A 32-bit Cipher for Cryptographic Mitigations Against Software Attacks. *IACR Transactions on Symmetric Cryptology*, 2025(4):31–69, 2025.

[36] Nicolas Gerlin, Endri Kaja, Fabian Vargas, Li Lu, Anselm Breitenreiter, Junchao Chen, Markus Ulbricht, Maribel Gomez, Ares Tahiraga, Sebastian Prebeck, Eyck Jentzsch, Miloš Krstić, and Wolfgang Ecker. Bits, Flips and RISCs. In *2023 26th International Symposium on Design and Diagnostics of Electronic Circuits and Systems (DDECS)*, pages 140–149, 2023.

[37] Sylvain Girbal, Jimmy Le Rhun, and Hadi Saoud. METrICS: a Measurement Environment for Multi-Core Time Critical Systems. In *Embedded Real Time Software and Systems*, ERTS '18, 2018.

[38] Sylvain Girbal, Jimmy Le Rhun, Daniel Gracia Pérez, and David Faura. Safety & Security monitoring convergence at the dawn of Open Hardware and Artificial Intelligence, June 2022.

[39] Google Project Zero. 0day "In the Wild" 2024. online, January 2025.

[40] Théophile Gousselot, Jean-Max Dutertre, Olivier Potin, and Jean-Baptiste Rigaud. Code Encryption for Confidentiality and Execution Integrity Down to Control Signals. In *2025 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, pages 403–414, 2025.

[41] Miltos D Grammatikakis, Stelios Ninidakis, George Kornaros, Dimitrios Bakoyiannis, Nikos Mouzakitis, and Alexis Staridas. Managing concurrent queues for efficient in-vehicle gateways. *J. Commun.*, 18(5):333–339, 2023.

[42] Yuko Hara, Hiroyuki Tomiyama, Shinya Honda, Hiroaki Takada, and Katsuya Ishii. Chstone: A benchmark program suite for practical c-based high-level synthesis. In *2008 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1192–1195, 2008.

[43] John L Hennessy and David A Patterson. Computer organization and design RISC-V edition: The hardware software interface, 2017.

[44] Intel Corporation. *Intel® 64 and IA-32 Architectures Software Developer's Manual Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4*. Intel Corporation, Santa Clara, CA, February 2026. Order Number: 325462, Version 090.

[45] Endri Kaja, Nicolas Gerlin, Monideep Bora, Gabriel Rutsch, Keerthikumara Devarajegowda, Dominik Stoffel, Wolfgang Kunz, and Wolfgang Ecker. Fast and accurate model-driven fpga-based system-level fault emulation. In *2022 IFIP/IEEE 30th International Conference on Very Large Scale Integration (VLSI-SoC)*, pages 1–6, 2022.

[46] Jinyeol Kim, Raehyeong Kim, Jongwon Oh, and Seung Eun Lee. Hardware-based WebAssembly accelerator for embedded system. *Electronics*, 13(20):3979, 2024.

[47] Yoongu Kim, Ross Daly, Jeremie Kim, Chris Fallin, Ji Hye Lee, Donghyuk Lee, Chris Wilkerson, Konrad Lai, and Onur Mutlu. Flipping bits in memory without accessing them: an experimental study of DRAM disturbance errors. *SIGARCH Comput. Archit. News*, 42(3):361–372, jun 2014.

[48] Gaëtan Leplus, Olivier Savry, and Lilian Bossuet. Secdec: Secure decode stage thanks to masking of instructions with the generated signals. In *2022 25th Euromicro Conference on Digital System Design (DSD)*, pages 556–563. IEEE, 2022.

[49] Hans Liljestrand, Thomas Nyman, Lachlan J Gunn, Jan-Erik Ekberg, and N Asokan. {PACStack}: an authenticated call stack. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 357–374, 2021.

[50] Simone Manoni, Emanuele Parisi, Riccardo Tedeschi, Davide Rossi, Andrea Acquaviva, and Andrea Bartolini. CVA6-CFI: A First Glance at RISC-V Control-Flow Integrity Extensions, 2026.

[51] Sven Mehlhop, Frank Oppenheimer, Eike Furuno, Andreas Rauh, and Friederike Bruns. Contract-based runtime verification for safe autonomous mobile robots development. Submitted to: 2026 Forum on Specification & Design Languages (FDL). Under review, 2026.

[52] Sven Mehlhop, Jörg Walter, and Frank Oppenheimer. Open-Source Timing-Monitor Co-Processor in RISC-V Safety Infrastructure. In *2025 Forum on Specification & Design Languages (FDL)*, 2025.

[53] Lev Mukhanov, Pavlos Petoumenos, Zheng Wang, Nikos Parasyris, Dimitrios S. Nikolopoulos, Bronis R. De Supinski, and Hugh Leather. Alea: A fine-grained energy profiling tool. *ACM Trans. Archit. Code Optim.*, 14(1), March 2017.

[54] Nithin Ravani Nanjundaswamy, Gregor Nitsche, Frank Poppen, and Kim Grüttner. RISC-V Timing-Instructions for Open Time-Triggered Architectures. In *2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W)*, pages 211–214, June 2023. ISSN: 2325-6664.

[55] Pascal Nasahl, Robert Schilling, Mario Werner, Jan Hoogerbrugge, Marcel Medwed, and Stefan Mangard. CrypTag: Thwarting Physical and Logical Memory Vulnerabilities using Cryptographically Colored Memory. In *Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security*, ASIA CCS '21, page 200–212, New York, NY, USA, 2021. Association for Computing Machinery.

[56] Shoei Nashimoto, Daisuke Suzuki, Rei Ueno, and Naofumi Homma. Bypassing Isolated Execution on RISC-V using Side-Channel-Assisted Fault-Injection and Its Countermeasure. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022(1):28–68, Nov. 2021.

[57] Taehyun Noh, Yingchen Wang, Tal Garfinkel, Mahesh Madhav, Daniel Moghimi, Mattan Erez, and Shravan Narayan. ARM MTE Performance in Practice (Extended Version), 2026.

[58] Wolfgang Puffitsch and Martin Schoeberl. picoJava-II in an FPGA. In *Proceedings of the 5th International Workshop on Java Technologies for Real-Time and Embedded Systems (JTRES)*. ACM, 2007.

[59] RISC-V International. Composable custom extensions: Requirements. RISC-V Tech Hub, 2024.

[60] RISC-V International. RISC-V base cache management operation ISA extensions. Technical report, RISC-V International, 2024.

[61] RISC-V International. RISC-V Confidential VM Extension (CoVE) for Confidential Computing. online, August 2024. Version 0.7.

[62] RISC-V International. RISC-V RERI Architecture Specification. online, May 2024.

[63] RISC-V International. RISC-V Shadow Stacks and Landing Pads. online, July 2024.

[64] RISC-V International. RISC-V Memory Tagging Extension. online, December 2025. Version 0.2.0.

[65] RISC-V International. The RISC-V Instruction Set Manual: Volume I. online, May 2025. Version 20250508.

[66] RISC-V International. The RISC-V Instruction Set Manual: Volume II. online, May 2025. Version 20250508.

[67] RISC-V International. Memory Encryption Contexts TG Proposal of Work. online, February 2026.

[68] RISC-V International. RISC-V Supervisor Domains Access Protection. online, January 2026. Version 0.49.

[69] RISC-V Shadow-stack and Landing-pads Task Group. Shadow Stack and Landing Pads (0036ff2). Technical report, RISC-V International, 2024. Commit: 0036ff20a7305f6705ad630c63a9aa58da7df285.

[70] Cyril Roscian, Alexandre Sarafianos, Jean-Max Dutertre, and Assia Tria. Fault Model Analysis of Laser-Induced Faults in SRAM Memory Cells. In *2013 Workshop on Fault Diagnosis and Tolerance in Cryptography*, pages 89–98, 2013.

[71] Alberto Sangiovanni-Vincentelli, Werner Damm, and Roberto Passerone. Taming Dr. Frankenstein: Contract-Based Design for Cyber-Physical Systems. *European Journal of Control*, 18(3):217–238, 2012.

[72] Olivier Savry, Mustapha El-Majihi, and Thomas Hiscock. Confidaent: Control flow protection with instruction and data authenticated encryption. In *2020 23rd Euromicro Conference on Digital System Design (DSD)*, pages 246–253. IEEE, 2020.

[73] Martin Schoeberl. *JOP: A Java Optimized Processor for Embedded Real-Time Systems*. PhD thesis, Vienna University of Technology, 2005.

[74] Martin Schoeberl and Carsten Nielsen. A stack cache for real-time systems. In *Proceedings of the 19th IEEE International Symposium on Real-Time Distributed Computing (ISORC)*. IEEE, 2016.

[75] Robin Sehm, Christian Ewert, Rainer Buchty, Mladen Berekovic, and Saleh Mulhem. Nail: Not another fault-injection framework for chisel-generated rtl. In *2025 28th Euromicro Conference on Digital System Design (DSD)*, page 1–7. IEEE, Sept 2025.

[76] Kostya Serebryany, Evgenii Stepanov, and Vlad Tsyrklevich. Hardware Memory Tagging makes C++ Memory-Safer. Poster at 2018 Bay Area LLVM Developers' Meeting, Oktober 2018.

[77] Darshak Sheladiya, Guillaume Chauvon, and Jean-Roch Coulon. CVA6 Repository - Instruction Trace Interface. https://github.com/openhwgroup/cva6/commit/f314dcb136ed373db8332e30a897fa06db4aae43. Accessed: 2025-03-31.

[78] Texas Instruments. *INA231 28-V, 16-Bit, I2C Output Current, Voltage and Power Monitor With Alert in WCSP Package*, 2022. Data sheet SBOS644D.

[79] Duc Do Tran, Kim Grüttner, Frank Oppenheimer, and Wolfgang Nebel. Timing Contracts and Monitors for Safety Relevant Controller Design in IEC 61499. In *2020 25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, volume 1, pages 156–163, 2020.

[80] Andrew Waterman, Yunsup Lee, David A. Patterson, and Krste Asanovic. The RISC-V Instruction Set Manual, December 2019.

[81] Robert N. M. Watson and others. Capability Hardware Enhanced RISC Instructions: CHERI Instruction-Set Architecture (Version 9). Technical Report UCAM-CL-TR-987, University of Cambridge, Computer Laboratory, September 2023.

[82] Mario Werner, Thomas Unterluggauer, David Schaffenrath, and Stefan Mangard. Sponge-based control-flow protection for IoT devices. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 214–226. IEEE, 2018.

[83] Mario Werner, Erich Wenger, and Stefan Mangard. Protecting the Control Flow of Embedded Processors against Fault Attacks. In Naofumi Homma and Marcel Medwed, editors, *Smart Card Research and Advanced Applications*, pages 161–176, Cham, 2016. Springer International Publishing.

[84] F. Zaruba and L. Benini. The Cost of Application-Class Processing: Energy and Performance Analysis of a Linux-Ready 1.7-GHz 64-Bit RISC-V Core in 22-nm FDSOI Technology. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 27(11):2629–2640, Nov 2019.

[85] Davide Zoni, Luca Cremona, and William Fornaciari. Design of side-channel-resistant power monitors. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 41(5):1249–1263, 2022.

[86] Davide Zoni, Andrea Galimberti, and William Fornaciari. A survey on run-time power monitors at the edge. *ACM Comput. Surv.*, 55(14s), July 2023.